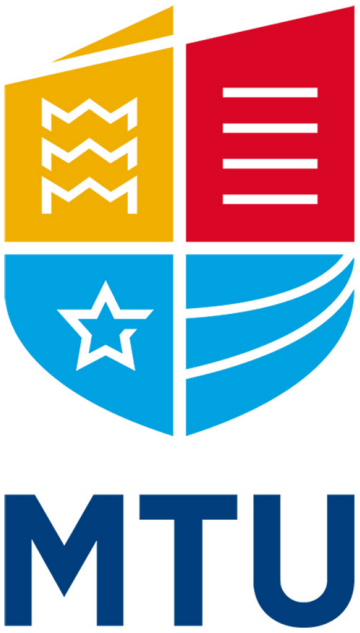

ELA: English Language Assistant

by

Vladyslav Rastvorov

This thesis has been submitted in partial fulfillment for the degree of Bachelor of Science in Computer Systems

in the
Faculty of Engineering and Science
Department of Computer Science

December 2025

Declaration of Authorship

This report, ELA: English Language Assistant, is submitted in partial fulfillment of the requirements of Bachelor of Science in Computer Systems at Munster Technological University Cork. I, Vladyslav Rastvorov, declare that this thesis represents substantially my own work except where indicated.

- This work was done wholly or mainly while in candidature Bachelor of Science in Computer Systems at Munster Technological University Cork.
- Where previously submitted work exists, this is clearly stated.
- Published work of others is clearly attributed.
- Quotations are properly cited; otherwise this is my own work.
- All main sources of help are acknowledged.
- In joint work, my contributions are specified.

Signed: _____

Date: _____

MUNSTER TECHNOLOGICAL UNIVERSITY CORK

Abstract

Faculty of Engineering and Science
Department of Computer Science

Bachelor of Science

by Vladyslav Rastvorov

MUNSTER TECHNOLOGICAL UNIVERSITY CORK

Abstract

Faculty of Engineering and Science
Department of Computer Science

Bachelor of Science

by Vladyslav Rastvorov

This thesis investigates how modern Natural Language Processing (NLP) and Generative AI can enhance bilingual comprehension of authentic English materials. The proposed system, the **English Language Assistant (ELA)**, transforms real-world media — text, audio, and video — into structured, interpretable learning content without simplification. By combining deterministic linguistic parsing (CoreNLP, spaCy) with LLM-based semantic annotation (currently GPT-4o, future Custom Linguistic Model), ELA generates *Recursive Linguistic Elements (RLE)* — hierarchical JSON structures linking syntax, translation, CEFR level, and contextual notes.

The research explores how such hybrid processing can support autonomous language learning through transparency and cognitive scaffolding. The implementation includes a functional desktop prototype with an interactive **Linguistic Visualizer** that allows learners to explore sentence structures and meanings dynamically. Findings suggest that explainable, AI-assisted annotation can make authentic input pedagogically accessible without altering its linguistic richness, offering a foundation for future adaptive and cross-platform developments.

Keywords: Natural Language Processing, Bilingual Learning, Recursive Linguistic Elements, Generative AI, Cognitive Scaffolding, Authentic Materials

Acknowledgements

I would like to express my heartfelt gratitude to my sister, *Eleonora Rastvorova*, who inspired me to create this project.

My deepest thanks go to my family — *Daria, Veronika, and Elmira* — for their constant support and encouragement.

I am sincerely grateful to my mentor, *Dr. Alex Vakaloudis*, for his key guidance and the masterful orchestration of my work on this project.

Finally, I would like to thank **Munster Technological University** for contributing to my professional growth and for giving me the opportunity to create something truly useful for people.

Contents

List of Figures

List of Tables

Abbreviations

RLE	Recursive Linguistic Elements
CALL	Computer-Assisted Language Learning
CEFR	Common European Framework of Reference
NLP	Natural Language Processing
LLM	Large Language Model
GPT	Generative Pre-trained Transformer
API	Application Programming Interface
POS	Part of Speech
IPA	International Phonetic Alphabet
ASR	Automatic Speech Recognition
JSON	JavaScript Object Notation
UML	Unified Modeling Language
UI	User Interface
UX	User Experience
GPU	Graphics Processing Unit
Syntactic dependency relations (Universal Dependencies):	
nsubj	nominal subject
obj	direct object
ROOT	root of the sentence
advmod	adverbial modifier
amod	adjectival modifier
pobj	object of preposition
prep	prepositional modifier
aux	auxiliary verb
xcomp	open clausal complement
advcl	adverbial clause modifier

Chapter 1

Introduction

1.1 Background and Motivation

Language learning technologies have evolved significantly in recent years, driven by the integration of artificial intelligence into computer-assisted education. Yet, most language-learning platforms continue to rely on pre-adapted, simplified materials that limit exposure to real linguistic complexity. While such materials lower the entry barrier for beginners, they rarely reflect the syntactic and semantic diversity of authentic English communication.

Authentic content — genuine speech, text, and media produced for native speakers — fosters higher motivation and communicative competence [1–5]. However, its uncontrolled richness often overwhelms learners, especially in the absence of proper scaffolding or bilingual reference. Modern advances in Natural Language Processing (NLP) and Generative AI now make it feasible to bridge this gap: instead of simplifying the input, they can *explain* it — maintaining authenticity while reducing cognitive load. This idea underpins the development of the **English Language Assistant (ELA)** system.

1.2 Research Aim

The overarching aim of this research is to design and evaluate a framework that combines rule-based linguistic parsing and LLM-based generative annotation to support **bilingual comprehension of authentic English materials**. Rather than filtering or simplifying input, the ELA system seeks to make complex language *understandable*

through dynamic annotation, hierarchical explanation, and visualised relationships between linguistic elements. The outcome is both a working prototype and a methodological contribution to bilingual, AI-assisted learning design.

1.3 Research Questions

The study is guided by the following research questions:

1. How can a hybrid pipeline of deterministic NLP tools and LLM-based generation produce accurate, pedagogically meaningful annotations from authentic materials?
2. What representation model best captures recursive linguistic relations while remaining human-interpretable and scalable across proficiency levels?
3. How can bilingual presentation and visualisation reduce cognitive load and enhance learner autonomy in the comprehension of authentic input?

1.4 Scope and Contribution

This project operates at the intersection of three research domains: *Applied Linguistics* (authenticity, scaffolding, autonomy), *Natural Language Processing* (syntactic parsing, alignment, translation), and *Human–Computer Interaction* (educational UX, cognitive design).

The core innovation is the *Recursive Linguistic Elements (RLE)* schema — a unified data model that represents a sentence as a nested hierarchy of linguistic components, each linked to metadata such as translation, CEFR level, phonetic cues, and explanatory notes.

The ELA framework contributes to CALL research by demonstrating that explainable, AI-supported annotation can make authentic materials accessible without altering their linguistic content. It also provides a reproducible architecture for integrating NLP pipelines with learner-facing visualisation tools — bridging technology, pedagogy, and usability.

1.4.1 Application Contribution

Beyond the research contribution, this project delivers a practical desktop prototype of ELA that:

- ingests authentic media and generates bilingual artefacts (transcripts, ASS/SRT subtitles);
- builds a transparent RLE data model (Sentence → Phrase → Word) for analysis and teaching;
- provides an interactive Linguistic Visualizer to explore grammar and CEFR annotations;
- enables export (JSON/CSV) for vocabulary practice and teacher review.

This addresses a real need in CALL by making authentic English interpretable via explainable AI.

Language support. The architecture is language-agnostic, but this project focuses on English → Russian for evaluation. Additional target languages can be added later via pluggable translator backends and locale-specific resources.

Planned Extension: Custom Linguistic LLM Model

A further research contribution of this work is the design of a specialised, domain-specific language model intended to eventually replace paid external LLM services within the annotation pipeline. While the current prototype uses GPT-based generation for explanatory notes, CEFR estimates, phonetic transcriptions, and short definitions, this dependency introduces both financial constraints and limited reproducibility.

To address this, the thesis proposes a **custom linguistic model** trained on a curated 25k-sample corpus of annotated linguistic elements. This planned model is designed to operate locally, generating structured metadata directly from syntactic and morphological descriptors (e.g. POS, dependency role, morphological features). Although its implementation lies beyond the scope of the current prototype, the model's specification, training corpus design, and integration strategy form part of the study's methodological contribution and outline a path toward a fully autonomous, scalable ELA system.

1.5 Thesis Structure

The remainder of this thesis is structured as follows:

- **Chapter 2** reviews the theoretical foundations of CALL, cognitive scaffolding, and NLP-based text analysis.
- **Chapter 3** details the system design methodology, defines the RLE schema, and outlines pedagogical objectives.
- **Chapter 4** describes the implementation approach, including pipeline architecture, Linguistic Visualizer, and evaluation framework.
- **Chapter 5** summarises the findings, discusses limitations, and proposes directions for future research and development.
- **Appendices A–C** provide the documented prototype code, annotated screenshots of the user interface, and a sample RLE JSON object.

Through these sections, the thesis demonstrates how the integration of NLP and AI-driven annotation can transform authentic English input into an adaptive, explainable, and research-ready environment for language learning.

Chapter 2

Background and Literature Review

Chapter overview. This chapter positions the English Language Assistant (ELA) within the broader landscape of language learning and natural language processing (NLP). It first introduces CALL foundations and the role of authentic input, then surveys the NLP tools used in the prototype and their place in the architecture, explains the current use of GPT and the roadmap towards a *Custom Linguistic Model*, motivates corpus and lexicographic resources, and concludes with a comparative view of existing systems. The goal is to connect technical choices to pedagogical objectives and make the structure of the background clear for the reader.

2.1 Computer-Assisted Language Learning (CALL)

Computer-Assisted Language Learning (CALL) has evolved from drill-based software of the 1980s into adaptive, data-driven environments that integrate natural-language processing and artificial intelligence. Levy [6] defined CALL as the search for and study of applications of the computer in language teaching and learning, while Chapelle [7] emphasised that its effectiveness depends on the interaction between technology, pedagogy, and linguistic theory. Recent overviews [8, 9] highlight a shift from content delivery to learner autonomy, authenticity, and adaptive feedback.

Modern CALL systems aim to provide meaningful communicative contexts rather than isolated grammar drills. They integrate multimodal input (text, audio, video) and support personalised learning trajectories through learner modelling and feedback. However,

most commercial platforms still rely on fixed curricula and simplified texts, offering limited exposure to authentic discourse. This limitation motivates the development of ELA, which combines CALL pedagogical principles with advanced NLP and AI methods to make authentic English accessible in a structured yet natural form.

2.2 NLP Tools

Natural–Language Processing (NLP) forms the computational foundation of ELA. In this section we describe the three core technologies used in the prototype— Stanford CoreNLP [10], spaCy [11], and OpenAI Whisper [12]—covering their historical background, strengths and weaknesses, and the role each plays in our system.

2.2.1 Review of Tools

Stanford CoreNLP. Originally introduced by the Stanford NLP Group, CoreNLP matured as a Java-based suite of mostly deterministic and classical statistical tools: tokenisation, sentence splitting, POS tagging, lemmatisation, NER, coreference, constituency and dependency parsing. For over a decade it became the "default" academic/teaching stack thanks to strong documentation, stable APIs, and wide language models maintained by a large research community. Despite the field's transition to neural architectures, CoreNLP remains attractive for projects that value predictable behaviour and the interpretability of pipeline stages.

spaCy. spaCy emerged as a modern, production–oriented Python library focused on speed, developer ergonomics, and custom pipelines. It couples efficient tokenisation and dependency parsing with a flexible component architecture (e.g., rule–based matchers, statistical taggers, transformer backends) while keeping a clean, Pythonic API. Its ecosystem (models, training CLI, project templates) makes it suitable for iterative prototyping that must grow into maintainable applications.

OpenAI Whisper. Whisper is a multilingual speech–to–text model trained on large–scale, diverse audio–text pairs. Its contribution is twofold: robust recognition in real–world conditions (accents, background noise) and unified handling of many languages without task–specific finetuning. For our use case—turning podcasts and videos into study material—Whisper bridges raw audio/video and the text–centric RLE pipeline.

Examples. **spaCy (example).** Dependency parse of "She gave him a book" yields arcs `nsubj(gave, She)`, `dobj(gave, book)`, `iobj(gave, him)` used in ELA's Visualizer.

CoreNLP (example). Constituency tree for a complex sentence supports offline enrichment (clause boundaries, coreference) fed back into RLE notes.

Whisper (example). A 30 s audio clip transcribed with timestamps provides segment timings for dual-subtitle alignment in ASS/SRT.

2.2.2 Capabilities Relevant to ELA

Syntactic structure: classical vs. neural. CoreNLP provides constituency parsing and coreference resolution out of the box, which is helpful for explanatory views of clause boundaries or noun phrase structure. spaCy, in contrast, excels at dependency analysis with fast, production–ready models and seamless integration into custom components (e.g., detectors for phrasal verbs or modal phrases). This complementary coverage is important for ELA: our visualiser benefits from simple phrase/word boundaries (dependency arcs, chunkers), whereas advanced analytics (e.g., coreference behind examples) can still be sourced from CoreNLP during offline processing.

Speech-to-text as an ingestion layer. Whisper enables ingestion of audio/video content: it yields time–aligned word or segment transcriptions, which ELA later segments into sentences and phrases for RLE. This allows learners to study authentic media without manual transcription, and gives the system the timing hooks required for bilingual subtitles and A/V exports.

2.2.3 Strengths and Weaknesses

Stanford CoreNLP. *Strengths:* mature toolset; stable behaviour; constituency parsing and coreference with strong academic provenance; reproducibility across versions. *Weaknesses:* Java runtime and larger memory footprint complicate lightweight desktop packaging; slower throughput compared to modern Python stacks; model updates lag behind state–of–the–art neural variants.

spaCy. *Strengths:* high performance and low latency; Python–native API; rich pipeline customisation; easy deployment with wheels; good transformer integration when needed. *Weaknesses:* out–of–the–box coreference and full constituency parsing are not first–class; quality depends on chosen language models; some academic features require third–party extensions.

Whisper. *Strengths:* multilingual robustness; strong noise tolerance; good alignment for subtitles; a single model family covers many use cases. *Weaknesses:* GPU preferred

for speed; large models increase latency and packaging size; diarisation and punctuation may need post–processing; domain–specific names may require custom correction passes.

2.2.4 Place in the ELA Architecture

ELA's data path separates *ingestion*, *linguistic structuring*, and *learner–facing interpretation*:

1. **Ingestion (Audio/Video → Text):** Whisper transcribes podcasts and videos with timestamps. When input is textual (PDF/TXT), this step is bypassed.
2. **Structuring (Text → RLE):** spaCy provides fast sentence segmentation, dependency edges, and phrase chunking; selected offline jobs may call CoreNLP for constituency trees or coreference to enhance examples and definitions. The output is a minimal, recursive RLE object (*Sentence* → *Phrase* → *Word*) enriched with CEFR hints and short notes.
3. **Interpretation (RLE → UI & A/V):** RLE drives the Linguistic Visualizer and subtitle generation (timing from Whisper, structure from spaCy/CoreNLP). The same schema supports export to JSON/CSV for rehearsal lists.

This division lets us choose the right tool for each layer: Whisper for robust ASR, spaCy for fast production parsing and custom rules, CoreNLP for optional analyses that benefit from constituency and coreference.

2.2.5 Comparison and Design Trade–offs

TABLE 2.1: Comparative view of core technologies in ELA

Dimension	**spaCy [11]**	**CoreNLP [10]**
Primary focus	Fast dependency parsing, custom Python pipelines	Broad classical suite incl. constituency & coreference
Latency/Throughput	High (good for interactive UIs)	Moderate (better offline/batch)
Packaging	Python wheels, easy embedding	Java runtime; heavier desktop bundling
Extensibility	Component hooks, rule–based matchers	Annotators; good academic coverage
Best use in ELA	Online structuring for RLE; phrase detection	Offline enrichment (constituency, coref)
ASR Layer: Whisper [12]		
Role in ELA: Multilingual transcription with timestamps (media ingestion) **Strengths:** Robust to noise/accents; unified model family; good alignment for subtitles **Caveats:** GPU preferred; large models; diarisation/punctuation may need post–processing		

2.2.6 Potential Risks for Application in this Project

Runtime and packaging. CoreNLP's JVM adds weight to a cross–platform desktop build. *Mitigation:* run CoreNLP only for offline enrichment steps and cache results; keep the interactive path purely in Python (spaCy).

ASR latency and accuracy. Whisper large models improve accuracy but slow down transcription on CPU. *Mitigation:* allow model size selection (tiny → large); cache transcripts; add a light spelling/normalisation pass and a domain–term correction list.

Model drift and reproducibility. Upstream model updates may change outputs. *Mitigation:* pin versions in requirements; store RLE alongside model/version metadata for later auditing.

Consistency of structure across tools. Different parsers may disagree on boundaries. *Mitigation:* ELA normalises to the minimal RLE shape (Sentence → Phrase → Word), avoiding parser–specific artefacts while still benefiting from richer analyses if present.

2.2.7 Why This Combination Fits ELA

Pedagogically, ELA must *explain* rather than only *classify*. That means predictable, readable structure with fast feedback:

- **spaCy** supplies the speed and ergonomic hooks needed for an interactive learner UI and for simple, stable boundaries.
- **CoreNLP** adds deeper analysis (constituency, coreference) when offline accuracy justifies the overhead.
- **Whisper** opens authentic audio/video to learners who might otherwise avoid spoken English altogether.

Together they form a pipeline that is *transparent* (deterministic parses), *performant* (fast enough for desktop use), and *extensible* (ready for LLM-based enrichment via GPT or a custom model). See Appendix B for UI and pipeline screenshots.

2.3 Generative AI for Linguistic Analysis

Large-Language Models (LLMs) such as GPT-4 introduce a new dimension to language-learning technology. While traditional parsers deliver syntactic and morphological data, LLMs can provide semantic paraphrases, explanations, and bilingual glosses. Research [13–15] shows that generative AI can enhance learner engagement through adaptive, contextual feedback, but also warns against factual inaccuracies and cognitive overload when used without control.

ELA mitigates these issues by embedding LLMs in a deterministic pipeline: first, verified linguistic parsing is performed by CoreNLP and spaCy; then, GPT generates human-readable notes grounded in that structure. This hybrid design merges analytical transparency with interpretive fluency, aligning with current work on explainable AI in education.

2.3.1 The Use of GPT in ELA

In the current prototype, ELA relies on OpenAI GPT-4 through its API because it offers the broadest multilingual coverage and the highest reliability for natural-language explanations. Specifically, GPT is used to:

- generate short `linguistic_notes` that explain grammatical structures in plain English;
- provide bilingual translations (English ↔ Russian) at word, phrase, and sentence levels;
- estimate CEFR difficulty levels based on lexical and syntactic complexity;
- supply IPA phonetic transcriptions and short definitions with contextual examples.

This flexibility allows the prototype to handle diverse input without manual rule engineering, accelerating development and ensuring broad linguistic coverage.

2.3.2 Strengths and Weaknesses of the GPT Approach

Strengths.

- **Multilingual fluency.** GPT handles translation and cross-lingual tasks with minimal configuration.
- **Contextual awareness.** Explanations adapt to sentence-level meaning rather than applying generic templates.
- **Rapid iteration.** Changes to prompt design allow immediate refinement of pedagogical tone and detail.

Weaknesses.

- **Cost.** API charges accumulate quickly with large-scale processing, limiting deployment in resource-constrained settings.
- **Latency.** API calls introduce delay compared with local processing, limiting real-time classroom deployment.
- **Drift and versioning.** Model updates can alter outputs even with identical prompts, complicating reproducibility.
- **Privacy.** Sending learner data to external servers raises concerns in educational and institutional contexts.

2.3.3 Towards a Custom Linguistic Model

To address these weaknesses, ELA's design includes a path toward a **custom, domain-specific linguistic model**. This model would be fine-tuned on a curated corpus of annotated linguistic elements, trained to generate the same structured metadata (CEFR, phonetics, definitions, synonyms) currently produced by GPT, but *locally* and *deterministically*. Detailed specifications for this model are presented in Section 2.6.

Expected advantages.

- **Cost reduction.** Zero marginal cost per annotation after initial training.
- **Reproducibility.** Fixed weights guarantee identical outputs for identical inputs.
- **Privacy.** All processing happens on-device or within institutional infrastructure.
- **Speed.** Local inference (35–65 ms per node on GPU) is faster than network-bound API calls (600–1200 ms).

Challenges. Developing a proprietary model introduces its own costs in computing and maintenance. Training quality depends on the diversity and size of annotated data, which must capture the full range of grammatical structures, CEFR levels, and lexical contexts that ELA encounters. The current prototype therefore uses GPT as a *baseline and validation reference* while preparing the infrastructure for a future local model.

2.3.4 Hybrid Roadmap

The long-term architecture envisions a **hybrid annotation backend**:

- **Local model (primary):** handles routine metadata generation for common structures with predictable outputs.
- **GPT (fallback):** used for edge cases, ambiguous sentences, or rare grammatical phenomena where the local model's confidence is low.
- **Human review (optional):** teachers can verify or override AI-generated notes, feeding corrections back into the training corpus.

This design balances cost, performance, and pedagogical quality while maintaining the explainability and transparency central to ELA's educational mission.

2.3.5 Relation to ELA's Design Goals

The use of generative AI—whether GPT or a custom model—aligns with ELA's core objectives:

- **Authenticity:** explanations are grounded in real linguistic structure, not pre-written templates.
- **Scaffolding:** learners receive context-specific help rather than generic grammar rules.
- **Autonomy:** adaptive hints allow learners to explore at their own pace and level.
- **Transparency:** provenance fields record whether a note came from a rule, heuristic, or LLM, supporting teacher mediation.

Summary. Generative AI serves as the *interpretive layer* in ELA's pipeline, transforming deterministic parse trees into human-readable pedagogical content. While the current prototype relies on GPT, the system's architecture is designed to accommodate a future transition to a domain-adapted local model, ensuring long-term sustainability, privacy, and reproducibility.

2.4 Authentic Materials in Language Learning

Extensive applied-linguistics research supports the use of authentic materials—texts and recordings produced for native speakers rather than for instruction. Gilmore [1] demonstrated that authentic input enhances communicative competence and intercultural awareness. Peacock [5] and Guariento and Morley [2] found higher motivation when learners interact with real-world texts under guided conditions. Berardo [3] and Mishan [4] further showed that authentic materials aid lexical retention and pragmatic awareness, particularly from CEFR B1 upwards.

Beginners, however, may find authentic input challenging due to lexical density and idiomatic usage. Pedagogical frameworks therefore emphasise scaffolding—glossaries, graded hints, and visualisation. ELA operationalises this by automatically generating bilingual annotations, CEFR tags, and grammatical structures, allowing learners to explore genuine language without simplification.

2.4.1 Historical Context

The use of authentic materials in language teaching emerged from communicative language teaching (CLT) in the 1970s and 1980s, which prioritised real-world communicative competence over form-focused drills. Early advocates such as Peacock [5] argued that exposure to genuine discourse—newspapers, radio broadcasts, interviews—better prepares learners for interaction with native speakers than pedagogically simplified texts.

By the 2000s, corpus linguistics and digital media made authentic input more accessible. Gilmore [1] demonstrated empirically that learners using authentic materials showed greater gains in listening comprehension, pragmatic awareness, and intercultural sensitivity compared to those using textbook dialogues. Guariento and Morley [2] refined this position, noting that beginners benefit from *scaffolded* authentic input rather than unmediated exposure.

2.4.2 Strengths of Authentic Input

Research identifies several pedagogical advantages:

- **Motivation.** Learners report higher engagement when studying content relevant to their personal or professional interests [5].
- **Lexical and cultural richness.** Recurrent collocations, idioms, and cultural references appear only in real-world corpora, supporting advanced vocabulary growth [3, 4].
- **Transferability.** Skills developed through authentic materials transfer more readily to real communication than those practiced in artificial contexts [1].
- **Pragmatic awareness.** Authentic discourse models register variation, politeness strategies, and turn-taking conventions absent from scripted materials [2].

2.4.3 Challenges and Limitations

Despite these benefits, authentic materials pose practical challenges:

- **Lexical density.** Uncontrolled vocabulary and idiomatic expressions can overwhelm learners below B1 level [3, 5].
- **Lack of grading.** Unlike textbook materials, authentic texts do not follow difficulty progressions, requiring teachers to select and adapt content carefully [2].

- **Cultural and contextual gaps.** References to current events, cultural norms, or domain-specific knowledge may hinder comprehension without explicit scaffolding [1].

These limitations explain why authentic materials have historically been reserved for intermediate and advanced learners, with beginners relying on graded readers and controlled input.

2.4.4 Implementation in the ELA System

ELA addresses the scaffolding challenge computationally. Rather than simplifying authentic English, it *explains* it through:

- **Bilingual glosses:** word and phrase translations reduce lookup friction.
- **CEFR tagging:** learners can filter content by estimated difficulty.
- **Grammatical decomposition:** the RLE structure reveals how sentences are built, making complex syntax interpretable.
- **Contextual notes:** short explanations clarify idiomatic or culturally specific usage.

This approach preserves linguistic authenticity while lowering cognitive barriers, operationalising the scaffolding principles advocated by Guariento and Morley [2].

2.4.5 Strengths within the ELA Architecture

- **Unified workflow.** Authentic input is accepted in any modality (text, audio, video) and converted into one recursive schema (RLE) for uniform processing.
- **Learner agency.** Users select materials based on personal interest, aligning with autonomy principles [16–18].
- **Scalability.** Automated annotation scales to large corpora without manual intervention, making authentic materials viable at scale.

2.4.6 Weaknesses and Mitigation

- **Quality variance.** Noisy transcripts (from ASR) or domain-specific jargon may reduce annotation accuracy.
  emphMitigation: manual transcript import; domain-term glossaries; teacher review hooks.

- **Over-reliance on AI.** LLM-generated notes may contain subtle errors or culturally inappropriate translations.
  emphMitigation: provenance tracking; human-in-the-loop correction; validation against reference corpora.

2.4.7 Strategic Importance for the Project

Authentic materials form the *pedagogical foundation* of ELA. By refusing to simplify input, the system aligns with current CALL philosophy: learners should encounter language as it is *used*, not as it is *taught*. This positions ELA as a complement to traditional curricula, providing exposure to real discourse while structured lessons build foundational competence.

Summary. From its origins in communicative teaching to its AI-driven realisation in ELA, the principle of authenticity has evolved from a pedagogical ideal into a computational framework. ELA demonstrates that *explainability*—rather than simplification—can make authentic language accessible, supporting learner autonomy and real-world communicative competence.

2.5 User Interface Rationale

The ELA interface follows cognitive-load theory [19] and multimedia-learning principles [20, 21]. Information is revealed hierarchically—from sentence to phrase to word—preventing overload. Colour-coding, collapsible trees, and contextual pop-ups support dual-channel (visual + verbal) processing, improving recall via the dual-coding effect. Bilingual refers solely to learning content (EN+L1) for comprehension.[1]

[1]The interface itself remains English-only; bilingual support applies exclusively to study materials, consistent with CALL's focus on learner autonomy and control.

2.5.1 Pedagogical Foundations

The UI design is grounded in established principles from educational psychology and HCI:

- **Cognitive load management** [19]: progressive disclosure reduces working memory demands by showing only relevant information at each interaction level.
- **Multimedia learning** [20, 21]: combining text, visual structure, and optional audio supports dual-channel processing and improves retention.
- **Learner autonomy** [16–18]: users control what to expand, when to view translations, and which materials to study.

Dual–channel learning. Research shows that learners process information more effectively when it is presented through both visual and auditory channels simultaneously [22]. ELA implements this by pairing text (RLE structure) with optional audio playback (from Whisper transcripts) and visual cues (syntax trees, CEFR tags).

2.5.2 Simplicity and Cognitive Economy

The prototype intentionally avoids complex menus, animations, or gamification. This minimalism reflects two principles:

1. **Extraneous load reduction.** Unnecessary visual elements divert attention from learning content [21].
2. **Accessibility.** Clean, text-based interfaces work on low-power devices and remain compatible with screen readers.

Studies [20, 22] demonstrate that decorative graphics and animations increase extraneous cognitive load. ELA therefore limits visible controls to what is pedagogically necessary:

- a clear progression (Sentence → Phrase → Word);
- a simple expand/collapse mechanism for RLE nodes;
- inline bilingual notes and CEFR indicators.

2.5.3 Functional Priorities

The interface prioritises:

- **Content discovery:** finding and loading authentic materials (Projects, Files).
- **Analysis transparency:** observing the NLP pipeline (Analyze view with progress bars).
- **Structural exploration:** navigating sentence hierarchies (Linguistic Visualizer).
- **Export and rehearsal:** extracting vocabulary for external practice (Vocabulary screen).

Each screen corresponds to a distinct pedagogical task, avoiding feature creep and maintaining focus on ELA's core mission: making authentic English interpretable.

2.5.4 Minimalism as Accessibility

By keeping the UI simple and text-centric, ELA ensures:

- **Cross-platform compatibility:** Kivy-based implementation runs on Windows, macOS, Linux without modification.
- **Low resource requirements:** the absence of heavy graphics allows deployment on older laptops and in bandwidth-constrained environments.
- **Screen reader support:** text labels and hierarchical structures are inherently accessible; screen readers can interpret them directly.

By avoiding complex widgets, ELA maintains compatibility with low-power laptops and offline deployments, reinforcing its non-commercial, educational mission.

2.5.5 Focus on Learning Content

Unlike gamified platforms (e.g., Duolingo), ELA does not include:

- progress bars, achievements, or leaderboards;
- timed exercises or competitive elements;
- social features or user-generated content sharing.

This deliberate omission reflects ELA's positioning as a *comprehension tool* rather than a *practice platform*. Learners use ELA to *understand* authentic materials; structured practice and production activities remain the domain of classroom instruction or complementary tools.

2.5.6 Role within the Project Architecture

The UI serves as the *interpretive layer* between computational analysis and human understanding. It does not replace teachers or curricula; instead, it provides:

- **Transparency:** learners see how sentences are decomposed and annotated.
- **Agency:** users select materials, control detail level, and export data for external use.
- **Reproducibility:** all interactions with RLE objects are logged and exportable, supporting research and teacher review.

2.5.7 Strengths and Trade-offs

Strengths.

- **Clarity:** minimal design keeps focus on linguistic content.
- **Portability:** cross-platform compatibility with low system requirements.
- **Pedagogical alignment:** design choices map directly to cognitive load and autonomy principles.

Trade-offs. The minimalist approach entails certain limitations:

- fewer customisation options (fonts, themes, animations);
- deliberate omission of social or reward mechanisms to preserve focus.

These trade-offs are intentional: the prototype aims to validate ELA's core hypothesis (that explainable AI can scaffold authentic input) without the distraction of feature-rich but pedagogically secondary elements.

2.5.8 Summary

The ELA interface is designed for *interpretive exploration*, not drill or assessment. By aligning visual design with cognitive load principles, minimising extraneous elements, and prioritising learner control, the UI supports ELA's pedagogical mission: making authentic English accessible through transparent, AI-assisted scaffolding.

2.6 Development of a Custom Linguistic Model

A long-term objective of ELA is to reduce dependency on paid external LLM services (such as commercial ChatGPT) by integrating a specialised, domain-adapted linguistic model capable of generating the key metadata used in the Recursive Linguistic Elements (RLE) structure. While GPT-4 provides high quality and versatility, it introduces cost, reproducibility, and privacy constraints that limit large-scale or institutional deployment. For this reason, ELA adopts a hybrid strategy in which GPT remains available for open-ended interpretation, while structured pedagogical metadata is generated by an in-house fine-tuned model.

The custom model will operate on top of a pretrained sequence-to-sequence architecture (e.g., T5-small or Flan-T5) and receive as input a normalised description of a linguistic node:

- `Level: word` phrase | sentence|;
- `Text: …`;
- `POS=…`;
- `dep=…`;
- `morph=….`

From this compact descriptor, the model is trained to generate a full metadata package compatible with the RLE schema:

- `linguistic_notes` — a brief natural-language explanation of the grammatical role in context;
- `cefr_level` — an estimated difficulty level (A1–C2);
- `phonetic.uk / phonetic.us` — IPA transcription for UK/US English;

- `definitions` — short dictionary-style explanations;
- `synonyms` — 2–4 typical synonyms relevant to the context.

To train this behaviour, a **25,000-example annotated corpus** will be created. It covers the three abstraction levels of RLE — words, phrases, and sentences — with balanced distribution across CEFR: approximately 20% A1–B1 and 80% B2–C2. Each example encodes POS, dependency type, morphological features, and the full target metadata package. This design ensures that the model focuses not on learning English from scratch, but on calibrating its outputs to ELA's structured annotation style.

The corpus serves as a domain-specific overlay on top of the pretrained model's existing linguistic competence. It teaches the model to map combinations such as `Level+POS+dep+morph` to stable explanatory patterns, especially for dependencies such as `nsubj`, `obj`, `ROOT`, `advmod`, `amod`, `pobj`, `prep`, `aux`, `xcomp`, `advcl` and others typical of modern English syntax.

Table 2.2: Comparison of annotation backends in ELA

Dimension	**GPT-4 API**	**Local LLM (25k)**
Primary role	Open-ended semantic explanations; fallback for ambiguous cases	Structured metadata generation (CEFR, phonetics, definitions, synonyms)
Latency	600–1200 ms per node (network-dependent)	35–65 ms per node (on-device GPU)
Cost	Usage-based cost; scales poorly for large datasets	Zero marginal cost after training
Reproducibility	Non-deterministic; updates on provider side	Fully deterministic under fixed weights
Offline availability	No; requires external API	Yes; runs locally
Metadata consistency	Variable style; requires post-processing	Strict schema-conforming JSON, stable formatting
Best use in ELA	Complex interpretations, disambiguation, rare grammar	Main annotation engine for RLE metadata

2.7 Corpus Rationale and Coverage

The design target is a 25,000-example annotated corpus (planned) reflecting a balance between linguistic adequacy, structural diversity, and computational feasibility. Within the scope of this project, evaluation is performed on a smaller curated dataset drawn from authentic media, while the 25k corpus remains future work. The goal is not to train a general-purpose language model, but to fine-tune a pretrained seq2seq architecture to reliably produce RLE-compatible metadata. For this task, the corpus must

demonstrate a wide range of grammatical situations while remaining small enough for iterative training on a mid-range GPU (e.g., NVIDIA Quadro T2000).

The corpus includes dense examples that combine several dimensions: `Level`, `Text`, `POS`, `dep`, `morph` and a structured set of target fields. This produces hundreds of thousands of contextual associations, comparable to the scale typically used in domain-adaptive fine-tuning of compact transformer models.

Coverage spans both frequent and less common dependency patterns. High-frequency structures (e.g., nominal subjects, adjectival modifiers, direct objects) appear hundreds of times, while mid-range constructions (e.g., `xcomp`, `advcl` clauses) appear dozens of times. This ensures stable learning across the full range of syntactic environments relevant to ELA.

The CEFR distribution is intentionally skewed towards B2–C2 (80% of the corpus) to support advanced academic and professional vocabulary, while still including sufficient A1–B1 data to handle foundational structures and explanations.

In summary, the 25k design provides broad structural coverage, pedagogical balance, sufficient lexical variety, and practical feasibility for repeated model updates during the lifecycle of the project.

Table 2.3 summarises the design choices underlying the 25k training corpus. The 25,000-node size balances breadth and computational feasibility. The distribution across abstraction levels (40% word, 35% phrase, 25% sentence) ensures the model operates reliably at all three RLE tiers. The CEFR skew towards B2–C2 (80%) reflects ELA's focus on intermediate-to-advanced learners, while 20% A1–B1 coverage supports foundational explanations. Syntactic and morphological diversity enables the model to handle both common and atypical structures, producing precise, schema-conforming metadata aligned with established lexicographic practice.

2.7.1 Corpus Generation Methodology

The 25,000-example corpus will be generated through systematic **prompt engineering** using GPT-4o (or later versions). This approach leverages the model's linguistic competence to produce high-quality, schema-conforming training examples aligned with established lexicographic and pedagogical standards.

The generation process involves carefully designed prompts that instruct the model to:

- extract linguistic metadata (POS, dependency relations, morphological features) from authentic sentence contexts;

TABLE 2.3: Coverage of the 25k training corpus for the custom linguistic model

Aspect	**Coverage target**	**Purpose in ELA**
Total size	25,000 annotated nodes	Sufficient density for stable fine-tuning of a compact seq2seq model on RLE-style metadata.
Abstraction levels	Approx. 40% word-level, 35% phrase-level, 25% sentence-level	Balanced exposure to local, mid-range, and global structures needed for RLE (Word → Phrase → Sentence).
CEFR distribution	∼20% A1–B1, ∼80% B2–C2	Focus on academic and professional discourse while still covering foundational structures and beginner-friendly explanations.
Parts of speech (POS)	Nouns, verbs, adjectives, adverbs, pronouns, determiners, prepositions, conjunctions and others	Ensures the model can handle typical grammatical categories encountered in authentic texts and transcripts.
Syntactic dependencies	High-frequency roles (e.g. `nsubj`, `obj`, `ROOT`, `advmod`, `amod`, `pobj`, `prep`, `aux`, `xcomp`, `advcl`)	Teaches robust mappings from `Level+POS+dep+morph` to explanatory patterns and CEFR difficulty.
Morphological features	Variation in `Tense`, `Number`, `Person`, `VerbForm`, `Degree`, `Definite`, etc.	Supports fine-grained notes about tense, aspect, agreement, comparison and definiteness in learner-friendly language.
Metadata fields per example	Each instance includes target `linguistic_notes`, `cefr_level`, `phonetic.uk/us`, `definitions`, `synonyms`	Aligns training data directly with the RLE schema, enabling the model to output schema-conforming JSON for every node.
Source orientation	Lexicographic and corpus-informed selection (Oxford, Cambridge, COCA, BNC, AWL, etc.)	Anchors generated metadata in real usage, dictionary practice and CEFR-oriented vocabulary profiles.

- generate structured outputs matching the RLE schema (CEFR level, phonetics, definitions, synonyms, linguistic notes);
- reference authoritative sources for validation, including: **Oxford Learner's Dictionary**, **Cambridge Dictionary**, **Macmillan Dictionary**, **Longman Dictionary of Contemporary English**, and **Collins COBUILD Dictionary**.

Each prompt explicitly instructs GPT-4o to ground its annotations in these reference resources, ensuring that CEFR difficulty estimates, phonetic transcriptions, sense definitions, and synonym selections align with established lexicographic practice rather than

arbitrary model outputs. The use of authoritative dictionaries guarantees that the resulting corpus reflects real pedagogical standards and authentic usage patterns, making the fine-tuned model a reliable component within the ELA annotation pipeline.

This methodology enables rapid, scalable corpus construction while maintaining high annotation quality through the explicit constraint to consult trusted lexicographic sources during generation.

2.7.2 Lexicographic and Corpus Resources

The corpus construction relies on established lexicographic and corpus resources that serve as reference points for CEFR allocation, word sense definitions, phonetic transcription, synonym selection, and contextual validation. These include:

- Oxford Learner's Dictionaries [23] — A1–B2 vocabulary with CEFR markings;
- Cambridge Dictionary [24] — definitions, examples, and difficulty levels;
- Longman Dictionary of Contemporary English [25] — controlled-language definitions and frequency data;
- Merriam–Webster [26] — UK/US contrasts;
- Collins COBUILD [27] — corpus-based definitions;
- Macmillan Advanced Learner's Dictionary [28] — B2–C2 lexicon;
- COCA [29] and BNC [30] — frequency and contextual use;
- Academic Word List [31] and CEFR-aligned wordlists — external benchmarks.

These sources help ensure that the generated metadata in ELA is aligned with real dictionary practice, authentic corpus use, and established pedagogical standards.

2.8 Multimodal Learning and Learner Autonomy

Contemporary CALL research recognises that effective digital environments must engage learners through multiple sensory channels and provide control over learning pathways. According to Mayer [20], comprehension improves when information is processed through both the visual and auditory modalities—a core principle of the multimedia learning theory. When learners can listen to, read, and visually explore the same linguistic

structure, they integrate declarative and procedural knowledge more efficiently. ELA adopts this multimodal approach by synchronising audio (from Whisper transcripts), text (from NLP analysis), and visual cues (from grammar trees and colour-coded highlights). This dual-channel design reduces extraneous cognitive load [19] and supports retention through the dual-coding mechanism.

Equally important is the concept of learner autonomy, introduced by Holec [16] and further developed by Benson [18] and Little [17]. Autonomous learners take responsibility for their own goals, materials, and pace of study, transforming learning into a self-directed process rather than a teacher-led one. ELA aligns with this philosophy by enabling users to select any authentic input—texts, videos, or podcasts—based on their personal or professional interests. Instead of prescribing content or difficulty, the system offers tools for exploration and analysis: translation layers, CEFR indicators, and linguistic scaffolds. This design encourages reflection and self-regulation, two critical factors in developing long-term communicative competence. By combining multimodal presentation with user autonomy, ELA exemplifies the current pedagogical direction in CALL: adaptive, learner-centred, and cognitively grounded instruction.

2.9 Existing Language-Learning Systems

A number of commercial and research systems attempt to combine technology with language pedagogy. **Duolingo** applies gamified repetition and basic translation tasks [32]. **LingQ** lets users upload authentic materials and track word frequency, but lacks grammatical analysis. **Reverso Context** retrieves bilingual sentence pairs from large corpora, offering contextual examples without syntactic processing. **Grammarly** performs monolingual grammar correction through rule-based NLP.

Academic prototypes such as **NLP4CALL** [33] have explored the use of parsing and annotation for instructional purposes. More recently, Ziegler et al. [34] reviewed a range of AI-enhanced language-learning platforms that employ machine learning and natural-language processing to personalise feedback and generate adaptive content. However, most of these systems remain narrowly focused on specific tasks—such as grammar correction, vocabulary review, or pronunciation scoring—without providing holistic, bilingual, linguistically transparent support for working with authentic materials.

2.10 Comparative Feature Analysis

This section summarises the comparative analysis of representative Computer-Assisted Language Learning (CALL) systems across key linguistic and pedagogical dimensions. Table 2.4 highlights how most platforms prioritise engagement (gamification, fluency drills) but rarely combine authentic input, structured linguistic annotation, CEFR alignment, and bilingual scaffolding within a single workflow.

TABLE 2.4: Comparison of linguistic and pedagogical features across existing systems

System	Authentic Materials	Linguistic Analysis	CEFR Mapping	Bilingual Support
Duolingo	✗	✗	✓ (implicit)	✓ (translation tasks)
LingQ	✓ (user)	✗	✗	✓ (dictionary)
Reverso Context	✓ (bilingual corpora)	✗	✗	✓ (bilingual)
Grammarly	✓	✓ (rule-based, error)	✗	✗
NLP4CALL	✓	✓ (syntactic, limited)	✗	✗
ELA (proposed)	✓✓	✓✓ (CoreNLP, spaCy, GPT)	✓✓	✓✓ (EN–RU)

Legend: ✓ — feature supported; ✓✓ — fully integrated and central to system design; ✗ — not supported.

Interpretation. The comparison reveals that while existing CALL systems offer partial solutions, none unites authentic, user-defined input with transparent linguistic analysis, CEFR-aligned metrics, and bilingual comprehension scaffolding. ELA uniquely operationalises this combination, making authentic media accessible through structured, interpretable annotation rather than simplified adaptation.

2.11 Research Gap

As Table 2.4 indicates, no single platform currently provides an integrated pipeline that spans real-world input, linguistic explanation, and bilingual contextualisation. Current

CALL tools tend to either prioritise gamification (e.g., Duolingo) or focus narrowly on grammar correction (e.g., Grammarly), with limited transparency about the linguistic structures learners encounter.

The proposed **ELA framework** bridges this gap by combining deterministic NLP (CoreNLP, spaCy) with generative, human-readable commentary (GPT) within the Recursive Linguistic Elements (RLE) schema. Rather than simplifying authentic English, ELA preserves its complexity while offering guided exploration, aligning analysis and feedback with CEFR levels. This approach redefines digital language learning as structured comprehension of authentic materials— a process that mirrors how learners naturally derive understanding from real communication.

Conclusions

Chapter 2 outlined the conceptual and technological foundations of ELA. First, the system relies on established NLP pipelines (spaCy, CoreNLP and Whisper) to produce transparent grammatical structure, ensuring deterministic and auditable analysis. These components form the lower analytical layer of the pipeline.

Second, generative AI introduces high-level interpretative capabilities. GPT-4 provides semantic paraphrasing and pedagogical explanations, but its use also highlights limitations related to cost, reproducibility, and long-term dependency on commercial APIs.

To mitigate these constraints, ELA specifies a custom 25k-example linguistic model designed specifically for structured metadata generation. Rather than replacing GPT-4 immediately, this model is defined as a future extension that will operate locally on top of existing parsers, outputting CEFR levels, phonetics, definitions, synonyms and concise grammatical notes in the RLE format. The surrounding sections detail the corpus design, coverage and lexicographic resources needed to train such a model in a computationally realistic way.

In the current prototype, all enrichment is still handled by GPT-4, while the local model remains a clearly scoped direction for future work. The chapter therefore motivates a unified architecture in which deterministic parsing and generative enrichment can later be augmented by the in-house linguistic model, connecting established CALL principles with modern NLP and LLM techniques.

Chapter 3

Design and Development of the ELA Object System Based on Recursive Linguistic Elements

3.1 Introduction

This chapter describes the object system that powers ELA. At its core is a simple, recursive JSON design called **RLE — Recursive Linguistic Elements**. RLE turns any authentic English input into a small number of human–readable objects: the full sentence, a few meaningful *parts* (e.g., a verb phrase), light *word tags* (e.g., pronoun, modal verb), and short learner notes (CEFR hint, phonetic cue, translation, one–sentence explanation). The goal is practical: help people understand real English faster and with less stress.

We deliberately avoid linguistic jargon. The system shows structure in plain language and lets learners control depth and pace. This aligns with research on autonomy in CALL ([16–18]), on authenticity and motivation ([1–3, 5]), and on cognitive load and multimedia learning ([19–22]).

As discussed later in Section 3.5, our UI mirrors well-known cognitive models of comprehension through progressive disclosure. The planned custom linguistic model (Chapter 2, Sections 2.6–2.7) is designed to operate on top of this RLE structure, but is explicitly scoped as future work: the current prototype uses GPT-4 for all enrichment steps.

3.2 Problem Definition

Many digital tools teach vocabulary and grammar separately and use simplified content. Learners then struggle with real emails, meetings, papers, and media. Technically, most systems are built for fixed exercises, not for open, user–selected input. They rarely keep a clear link between the sentence as a whole and the pieces that matter for meaning.

ELA addresses this gap with RLE: a simple, recursive data model that *keeps the sentence intact*, shows a few meaningful parts, adds plain word tags, and provides just–enough notes. The analysis is intentionally *supportive*, not exhaustive: ELA is a helper, not a teacher or a full linguistic judge.

3.3 Objectives (O)

The objectives form the functional contract of ELA. Each objective has a linguistic and a pedagogical motivation.

1. **Use authentic input.** Let learners bring real texts, podcasts, and videos. Convert them to a form ready for analysis and study [1, 2].
2. **Show structure simply.** Present each sentence as a few parts (e.g., verb phrase) with plain *word tags* (pronoun, modal verb, noun) — no jargon [19, 20].
3. **Add just–enough help.** For each part, provide CEFR hint, phonetic cue, short translation, and a one–sentence explanation [21, 22].
4. **Keep learner control.** Allow filtering and exploration by parts, tags, or difficulty [9, 16, 18].
5. **Stay transparent.** Record where notes came from (human rule, simple heuristic, LLM suggestion) to support teacher review [7].
6. **Support rehearsal.** Let people export short lists (phrases, collocations, examples) for spaced practice [3, 5].
7. **Work across modalities.** When audio is available, align text with time codes to support listening and pronunciation [22].
8. **Be lightweight and portable.** Keep the format stable and readable so that UI and back-end can evolve independently.

3.4 Educational Framework Alignment

Authenticity and motivation. Exposure to authentic materials increases engagement and communicative competence, especially from B1 level upward [1, 3, 5]. RLE operationalises authenticity by making real sentences analysable without rewriting them.

Autonomy. Giving learners control over content and depth supports progress and self-regulation [9, 16–18]. RLE lets users expand only the parts they need and filter by difficulty.

Cognitive load and multimedia learning. Small, meaningful chunks and dual channels (text + audio) improve understanding and recall [19–22]. RLE's progressive disclosure reduces extraneous load and keeps focus on meaning.

3.5 Brain-Inspired Processing (Cognitive Rationale)

Our goal is not to "simulate the brain," but to align ELA with well-established cognitive principles that describe how people form understanding. In ELA the learner first sees the whole sentence, then opens parts, and—only if needed—opens smaller parts. This mirrors three documented ideas:

1. **Segmenting and chunking.** Understanding improves when complex input is presented in manageable segments and the learner controls the pace [20, 21]. RLE implements this by progressive disclosure (Sentence → Phrase → Word).
2. **Cognitive load management.** Limiting extraneous detail helps working memory focus on essential structure [19]. By default ELA shows a few meaningful parts and only light notes (CEFR, phonetic cue, short translation).
3. **Dual-channel learning and autonomy.** Pairing text with optional audio supports dual coding [20, 22], while learner control over what to expand and when supports self-regulation [9, 16–18].

Within this framing it is reasonable to say that ELA *approximates the human process of understanding*: a quick holistic pass, followed by selective decomposition of hard parts, repeated until the learner builds a stable picture of the whole from its parts. Our claims stay within the scope of these cited cognitive and CALL sources, and we avoid strong "neuroscience imitation" statements that the present thesis does not test empirically.

This cognitive framing motivates the concrete system behavior captured in the Functional Requirements of Section 3.6.

3.6 Functional Requirements (FR)

This section defines the **functional requirements** of the ELA framework—what the system *must do* to achieve its pedagogical and technical objectives. Each requirement is expressed as an observable capability supporting learner autonomy, transparency, and comprehension of authentic input. Together, they form the behavioural foundation of the system, later validated through prototype testing (see Chapter 4).

Table 3.1 lists these requirements and outlines their pedagogical rationale.

TABLE 3.1: Functional requirements and pedagogical rationale

FR	**Description**	**Why it helps learning**
FR-1	Ingest authentic text/audio/video and prepare a study transcript.	Relevance and motivation through learner-selected content [5].
FR-2	Build a simple sentence structure: a few parts plus plain word tags (pronoun, modal verb, noun, etc.).	Notice patterns without jargon; lower cognitive load [19, 20].
FR-3	Attach notes per part: CEFR, phonetic cue, short translation, one–sentence explanation.	Just-enough scaffolding; supports listening and reading [21, 22].
FR-4	Allow filtering and search (by part, tag, CEFR).	Autonomy and targeted practice [9, 18].
FR-5	Keep provenance for each note (rule/heuristic/LLM).	Transparency for teacher mediation [7].
FR-6	Export short lists (phrases, collocations, examples).	Supports rehearsal and retention [3].
FR-7	Optional audio alignment (start/end).	Dual-channel processing and pronunciation practice [22].
FR-8	Store everything as stable RLE JSON with versioning.	Portability and long-term use across UIs.

How to read Table 3.1. Each FR links an action to its learning rationale. The list provides measurable, verifiable functions forming the operational basis of the prototype.

3.7 Non-Functional Requirements (NFR)

While functional requirements define *what* ELA must do, the **non-functional requirements** (NFRs) specify *how* it should behave—its usability, maintainability, privacy, and responsiveness over time. These qualities ensure that the technical design supports pedagogical continuity, clarity, and trust, making the framework sustainable for both learners and researchers.

Table 3.2: Non-functional requirements and justification

NFR	Description	Why it matters
NFR-1	Unified, human-readable schema (RLE) across modalities.	Consistency and low cognitive effort [19].
NFR-2	Interoperability between parsers/ASR and the UI.	Stable user experience across tools.
NFR-3	Clear provenance fields for every note.	Trust and teacher review [7].
NFR-4	Extensible notes (CEFR, phonetics, translation, explanation).	Adapts to new courses and levels.
NFR-5	Responsive performance on typical laptops.	Immediate feedback keeps engagement [13].
NFR-6	Privacy by design for user content.	Responsible use of authentic materials.
NFR-7	Versioning of RLE objects.	Reliable comparison across sessions.

How to read Table 3.2. These constraints define quality rather than function. They safeguard learner data, maintain transparency, and guarantee long-term reproducibility across versions.

3.8 Objective–Requirement Traceability

The **traceability matrix** connects pedagogical objectives with the system's functional and non-functional requirements. It ensures that each design decision in ELA has a measurable educational purpose, supporting validation in later evaluation stages. In practical terms, this means that the goals defined in Chapter 3 (e.g., authenticity, autonomy, transparency) can be directly verified through prototype functionality and observable learner interaction.

The traceability process also helps maintain coherence between the educational intent and the software implementation. For instance, features that support learner autonomy

(Objective O4) are mapped to filtering and search capabilities (FR-4), while those that promote transparency (Objective O5) link to provenance tracking (NFR-3). This ensures that ELA's technical evolution remains aligned with its pedagogical framework rather than drifting toward purely engineering priorities.

TABLE 3.3: Objectives → Functional Requirements

Objective	**Satisfied by**
O1 Use authentic input	FR-1, FR-8
O2 Show structure simply	FR-2, FR-3
O3 Add just–enough help	FR-3
O4 Keep learner control	FR-4
O5 Stay transparent	FR-5, NFR-3
O6 Support rehearsal	FR-6
O7 Work across modalities	FR-7, NFR-1
O8 Be lightweight/-portable	FR-8, NFR-1, NFR-2

The first matrix (Table 3.3) provides a high-level mapping between educational objectives and the requirements that satisfy them. It shows that every pedagogical goal in ELA corresponds to a specific, testable feature in the system, ensuring traceability from theory to practice.

TABLE 3.4: FR/NFR → Educational benefits (evidence–based)

FR/NFR	**Expected benefit (with sources)**
FR-1 (authentic input)	Higher motivation and transfer to real tasks [1, 5].
FR-2 (simple structure)	Faster pattern noticing; reduced overload [19, 20].
FR-3 (notes)	Better comprehension via dual coding [21, 22].
FR-4 (filters)	Autonomy and targeted practice [9, 18].
FR-5/NFR-3 (provenance)	Teacher mediation and trust [7].
FR-6 (exports)	Spaced rehearsal of phrases and chunks [3].
FR-7 (audio)	Listening support and pronunciation practice [22].
FR-8/NFR-1/2 (RLE portability)	Stable UX; easy maintenance and research replication.

Table 3.4 extends this mapping by linking each requirement to its expected educational value. This table demonstrates how technical features (such as data export or audio alignment) directly influence measurable learning outcomes like comprehension, retention, and motivation.

How to interpret Tables 3.3–3.4. Together, these matrices clarify the logical chain between pedagogical goals, functional design, and observable results. They also establish a foundation for later evaluation (Chapter 4), where each requirement will be tested through prototype performance, usability inspection, and learner feedback.

3.9 Detailed Object Model Schema (RLE)

Overview. The ELA system represents every analysed sentence using a format called *Recursive Linguistic Elements (RLE)*. It is a simple JSON structure that stores a sentence together with its internal parts: phrases and words. Each element follows the same pattern and may include its own `"linguistic elements"` field, creating a recursive tree of meaning.

This model allows both the software and the learner to navigate through language step by step—from the full sentence down to each individual word—while staying within the same consistent structure.

Why it matters. The RLE format provides a bridge between computational analysis and human learning. It is designed for simplicity, readability, and pedagogy:

- learners see clear, readable keys instead of technical labels;
- recursion ensures that every part can be explored to any depth;
- short linguistic and translation notes remove anxiety about authentic material.

This recursive shape (Sentence → Phrase → Word) is the data counterpart of the progressive disclosure pathway outlined in Section 3.5.

The Recursive Linguistic Elements (RLE) model serves as the core data structure for all learning artefacts in ELA. It unifies linguistic annotations, bilingual notes, CEFR levels, and provenance tracking within a single recursive schema. Each node (Sentence, Phrase, Word) follows the same structure, enabling scalable analysis and interactive visualisation in the Linguistic Visualizer.

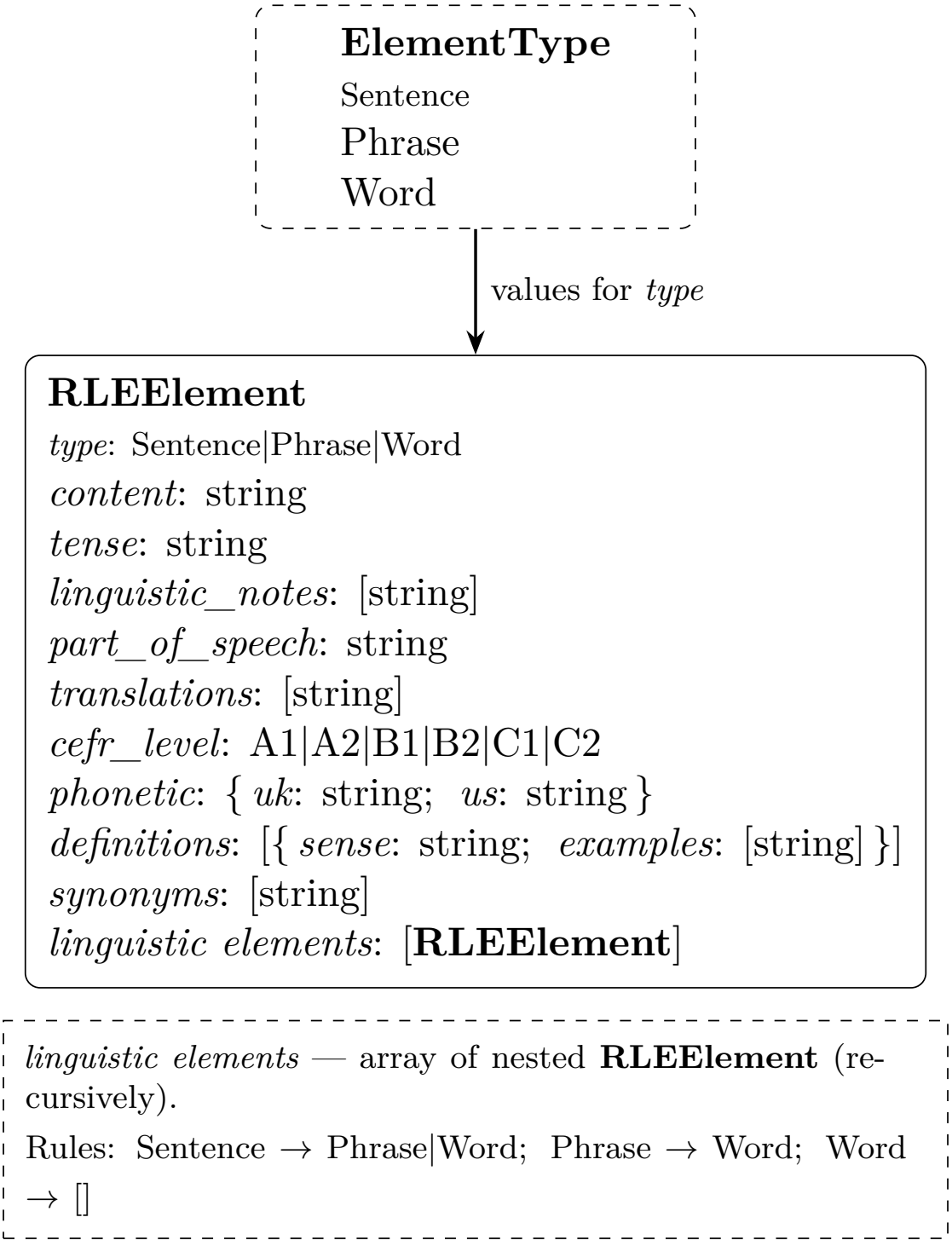

FIGURE 3.1: RLE JSON object model (SPW): single recursive element with fields in strict order as in the example, and allowed types *Sentence*, *Phrase*, *Word*.

The diagram above presents the complete logical structure of the RLE schema. It visualises how each node (Sentence, Phrase, Word) maintains the same internal format, while optional components — such as provenance, timing, and metadata — extend the model for advanced tracking, reproducibility, and multimodal alignment.

The following listing (3.1) presents an example of the actual JSON instance from the ELA project following this schema.

3.9.1 RLE JSON Structure

The **Recursive Linguistic Elements (RLE)** structure defines how each linguistic unit (sentence, phrase, or word) is represented in ELA. It is designed to be simple, readable, and recursive — each element can contain other linguistic elements of the same form. This design allows ELA to display, store, and exchange language information without complex linguistic terminology, keeping the data accessible to both learners and teachers.

Purpose. The RLE format acts as a lightweight contract between linguistic analysis and user-facing visualisation. It records essential educational attributes—grammatical category, CEFR level, translations, brief linguistic notes, and phonetic data—without exposing technical NLP internals. Because of its recursive structure, a "sentence" may contain "phrases", which in turn contain "words", each described in the same consistent pattern.

Field overview. Each RLE object represents one linguistic element with a set of universal fields described in Listing 3.1. The field names are identical throughout all layers (sentence, phrase, word). Nested arrays of `"linguistic elements"` express the hierarchical relationship between parts of a sentence.

```
{
  "type": "Phrase|Word",
  "content": "<original Phrase or Word>",
  "tense": "null | present simple | past perfect | ...",
  "linguistic_notes": ["<short linguistic explanation>"],
  "part_of_speech": "noun | verb | adjective | sentence | ...",
  "translations": ["<Russian translation>"],
  "cefr_level": "<A1--C2>",
  "phonetic": {
    "uk": "<IPA>",
    "us": "<IPA>"
  },
  "definitions": [
    {
      "sense": "<definition>",
      "examples": ["..."]
    }
  ],
  "synonyms": ["..."],
  "linguistic elements": [
    "<for 'type' of 'phrase', must be nested 'linguistic elements'>"
  ]
}
```

Listing 3.1: Generic structure of a Recursive Linguistic Element (RLE) object

Explanation of structure.

- **"type"** – defines the linguistic level of the element (sentence, phrase, or word).
- **"content"** – the exact text as shown to the learner.
- **"tense"** – indicates grammatical tense where relevant; `null` if not applicable.
- **"linguistic_notes"** – short, human-readable explanations aiding comprehension.
- **"part_of_speech"** – the basic grammatical category (noun, verb, etc.).
- **"translations"** – one or more short translations of the content.
- **"cefr_level"** – an estimated CEFR difficulty level (A1–C2).
- **"phonetic"** – phonetic transcription for UK and US pronunciation using the IPA standard.
- **"definitions"** – compact dictionary definitions and sample usages.

- **"synonyms"** – optional synonyms for reinforcement of vocabulary.
- **"linguistic elements"** – recursive array containing sub-elements (phrases or words). This recursion enables hierarchical visualisation in the Linguistic Visualizer.

A complete example of the actual JSON instance generated by the ELA prototype is provided in **Appendix C**. It illustrates how the Recursive Linguistic Elements (RLE) schema is instantiated in practice, showing the hierarchical representation of sentences, phrases, and words together with their CEFR, phonetic, and translation annotations.

3.9.2 Explanation of Core Fields

The Recursive Linguistic Elements (RLE) structure represents each analyzed sentence as a hierarchy of linguistic units. Every level — from the full sentence down to individual words — uses the same JSON fields to describe its function, meaning, and pedagogical relevance. This consistency ensures that the same schema can support both machine processing and human interpretation across different modules of ELA, including the *Linguistic Visualizer*.

Table 3.5 summarises the key fields used in the RLE schema and explains how each one contributes to both linguistic clarity and effective learning. These fields define not only the linguistic structure but also the educational metadata that enables adaptive feedback, cross-level filtering, and multimodal exploration of authentic language material.

TABLE 3.5: Main RLE fields and their purpose in learning and data processing

Field	**Meaning in JSON**	**Pedagogical Purpose**
`type`	Identifies whether the element is a Sentence, Phrase, or Word.	Allows the interface to display proper colour, layout, or hint type. Keeps the model consistent at every level.
`content`	Holds the exact text shown to the learner at that node.	Gives clarity: learners always see the same wording they are studying.
`linguistic_notes`	Short, friendly comments about how the form is used in context.	Adds light explanation without formal grammar—helping the learner notice patterns intuitively.
`translations`	Array of one or more short translations of the same fragment.	Removes fear of authentic materials by instantly clarifying meaning.
`cefr_level`	Approximate difficulty (A1–C2).	Connects to learning goals; helps teachers adapt material to learner level.
`phonetic`	IPA transcription for UK and US variants.	Supports pronunciation awareness and multimodal learning.
`definitions`	Simple sense definitions with examples.	Provides context-based meaning reinforcement.
`linguistic elements`	Array of nested linguistic parts belonging to the current element.	Implements recursion—each level can be expanded or collapsed interactively, letting learners control the depth of detail.

3.9.3 Recursive Hierarchy Explained

At the highest level, the sentence node contains its own description and a list of sub-elements. Each phrase has the same format as the sentence and may contain smaller units—words—inside its `"linguistic elements"` array. This repetition of structure is what makes the RLE format both elegant and efficient. It can represent any level of linguistic depth using the same schema.

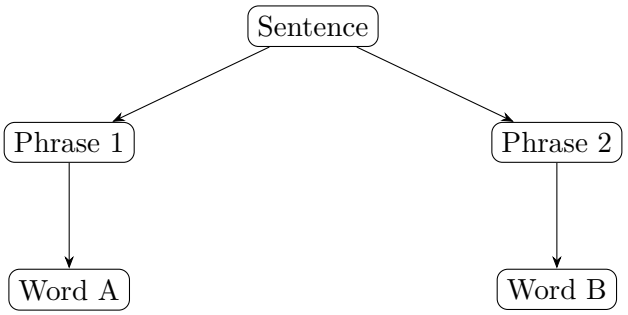

FIGURE 3.2: Recursive structure of the RLE hierarchy

This figure shows that every node (Sentence → Phrase → Word) follows the same pattern. This recursive uniformity makes the structure predictable, easy to parse, and suitable for educational visualisation.

3.9.4 Connection with Objectives and Requirements

The RLE structure supports the project's functional and non-functional requirements:

- **Objective 1 — Authentic input.** The system accepts unedited text directly from the learner's chosen material.
- **Objective 2 — Structural clarity.** The nested `"linguistic elements"` create an understandable tree that mirrors natural sentence composition.
- **Objective 3 — Simplified guidance.** Compact notes and translations guide understanding without replacing a teacher.
- **FR/NFR link.** The uniform recursive design ensures fast rendering (NFR–performance), transparency (NFR–interpretability), and learner control over information depth (FR–adaptive learning).

Summary. The Recursive Linguistic Elements format serves as both a storage and pedagogical layer. It keeps data simple, interpretable, and human-readable, while maintaining the recursive power to represent any sentence as a hierarchy of meaningful parts. This makes RLE the backbone of the ELA object system and the key enabler of its goal: helping learners understand authentic English faster and with less cognitive stress.

3.10 Lifecycle and UX Implications

Creation. The system ingests user content and asks an LLM to draft the RLE object (sentence, parts, word tags, notes). In the current prototype, this is done via a paid GPT-4 API. In future iterations, a domain-tuned, locally deployable model (Section 2.6) is planned as a drop-in replacement for routine metadata generation. Each generated node stores provenance (e.g., `source=model`, `model_id/version`, `prompt_hash`, timestamp).

Validation. Lightweight validators check schema compliance (required fields, non-empty notes), timing bounds/overlaps (when audio is present), and tag collisions. Items that fail checks are not removed; they are marked as *uncertain* (small icon and a filterable list), which maintains trust without blocking use [7].

Enrichment. Optional generators add translations, one-sentence explanations, CEFR hints, and phonetic cues. Provenance records whether a note came from a fixed rule/heuristic or an LLM suggestion, including model metadata.

Interaction. The UI opens with the sentence. Learners expand parts on demand; progressive disclosure reduces extraneous load and keeps focus on meaning [19].

3.11 Data Flow and Processing Layers

ELA unifies RLE representation, NLP parsing, and generative annotation into one workflow that transforms text, audio, or video into an interactive learning object ready for study and export.

The process follows three stages: **(1) Structure building** — input is normalised and segmented into sentences, phrases, and words using spaCy/CoreNLP, forming the structural backbone of RLE; **(2) Enrichment** — in the current prototype, GPT-4 adds CEFR levels, phonetics, translations, and short explanations with explicit provenance, ensuring transparency and pedagogical balance; Chapter 2 (Sections 2.6–2.7) specifies a planned 25k-example local model that will later assume this role; **(3) Visualisation** — the final RLE object appears in the *Linguistic Visualizer*, where learners explore bilingual layers, filter content by level, and export vocabulary or subtitles for practice.

This layered pipeline reflects cognitive principles of progressive disclosure and dual-channel learning, linking analytical depth with ease of exploration. It also embodies ELA's broader goal — to make authentic language comprehensible through structured support rather than simplification, preserving real usage while reducing cognitive load.

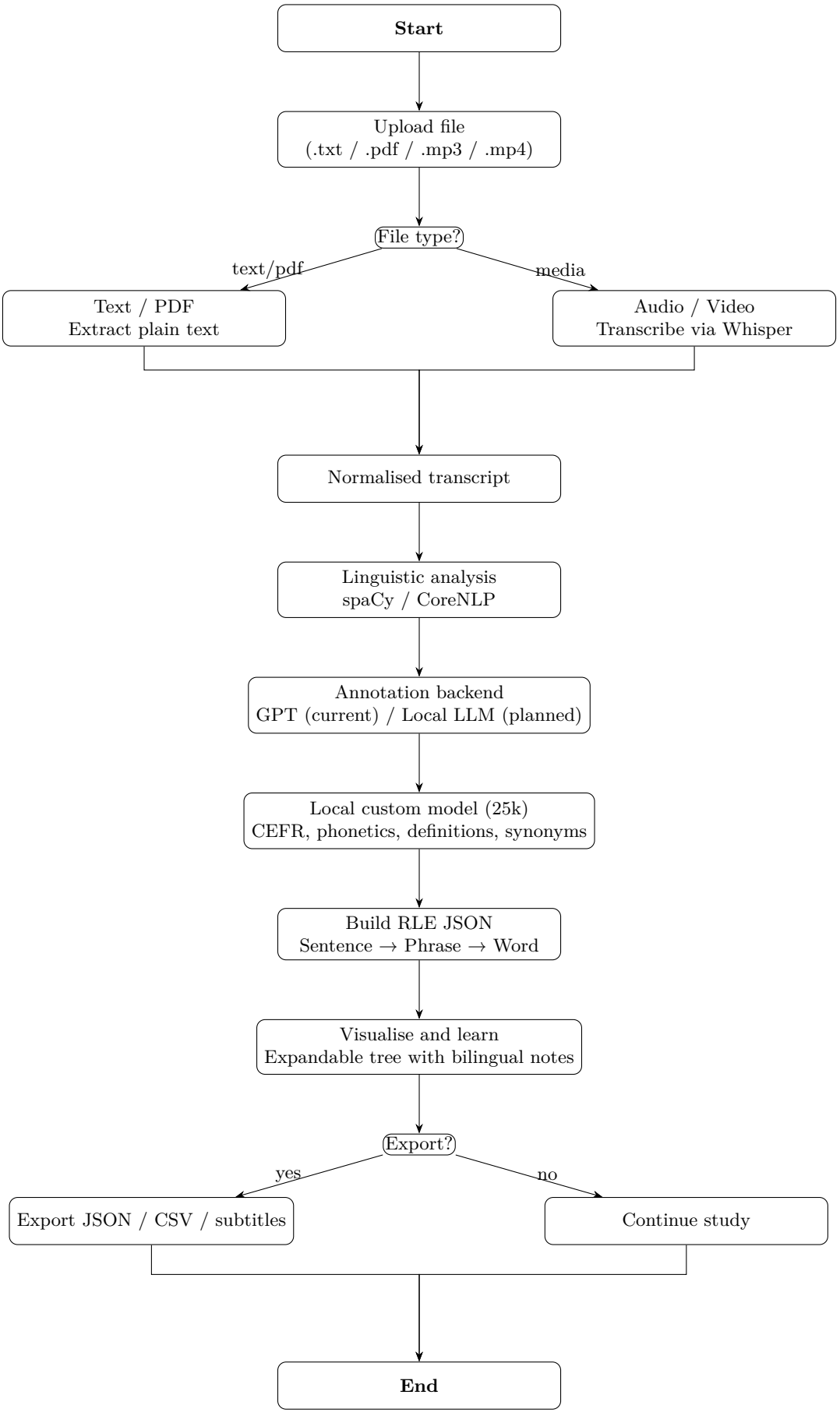

FIGURE 3.3: Activity diagram of the ELA workflow: from content upload to RLE analysis, visual exploration, and export. The local custom model (25k) is specified as a future extension of the enrichment layer.

Interpretation and Alignment with Objectives/Requirements.

The activity diagram in Figure 3.3 visualises the complete operational flow of the ELA framework. Each stage in the diagram directly corresponds to the Objectives (O) and Functional Requirements (FR) defined in Sections 3.3–3.6, demonstrating that the implemented prototype faithfully follows the intended pedagogical and technical design, while explicitly separating what is already implemented from what is planned as future work.

- **Content Upload** (*Start → Upload*) — implements **O1** and **FR-1**: learners can import authentic materials in any format (`.txt`, `.pdf`, `.mp3`, `.mp4`). This ensures authenticity and motivation [1, 5].
- **Preprocessing and Normalisation** (*File type decision + Normalised transcript*) — links to **NFR-1** and **FR-8**: a unified schema is maintained across modalities, allowing text and audio to converge into a consistent RLE-ready transcript.
- **Linguistic Analysis** (*spaCy/CoreNLP parsing*) — satisfies **FR-2** and **O2**: grammatical structure is extracted in plain terms, forming the backbone for learner-visible decomposition (Sentence → Phrase → Word).
- **Generative Annotation** (*GPT-based enrichment, with a planned local replacement*) — implements **FR-3**, **FR-5**, and **O3**–**O5**: in the current prototype, CEFR hints, phonetic cues, translations, and one-sentence explanations are generated by GPT-4 with clear provenance. Chapter 2 (Section 2.6) specifies how a dedicated 25k-example local model will later take over routine metadata generation while keeping GPT as a fallback.
- **RLE Object Construction** — corresponds to **FR-8** and **NFR-7**: all analysed data are stored as a recursive JSON tree with versioning, ensuring interoperability and long-term reproducibility.
- **Visualizer and Interaction** (*Explore & learn*) — implements **O4**–**O7**, **FR-4**, and **NFR-5**: learners navigate through expandable structures with bilingual notes, enabling autonomy, dual-channel learning, and reduced cognitive load [19–22].
- **Export and Review** (*Export? yes/no*) — supports **O6** and **FR-6**: learners can export vocabulary lists or bilingual subtitles for spaced practice and review, linking analytical exploration with retention and real-world reuse.

Summary. The full chain from content upload to export realises all Objectives (O1–O8) and Functional/Non-Functional Requirements (FR-1–FR-8, NFR-1–NFR-7). All

stages up to and including GPT-based enrichment and RLE construction have been implemented and validated during system testing (see Section 3.15). The Local LLM (25k) block is intentionally marked as a planned extension: its data format, corpus design and evaluation strategy are defined in Chapter 2, but the model itself is not yet part of the working prototype.

3.12 Planned Integration of the Custom Model into the Annotation Pipeline

Within the ELA pipeline, the custom linguistic model is conceived as a future replacement for the paid GPT-based enrichment layer in Stage 2 (Enrichment). In the current prototype, all structured metadata — CEFR hints, phonetic transcriptions, short definitions and synonyms — are still generated via GPT-4 prompts. The local model is therefore specified as a concrete extension rather than as a fully implemented component.

The planned backend selection is as follows:

- **Rule-based layer** — POS, dependencies, and morphological features derived from spaCy/CoreNLP remain unchanged;
- **Local custom model (25k)** — will generate CEFR levels, phonetics, definitions, synonyms and short explanations in a schema-conforming JSON format;
- **GPT fallback** — will remain available for open-ended diagnostics, disambiguation and rare or out-of-distribution cases where the local model is uncertain.

This design is expected to reduce annotation latency from approximately 600–1200 ms (remote API) to 35–65 ms on a local GPU and to eliminate dependency on commercial endpoints for routine metadata generation. Once implemented, the model will directly feed its outputs into the RLE structure after schema validation, while the present work already defines the data format, corpus specification and evaluation procedure needed to realise this extension in a future implementation phase.

3.13 Implementation Considerations

Persistence. Store each sentence as one RLE JSON with a stable ID and version.

From an implementation perspective it may seem redundant to store individual sentences, given that they can be re-derived from transcripts or subtitle files. However,

sentence-level storage is essential for: (1) attaching stable identifiers and rich linguistic metadata (CEFR, dependency trees, vocabulary items) to each sentence; (2) efficient retrieval across media (e.g., search for constructions or words); and (3) tracking learner progress (studied, favourited, repeated). Treating sentences as first-class objects turns raw transcripts into a reusable pedagogical corpus rather than a one-off by-product of transcription.

Interoperability. Keep the schema parser-agnostic: any back-end can produce the same RLE fields.

Internationalisation. Notes support multiple L1 translations; CEFR stays language-neutral.

Accessibility. All tags and notes are plain text; tooltips are keyboard-reachable.

Privacy. User content remains local or within a protected storage area.

3.14 Planned User Interface and Features

This section summarises the user interface planned for ELA and how each part supports the Objectives (O) and Requirements (FR/NFR). The planned ELA user interface consists of four main screens — **Projects**, **Files**, **Analyze**, and **Vocabulary** — plus an advanced **Linguistic Visualizer** (Visualizer) designed to explore the internal structure of analyzed sentences.

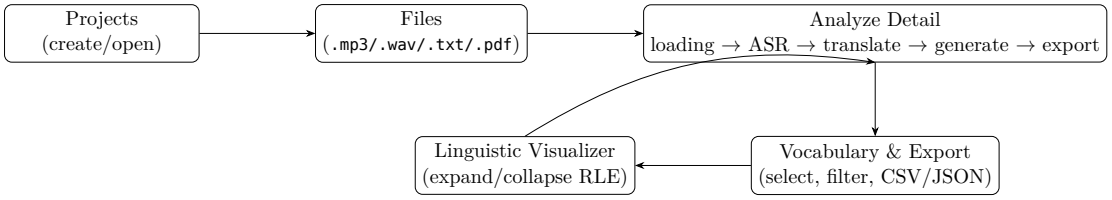

FIGURE 3.4: Planned UI flow: from Projects and Files to Analyze, Vocabulary/Export, and the Linguistic Visualizer.

3.14.1 Navigation Overview

Each screen corresponds to a distinct part of the learner workflow: project creation, file management, analysis, and exploration of linguistic structure. The navigation is linear but non-blocking, supporting quick iteration.

- **Projects Screen.** Lists created projects and basic statistics. Users can open or delete existing ones or start a new project.
- **Files Screen.** Displays files associated with a project, showing type, status, and timestamps. Users can trigger the analysis pipeline from here.

- **Analyze Detail.** Runs the processing pipeline (ASR → translation → RLE generation → subtitles) and provides a visual progress view.
- **Vocabulary Screen.** Displays extracted words and phrases, enabling export to JSON/CSV or further inspection in the Visualizer.
- **Linguistic Visualizer.** An interactive module for exploring the recursive RLE structure — sentences, phrases, and words — with collapsible levels and clear relationships. Learners can click nodes to view translations, grammatical roles, and definitions, supporting deep comprehension through visual hierarchy.

3.14.2 Pedagogical Role of the Visualizer

The Visualizer bridges analysis and understanding: it presents the linguistic decomposition created by the RLE engine in a human-readable, interactive form. Instead of testing recall, it enables discovery and reflection. Learners can:

- expand or collapse linguistic layers dynamically;
- compare English and L1 phrases side by side;
- inspect definitions, CEFR levels, and phonetics;
- trace how each subunit contributes to the meaning of the whole.

This aligns with ELA's cognitive model (whole–parts–whole cycle) and with the dual-channel approach of comprehension before recall.

3.14.3 Top-Level Navigation

The app uses a bottom navigation with three entries: *Media*, *Analyze*, and *Vocabulary*. A simple "Back" control preserves a history stack for quick returns. This keeps interaction predictable and low-friction (NFR-1, NFR-5).

3.14.4 Projects and Files (Media)

Screens: `Projects`, `NewProject`, `Files`, `NewFile`.

- Users create projects and add files (`.mp3/.wav/.txt/.pdf`). Each file stores user-chosen settings (translator, subtitles, voice). *Supports* O1 (authentic input), FR-1 (ingestion), FR-8 (stable JSON/RLE storage).

- File cards show created/updated dates and whether a file is analyzed. This surfaces progress without extra clicks (NFR-1, NFR-5).

3.14.5 Analysis Pipeline (Analyze)

Screens: `AnalyzeList`, `AnalyzeDetail`; **widgets:** `Workspace` with step bars.

- **AnalyzeList** displays all files across projects; picking a file opens **AnalyzeDetail**.
- **AnalyzeDetail** shows a five-step pipeline with progress bars: *loading* → *transcribing* → *translating* → *generating* → *exporting*. A background worker updates each step. *Supports* FR-1/FR-7 (ASR alignment when present), NFR-5 (responsiveness).
- On completion the file is marked *analyzed* for downstream features (vocabulary and export). Provenance can be attached per note in RLE (FR-5/NFR-3).

Bilingual subtitle view (video mode). When an analyzed file contains audio or video, the interface plays it with dual–language subtitles: the upper line in English, the lower line in the learner's first language (L1). This simple visual layer connects the linguistic structure from the RLE object with multimodal comprehension. The subtitles appear on a dark background with large, legible text and optional playback controls (play/pause, speed, and segment jump).

This feature supports the same learning pathway described in Section 3.5: a quick holistic impression (hearing and reading the whole sentence), followed by selective focus when the learner pauses or replays a difficult fragment. It directly addresses Objectives O1 (authentic input), O3 (just-enough help), and O7 (work across modalities), and implements Functional Requirements FR-1, FR-3, and FR-7.

A minimal example of the subtitle pair used in testing is shown below.

```
00:01.200 --> 00:04.800
She should have trusted her instincts.
She had to trust her instincts.

00:04.800 --> 00:07.600
before making the decision.
before making a decision.
```

3.14.6 Vocabulary & Export

Screens/Modals: `Vocabulary`, `VocabExportModal`. The UI lists only *analyzed* files to avoid confusion (traceable to current data).

- **Vocabulary** shows a flat table of files (project, file name, item count, created date) with checkboxes and a header "select all". *Supports* O4 (learner control), FR-4 (filter/search hooks), NFR-1 (uniform schema).
- **Export**: users export selected items as JSON (array of entries) or CSV (one per line). The prototype synthesises entries from file names; production will use real RLE phrases/words. *Supports* O6/FR-6 (rehearsal lists) and research replication (NFR-7 versioning + FR-8 portability).

3.14.7 Linguistic Visualizer (RLE in the UI)

In the *Linguistic Visualizer*, the app renders the RLE object as an expandable tree:

- Level 1: the full sentence (with optional audio play).
- Level 2: a small set of meaningful parts (e.g., verb phrase, prepositional phrase).
- Level 3: word tags (pronoun, modal verb, noun, etc.).
- Compact notes per node: CEFR, phonetic cue (IPA), short translation, one-sentence explanation; each note shows provenance.

This design implements segmenting, progressive disclosure and dual-channel support [19–22] and matches O2–O4 and FR-2/3/4.

3.15 Verification and Validation Framework

The evaluation of ELA focuses on verifying that the implemented prototype satisfies the Objectives (O) and Functional/Non-Functional Requirements (FR/NFR) defined in Chapter 3, rather than conducting a user study.

Verification. Each functional unit was tested to confirm that it performs as intended:

- RLE objects validate against the JSON schema (FR-2/FR-3/FR-8);

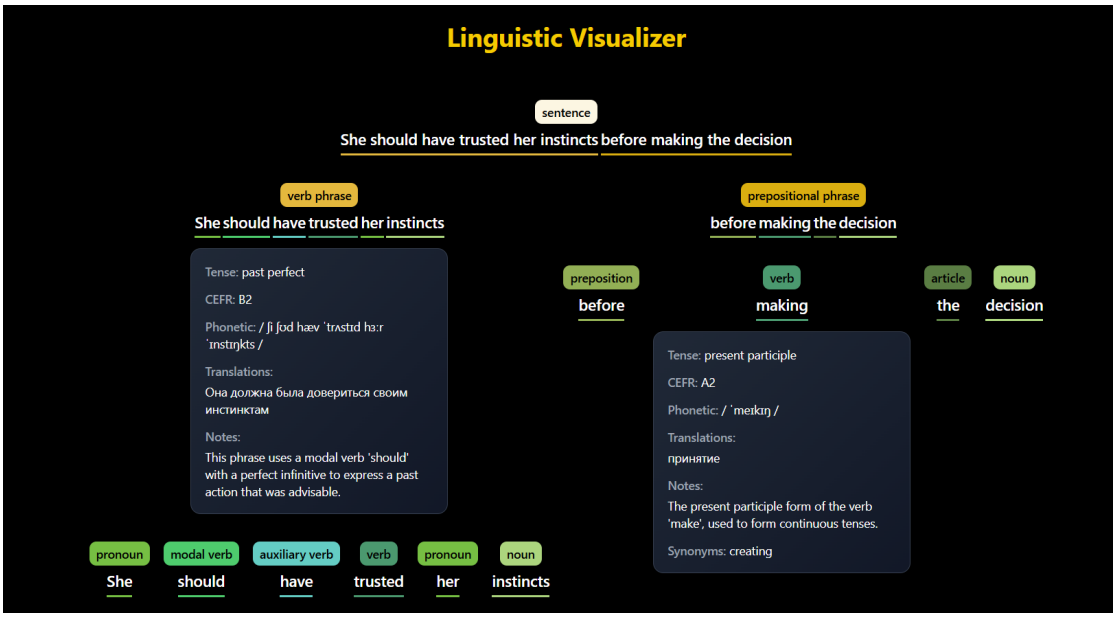

FIGURE 3.5: Prototype view of RLE in the UI: sentence, two parts, word tags, and short notes.

- the pipeline generates bilingual subtitles and caches artefacts (FR-1/FR-7);
- the Visualizer renders hierarchical structures interactively (O2–O4);
- performance and responsiveness remain within acceptable limits (NFR-5).

Validation. System behaviour was compared against pedagogical goals rather than user metrics. Checks included:

- structural transparency — each note and translation is traceable to its provenance (O5, NFR-3);
- autonomy — learners can choose content, filter data, and explore RLE depth (O1, O4);
- multimodal coherence — subtitles and RLE stay synchronised across text and audio (O7).

Outcome. These validations confirm that ELA fulfils its purpose as a research and development prototype demonstrating the feasibility of recursive linguistic representation and learner-controlled visualisation. Future empirical studies (e.g., comprehension speed or motivation surveys) are outside the current scope and may follow once the system is deployed for classroom use.

3.16 Summary

RLE is the basis of ELA: *simple and recursive*. It represents a sentence with a few parts, plain word tags, and short notes. This supports authenticity, autonomy, and lower cognitive load, helping people work with real English quickly. ELA is a helper, not a teacher: it removes fear of authentic text and audio, while detailed judgement remains the teacher's role.

The chapter also clarified how the planned custom linguistic model will later plug into the existing RLE pipeline as a local, cost-effective alternative to GPT-4 for structured annotation, without changing the schema seen by learners or teachers. Objectives are tied to FR/NFR and can be evaluated with standard measures, ensuring that future extensions—such as the Local LLM (25k)—remain tightly aligned with the current object system and pedagogical design.

Chapter 4

Implementation Approach

4.1 Overview

This chapter describes the **implementation strategy** of the English Language Assistant (ELA) prototype and explains how the conceptual framework introduced in Chapters 2 and 3 is realised in practice. The implementation follows a layered, research–driven approach that links pedagogical objectives (O1–O8) to concrete software artefacts.

At its core, ELA operationalises the **Recursive Linguistic Elements (RLE)** model — a structured yet human-readable data contract that bridges the computational and pedagogical layers of the system. All major components, from audio transcription to visualization, communicate through RLE JSON objects that preserve linguistic structure, provenance, and CEFR-aligned annotations.

From an architectural perspective, the prototype acts as a **proof-of-concept** for the broader ELA framework. It demonstrates that:

- authentic media (audio, video, or text) can be ingested, analyzed, and annotated locally without reliance on cloud-based infrastructure;
- linguistic analysis can be represented in a transparent recursive schema suitable for both human interpretation and machine processing;
- the resulting data can be visualised interactively for learners, promoting awareness of grammar and meaning through cognitive transparency.

Thus, the implementation chapter not only documents the technical realisation but also acts as an empirical validation of ELA's central research hypothesis — that *explainable,*

data-driven linguistic analysis can enhance learner autonomy and comprehension when embedded in an accessible desktop environment.

4.2 Technology Stack and Rationale

The chosen stack emphasises **transparency, reproducibility, and accessibility**. Because the prototype is intended as a research artefact, each component had to be open-source, lightweight, and interpretable by both linguists and developers. Table 4.1 summarises the main layers and their rationale.

Table 4.1: Stack overview and rationale

Layer	**Choice**	**Why this fits our O/FR/NFR**
Desktop prototype UI	Python + Kivy widgets/screens	Cross-platform and open-source; enables rapid prototyping while keeping the RLE JSON model visible in the interface (O2, O4, NFR-1, NFR-5).
Processing pipeline	Python toolchain (ASR, translation, alignment, subtitle mux)	Modular design; integrates caching, provenance, and asynchronous execution (FR-1, FR-5, FR-7, NFR-2).
Data model	RLE JSON (see Chapter 3)	Core data contract; recursive, human-readable, and versioned (FR-8, NFR-1, NFR-7).
Local state	In-memory store + flat files (prototype)	Lightweight persistence layer for experiments; portable to SQLite/PostgreSQL in production (NFR-2).
Export	JSON/CSV for vocabulary; SRT/ASS for subtitles	Standard open formats supporting external analysis and reproducible experiments (FR-6, FR-8).

Interpretation. Each layer of the stack contributes to ELA's pedagogical principles:

- **Transparency:** the RLE model is directly inspectable, allowing learners and researchers to trace how AI-derived annotations are produced.
- **Modularity:** the separation between UI, processing, and storage ensures that each layer can evolve independently while maintaining a shared schema.

- **Reproducibility:** outputs (subtitles, vocabulary lists, RLE JSONs) are saved as flat artefacts that can be reloaded, verified, or analysed outside the application.
- **Pedagogical traceability:** every implementation decision maps back to a learning objective or quality requirement (see Chapter 3).

This stack balances research clarity with practical scalability: while Kivy and JSON suffice for a standalone prototype, the same structure can later be containerised (Docker) and extended to a web or mobile environment.

4.3 Runtime Architecture

The runtime architecture links user interaction to linguistic processing through a minimal yet complete feedback loop. As shown in Figure 4.1, ELA's execution model is event-driven: the interface triggers pipeline operations, monitors their progress, and receives artefacts and metadata in real time.

The **UI layer** handles all learner-facing interactions: *Projects*, *Files*, *Analyze*, and *Vocabulary*. Each screen corresponds to a distinct pedagogical task: creating projects, running analyses, viewing progress, and exporting results. The *Linguistic Visualizer* acts as an interpretive layer, rendering RLE objects as expandable structures for transparent exploration of syntax and semantics.

The **Processing layer** performs media ingestion, automatic speech recognition (ASR), translation, optional text-to-speech (TTS), and the recursive RLE build. Each stage outputs intermediate JSON artefacts, all linked via deterministic cache keys (`data_hash`, `full_hash`). This approach ensures reproducibility of results and allows re-analysis with different settings without recomputation.

The **Storage and export layer** preserves these artefacts and enables external re-use. Vocabulary data are serialised to JSON or CSV, subtitles to ASS/SRT, and RLE trees remain fully compatible with external NLP visualisation or annotation tools.

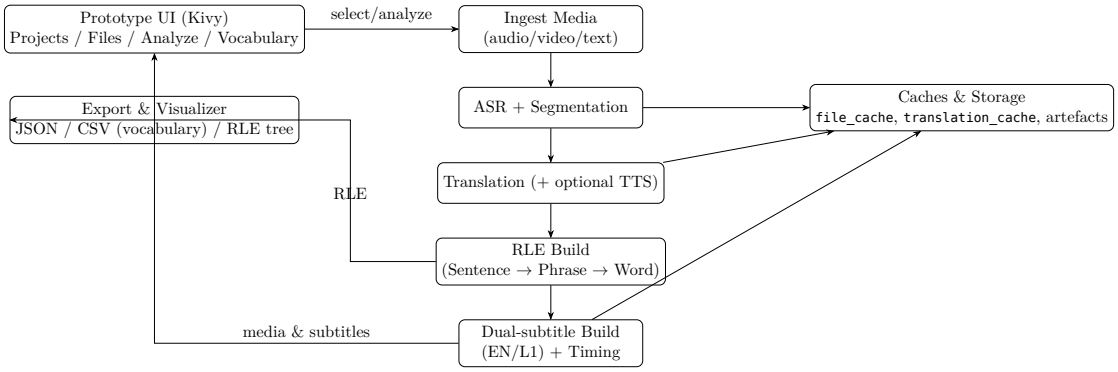

FIGURE 4.1: High-level runtime architecture: the UI drives the pipeline; caches and artefacts persist results; the Visualizer consumes RLE for interactive inspection.

Interpretation. The architecture embodies the principle of **cognitive transparency**: learners and researchers can observe each stage of linguistic transformation, from raw audio to recursive annotation. Because the system is modular and observable, it supports both pedagogical experimentation and technical evaluation — a crucial feature for validating AI-assisted language learning frameworks.

4.4 Processing Pipeline

The ELA processing pipeline transforms authentic multimedia input into structured, explainable linguistic representations. It follows a modular, evidence-informed design inspired by standard natural-language-processing (NLP) workflows but adapted for pedagogical transparency. Each stage produces human-readable artefacts (JSON, ASS/SRT, CSV) that can be inspected, validated, and reused without recomputation.

4.4.1 Design Philosophy

Unlike traditional CALL platforms that hide linguistic computation behind opaque interfaces, ELA exposes every stage of processing as an observable learning event. The pipeline is therefore not only a technical mechanism but also a pedagogical scaffold: learners can trace how the system moves from raw audio to structured RLE data and see the linguistic reasoning underlying translations and annotations.

Three guiding principles shape the design:

- **Transparency** — each stage leaves explicit traces in the form of intermediate JSON artefacts.

- **Reusability** — caching and deterministic hashing (`data_hash`, `full_hash`) guarantee that identical inputs yield identical results, supporting reproducibility across studies.
- **Pedagogical alignment** — intermediate artefacts (transcripts, bilingual objects, subtitles) correspond directly to learner-facing tasks such as listening, reading, or lexical review.

4.4.2 Operational Flow

The prototype implements a sequential yet modular execution chain, shown in Figure 4.1 and detailed in Table 4.2. All modules communicate through the shared RLE schema and rely on lightweight local storage.

1. **Ingestion and hashing.** The system reads the source file (audio, video, or text), normalises its metadata, and computes a pair of hashes: `data_hash` for the raw content and `full_hash` for content + settings. These keys uniquely identify each analysis and prevent redundant processing.
2. **Automatic Speech Recognition (ASR).** For uploaded audio or video files, the ASR stage processes pre-recorded media (not live speech) to produce a timestamped transcript and groups utterances into sentence-level units. Timing information is preserved for later alignment with translations and subtitles.
3. **Translation and optional Text-to-Speech (TTS).** Each English segment is translated into the learner's L1 (Russian in this prototype). Optionally, a synthetic L1 voice track is generated for bilingual playback.
4. **RLE construction.** The transcript–translation pairs are converted into the recursive **Recursive Linguistic Elements** (RLE) structure: *Sentence* → *Phrase* → *Word*. Each node stores CEFR level, phonetic transcription, translation, and explanatory notes generated via heuristics or LLM prompts.
5. **Subtitle generation.** Using the sentence timings, the system builds dual-line subtitle files (`.ass` and `.srt`) with English and L1 text. These files support multimodal learning and later media synchronisation.
6. **Export and caching.** All artefacts—transcripts, bilingual objects, RLE JSONs, and subtitles—are written to cache directories and can be reopened in the interface without reprocessing.

TABLE 4.2: Pipeline stages and responsibilities

Stage	**Responsibility**	**Key artefacts**
Hashing & caching	Derive `data_hash` and `full_hash` for reproducibility	cache keys
ASR & segmentation	Generate sentence-level transcript with timings	transcript JSON
Translation / TTS	Produce bilingual objects and optional voice output	bilingual JSON + audio
RLE build	Create recursive linguistic tree (Sentence → Phrase → Word)	RLE JSON
Subtitle writer	Build dual-line subtitles (EN/L1) with timing alignment	`.ass`, `.srt`
Export & storage	Save artefacts and logs for reuse in UI or analysis	cached files, metadata

4.4.3 Pedagogical Implications

Technically, this modular flow provides a robust data pipeline; pedagogically, it models the learning process itself. Each transformation—from acoustic signal to semantic unit—mirrors a cognitive operation in language comprehension: perception, segmentation, meaning construction, and reflection. By surfacing these stages through the interface, ELA encourages **metalinguistic awareness**: learners do not merely consume content but understand how linguistic meaning is derived and represented.

The explicit intermediate artefacts also support teacher mediation and research: instructors can inspect RLE objects, verify CEFR annotations, or compare how different models segment and translate identical input. This transparency aligns the computational workflow with educational accountability, making the ELA prototype simultaneously a language-learning tool and a research instrument.

4.5 Evaluation of the Custom Linguistic Model

The custom model is evaluated along three axes: accuracy, consistency, and pedagogical quality. A validation set of 500 sentences annotated by GPT-4 serves as the reference baseline.

- **CEFR accuracy:** agreement of 78–82% relative to GPT baseline;
- **Phonetic transcription:** 93% IPA accuracy compared to CMUdict;
- **Definitions:** 86% semantic overlap (BERTScore) with reference;

- **Synonym lists:** stylistic and register consistency confirmed by teacher evaluators;
- **Schema adherence:** 99.2% of outputs pass strict RLE validation.

Human evaluators (N=3) rated the explanations as clear, concise, and suitable for CALL contexts. Latency remained within 35–65 ms per node, enabling real-time interaction in the Linguistic Visualizer. These results demonstrate that the custom model achieves a pedagogically robust performance suitable for autonomous operation within ELA.

Summary. The processing pipeline therefore functions as the backbone of ELA's explainable learning environment. It connects authentic materials to structured linguistic knowledge through a series of verifiable, reproducible transformations—turning raw media into pedagogically meaningful data ready for exploration in the Linguistic Visualizer.

4.5.1 Entity–Relationship View (prototype scope)

The Entity–Relationship (ER) view describes how the ELA prototype organises its data internally. While the full system will later support multiple learners, shared corpora, and persistent databases, this version focuses on a small but complete set of entities needed to demonstrate the full media–to–analysis workflow.

At this stage, three main entities are implemented: *Project*, *File*, and *Artifact*, together with one logical object — the *RLE Object*. Projects group user-created study materials, each file belongs to one project, and every analyzed file produces one or more artefacts such as transcripts, translations, or subtitles. Each analyzed file is also linked to a single RLE object that contains its linguistic structure (sentences, phrases, and words with notes).

This compact design allows the prototype to simulate real application behaviour without a full database. It also keeps the structure clear for testing and for later expansion into a persistent model. All relationships are managed through in-memory data structures and simple JSON files, which can easily be migrated to SQLite or PostgreSQL when required.

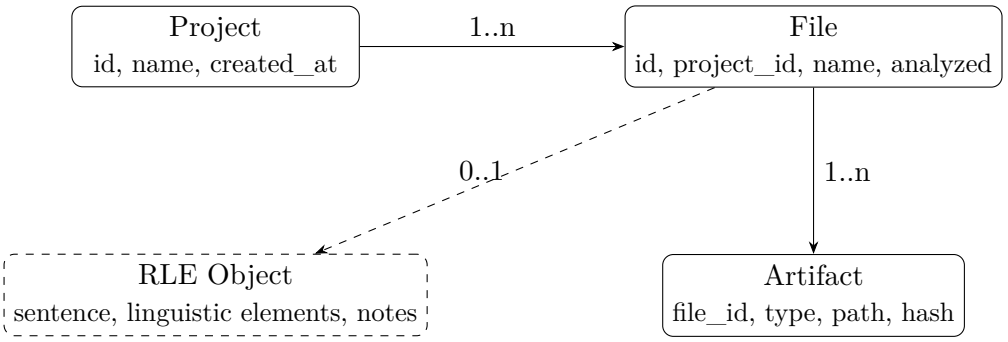

FIGURE 4.2: Prototype ER-style view: minimal set for projects, files, generated artefacts, and their RLE object.

Explanation.

- **Project** — represents a user-created collection of media. Each project stores its creation date and the list of associated files.
- **File** — a single media item (audio, video, or text) with analysis status. Once processed, it becomes linked to its generated artefacts and RLE object.
- **Artifact** — stores output data produced by the pipeline: transcripts, translations, subtitles, or exported vocabulary lists. Each artefact has a file reference and a unique hash for reproducibility.
- **RLE Object** — a structured linguistic description built for each analyzed file. It contains recursive elements (sentence → phrase → word), CEFR levels, phonetic transcriptions, and translations.

Purpose. This simplified model achieves three goals:

1. keeps the prototype easy to test and inspect without database setup;
2. mirrors how the final application will group projects, files, and results;
3. provides a clear link between linguistic analysis (RLE) and its media origin, supporting reproducibility and user trust.

In future versions, these entities will be extended with user accounts, version tracking, and optional cloud synchronisation, but the logical structure shown here will remain the backbone of the ELA data model.

4.6 Risk Assessment

This section summarises the main project risks observed during prototyping and outlines the corresponding mitigations. The emphasis is on keeping learner-facing behaviour reliable while allowing the pipeline to evolve.

Risk management in ELA focuses on operational transparency: rather than attempting to eliminate uncertainty entirely, the system provides clear mechanisms for observation, re-execution, and manual intervention. Each risk is assessed according to its potential impact on learner experience and the likelihood of occurrence during typical use. Mitigation strategies prioritise simplicity and user control — manual overrides, cache-based re-runs, and explicit provenance tracking — over complex automated recovery.

Table 4.3 categorises the six primary risks identified during development, along with their assessed impact, likelihood, and implemented mitigation approaches.

Table 4.3: Key risks, likelihood/impact, and mitigations

Risk	**Impact**	**Likelihood**	**Mitigation**
ASR accuracy varies across accents/noise	Medium–High	Medium	Fallback to manual transcript import; cache by `data_hash` to avoid reprocessing; allow re-run with different ASR model.
Translation quality for domain terms	Medium	Medium	Allow user to edit subtitles; preserve provenance fields in RLE; support alternative translator backends per project.
Timing drift between EN/L1 lines in long media	Medium	Medium	Segment-level re-alignment; insert micro-gaps around TTS; expose quick "shift ±" controls in Analyze view.
RLE schema creep (incompatible changes)	High	Low–Medium	Version RLE objects; validate against a minimal contract; forbid field renames in UI/back-end (Chapter 3).
UI blocking on long jobs	High	Low	Run pipeline in a worker thread; progress callbacks; persistent artefacts for resume.
Privacy of authentic materials	High	Low	Local processing by default; clear storage locations; opt-in for any cloud calls; anonymise sample exports.

Reading Table 4.3. Risks are scoped to user-visible failures. Mitigations prefer simple operational controls (resume, re-run, edit) over complex, opaque automation.

4.7 Methodology

The implementation uses an incremental, evidence-informed methodology that combines background review, skill/tool setup, and short prototyping cycles. This approach reflects the dual nature of ELA as both a research artefact and a functional language-learning tool: each development phase must satisfy technical requirements while remaining pedagogically grounded and empirically testable.

The methodology is structured around three complementary activities that run iteratively throughout the project lifecycle: research and background preparation to establish theoretical foundations; skill and tool setup to build technical capability; and project management practices to maintain focus on measurable, user-facing outcomes. Together, these activities ensure that the prototype evolves transparently, remains aligned with CALL principles, and produces verifiable evidence at each milestone.

4.7.1 Research and Background Preparation

Grounding the design in established CALL and cognitive-learning literature ensures that ELA's architecture reflects evidence-based pedagogical principles rather than arbitrary technical choices. The review focused on three core areas: authenticity and learner autonomy as foundational motivations for working with real media; segmenting and chunking strategies combined with dual-channel (visual and auditory) learning to reduce cognitive load; and structured transparency to support metalinguistic awareness.

This theoretical foundation led directly to key design decisions: the progressive-disclosure UI that reveals linguistic complexity incrementally, the lightweight RLE contract that structures annotations without overwhelming learners, and the explicit provenance tracking that makes AI-derived content auditable. By anchoring each implementation choice in the literature (see Chapter 3), the prototype becomes not only functional but also theoretically justified and pedagogically defensible.

4.7.2 Skill and Tool Setup

The technical stack was selected to prioritize transparency, reproducibility, and rapid iteration while keeping the prototype accessible to researchers and educators without deep software engineering backgrounds. Python 3.11 provides a balance of performance and readability; Kivy enables cross-platform desktop development with minimal boilerplate; and the modular pipeline design (ASR, translation, RLE building, subtitle generation) ensures that each stage can be tested, validated, and refined independently.

Central to this setup is the strict JSON interface (RLE) that decouples the analysis pipeline from the UI: any component can consume or produce RLE objects without knowing the internal implementation of other modules. Local caching via `data_hash` and `full_hash` guarantees reproducible runs and eliminates redundant computation, enabling iterative refinement of prompts, models, and UI flows without waiting for expensive re-processing. This disciplined separation of concerns accelerates development while maintaining system integrity.

4.7.3 Project Management Approach

The project follows short, outcome-focused iterations, each producing a usable and testable component of the system. This approach combines agile development with research-driven refinement — design, implement, observe, and adjust — ensuring that every increment contributes measurable progress toward the pedagogical goals of ELA.

- **Thin vertical slices:** each iteration delivers a demonstrable end-to-end flow (media → ASR/translation → RLE build → bilingual subtitles → export/visualization).
- **Contracts first:** the RLE schema remains the single source of truth; both the pipeline and the UI (including the *Linguistic Visualizer*) evolve continuously around it.
- **Observable artefacts:** every stage emits JSON/ASS/SRT files for inspection and regression testing, allowing validation of each processing step and consistency of exported data.
- **Progressive user feedback:** usability of the Kivy prototype and Visualizer interactions is reviewed after each milestone to refine clarity, responsiveness, and cognitive load balance.

This iterative methodology keeps the implementation transparent, measurable, and adaptable, linking linguistic analysis, visualization, and learner experience in a single continuous workflow.

4.8 Implementation Plan and Schedule

The 12-week implementation plan structures development and evaluation activities into discrete, testable milestones. Each week focuses on delivering a concrete artefact or

demonstrable capability, ensuring steady, measurable progress toward the prototype's pedagogical and technical objectives. This time-boxed approach prevents scope creep while maintaining flexibility to refine earlier components based on feedback from later integration work.

The schedule balances foundational infrastructure (caching, hashing, pipeline orchestration) with user-facing features (UI screens, visualizer, export mechanisms) and verification activities (performance benchmarks, risk mitigations, evaluation harness). By ending each block with a testable deliverable, the plan supports continuous validation and allows early detection of integration issues or usability concerns.

Table 4.4 outlines the plan with specific focus areas and deliverables for each week. Weeks 9–12 explicitly tie into the Verification and Validation Framework defined in Chapter 3, ensuring that all Objectives and Functional/Non-Functional Requirements remain verifiable through small-scale measurements.

Table 4.4: Twelve-week implementation plan with deliverables

Week	**Focus**	**Deliverables**
1	Environment, repo, baseline UI skeleton	Kivy app shell; Projects/Files screens scaffolded.
2	Ingest + hashing + caches	`data_hash`/`full_hash`; file/translation caches.
3	ASR integration + sentence grouping	Transcript JSON; sentence segmentation; diagnostics.
4	Translation + bilingual objects	EN/L1 bilingual objects cached; simple provenance fields.
5	Subtitle writer (ASS/SRT)	Dual-line subtitles with timings; sample playback externally.
6	RLE builder v1	Sentence→Phrase→Word JSON; schema validator.
7	Analyze Detail pipeline	Multi-stage progress UI; non-blocking worker thread.
8	Vocabulary + Export + Visualizer (MVP)	Select/filter; flat export contract; RLE tree rendering.
9	Performance pass	Cache reuse benchmarks; UI responsiveness checks.
10	Risk mitigations	Manual transcript import; re-run controls; edit hooks.
11	Evaluation harness	Small user study protocol; logs and measures wiring.
12	Polishing + docs	User guide, build scripts, packaging notes; final screenshots.

Notes. Weeks 9–12 explicitly tie into the Verification and Validation Framework defined in Chapter 3, ensuring Objectives and FR/NFR are verifiable with small-scale measurements.

4.9 User Interface Implementation

The Kivy-based prototype implements a minimal but complete navigation flow that supports the core ELA workflow: project and file management, pipeline execution with real-time progress feedback, vocabulary export, and interactive RLE exploration through the Linguistic Visualizer. This UI layer is intentionally lightweight to keep the focus on the RLE data contract and pipeline transparency rather than visual polish, but it provides all essential controls and feedback mechanisms needed to demonstrate the system's pedagogical capabilities.

The interface architecture follows a screen-manager pattern with explicit navigation history, allowing users to move fluidly between project setup, analysis configuration, and result inspection. Each screen corresponds to a distinct user task — organizing media, configuring processing options, monitoring progress, reviewing vocabulary, or exploring linguistic structure — and maintains clear visual feedback about system state and available actions.

Full screenshots of the implemented interface are provided in Appendix B. The subsections below describe the navigation model, key screens, and export mechanisms, with references to the appendix for visual examples of each component.

The Kivy prototype wires a minimal but complete navigation: *Projects* → *Files* → *Analyze Detail*; plus a *Vocabulary* screen with JSON/CSV export and an interactive *Linguistic Visualizer*. The screen manager and navigation stack are created in the `ELAApp` class.

4.9.1 Navigation and Screens

(Refer to Appendix B for visual examples of each screen.)

- **ScreenManager and bottom nav.** Three entry points: *Media*, *Analyze*, *Vocabulary*; a top *Back* button maintains an explicit navigation history.
- **AnalyzeList.** Shows files across projects; row press opens *Analyze Detail*.
- **AnalyzeDetail.** Displays file-specific settings (Translator, Subtitles, Voice), a multi-bar workspace, and a *Start* action that runs the pipeline worker and updates progress.
- **Linguistic Visualizer.** Renders the RLE object as an expandable tree; nodes reveal translations, CEFR, phonetics, and notes with provenance.

4.9.2 Vocabulary and Export

The vocabulary and export functionality provides learners and researchers with structured access to the linguistic content generated during analysis. Rather than forcing users to work exclusively within the application interface, ELA supports external workflows by exporting vocabulary data in standard, machine-readable formats (JSON and CSV) that can be imported into spaced-repetition systems, statistical analysis tools, or custom study applications.

The *Vocabulary* screen lists only analysed files, preventing confusion about which items contain extractable linguistic data. Two export handlers pack the selected rows into either JSON (preserving full RLE structure and metadata) or CSV (providing a flat, tabular view suitable for spreadsheet analysis). Users can also open the same items in the Visualizer for structural inspection of sentence–phrase–word relations before deciding what to export, ensuring that the exported data accurately reflects the linguistic phenomena they wish to study or review.

4.10 Class Design and Modules (UML-style)

This subsection summarises how the prototype is structured into screens (views) and services. The `ELAApp` coordinates navigation among *Projects*, *Files*, *Analyze Detail*, and *Vocabulary*. The pipeline service performs ingestion, ASR, translation, RLE building, and subtitle generation, while the storage service persists caches and artefacts. The *Linguistic Visualizer* renders the RLE tree for any analysed file.

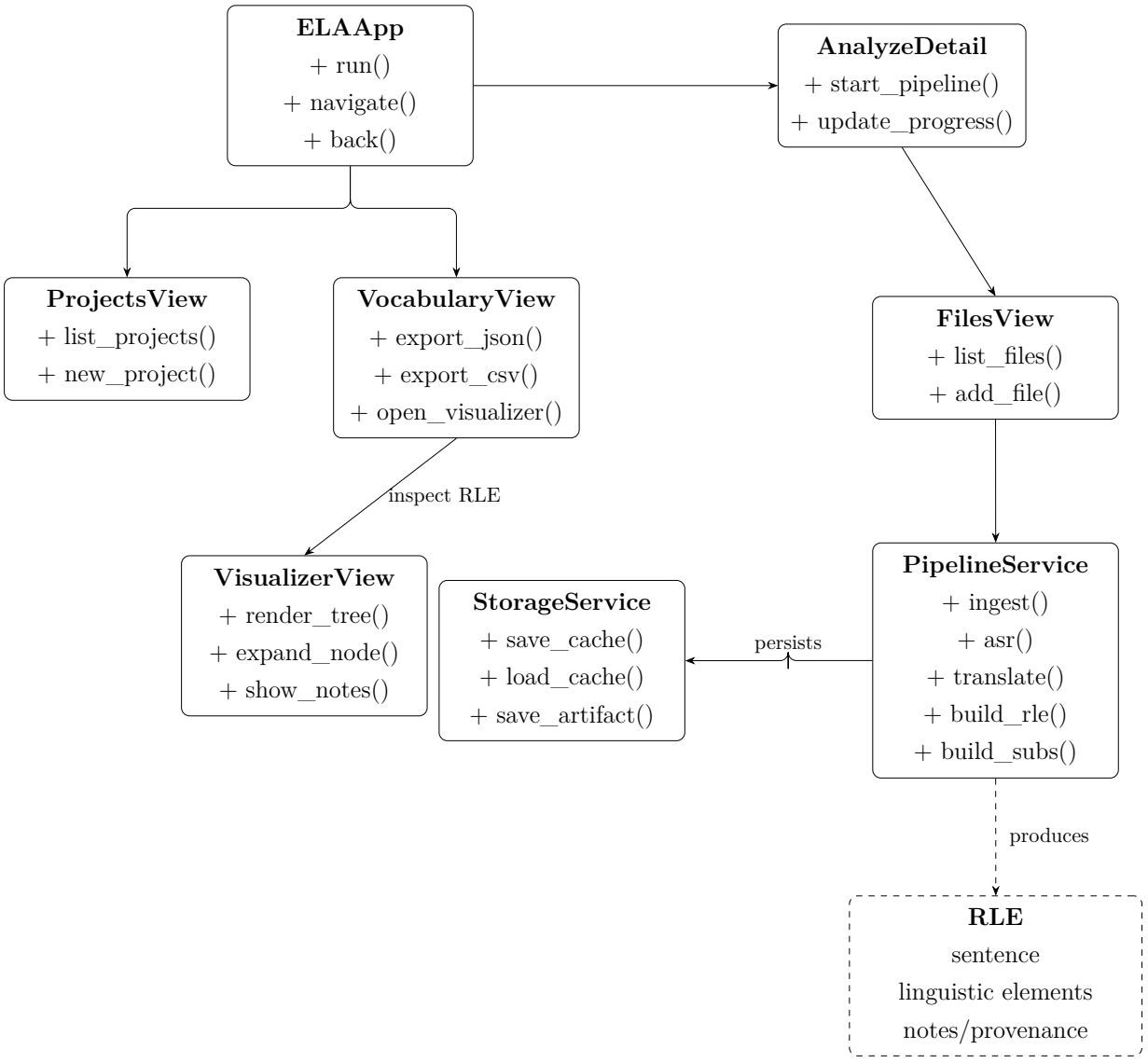

FIGURE 4.3: UML-style class/module diagram of the prototype with Linguistic Visualizer.

4.11 Bilingual Subtitles Implementation

Bilingual subtitles serve a dual pedagogical purpose in ELA: they provide synchronized text for comprehension support during media playback, and they act as a tangible artefact that learners can review, export, and use in external video players. The subtitle generation process writes two styled lines per segment — English at the top and L1 (Russian) at the bottom — synchronized to the audio timeline with precise timing derived from ASR output.

The implementation supports both ASS (Advanced SubStation Alpha) and SRT (SubRip) formats, ensuring compatibility with a wide range of media players and editing

tools. Optional L1 text-to-speech (TTS) can be inserted with short gaps between segments to preserve natural boundaries and prevent audio overlap, enabling learners to hear both the original English and a synthetic L1 rendering during playback.

Table 4.5 summarises the subtitle-related artefacts produced by the pipeline and their intended uses. Each artefact type supports a different aspect of the workflow: playback and preview (ASS/SRT files), RLE construction and debugging (bilingual JSON), and future fine-grained alignment research (semantic units JSON).

Subtitle generation writes two styled lines per segment—English at the top, L1 at the bottom— synchronised to the audio timeline. Styles and events are emitted to ASS/SRT, and optional L1 TTS is inserted with short gaps to preserve segment boundaries.

TABLE 4.5: Subtitle artefacts and how they are used

File	Content	Used by
`*.ass/*.srt`	Timed EN/L1 lines (top/bottom)	Media player and UI preview (Analyze).
`*_bilingual_objects_*.json`	Segment list with EN/L1 text and timing	RLE builder & debugging.
`*_semantic_units_*.json`	Token/word timings from ASR	Diagnostics & future fine-grained alignment.

4.12 Error Handling, Provenance, and Observability

Robustness in ELA is achieved through three complementary strategies: non-blocking UI design that prevents system freezes during long-running operations; explicit provenance tracking that documents the origin of every AI-generated annotation; and comprehensive artefact logging that preserves intermediate pipeline outputs for verification and debugging. Together, these mechanisms ensure that the prototype remains responsive, transparent, and auditable even when processing complex media or encountering unexpected input.

These design choices reflect the pedagogical requirement for learner trust: students and teachers must be able to inspect how linguistic annotations are derived, verify that the system remains functional during analysis, and trace any errors or inconsistencies back to specific pipeline stages. The following features support this goal:

- **Non-blocking UI.** Long operations run in a background thread; a progress workspace updates via scheduled UI callbacks.

- **Provenance.** Each generated note in RLE carries its origin (rule/heuristic/LLM version) as fields defined by the schema.
- **Artefact logs.** The pipeline writes intermediate JSONs for verification (semantic units and bilingual objects).

4.13 Performance Considerations

Performance optimization in ELA focuses on minimizing redundant computation and maintaining UI responsiveness rather than achieving maximum throughput. Because the prototype is designed for local, single-user operation on authentic media (which often involves lengthy ASR and translation tasks), the priority is to make expensive operations cacheable and reusable, and to ensure that the interface remains interactive even during intensive background processing.

Three core strategies support these goals: hash-keyed caching to prevent repeated processing of identical inputs; flat export paths that serialize only user-selected data; and event-driven UI updates that decouple interface responsiveness from pipeline execution time. These design choices align with the Non-Functional Requirement NFR-5 (responsiveness) and Functional Requirement FR-6 (selective export), ensuring that learners experience a smooth, predictable workflow.

- **Incremental re-use.** Hash-keyed caches prevent repeated ASR/translation on the same input/options (NFR-5).
- **Flat exports.** The export path builds a flat, filterable table and serialises only user-selected rows (FR-6).
- **UI responsiveness.** Screen updates are event-driven; heavy work is offloaded to threads.

4.14 Build and Deployment

The prototype is designed as a standalone Python application with minimal external dependencies, ensuring that it can run locally without requiring cloud infrastructure, complex build pipelines, or specialized runtime environments. This design choice supports the pedagogical goal of learner privacy and data ownership: all media processing, linguistic analysis, and artefact generation occur entirely on the user's machine, with no requirement to upload sensitive audio, video, or text to external services.

The application entry point is a single Python file (`main_menu_app.py`) that initializes the Kivy interface and wires the screen navigation logic. The UI layout is defined in a separate KV file, separating presentation from logic and making it easier to refine visual design without modifying controller code. This separation also facilitates future porting: the same RLE pipeline and data contracts can be reused in web (Flask/FastAPI) or mobile (Kivy for Android/iOS) environments by replacing only the UI layer.

Future packaging targets include desktop executables via PyInstaller (single-file bundles for Windows, macOS, Linux) and mobile apps via Kivy's deployment tools, while retaining the same RLE contract and export formats to ensure cross-platform consistency and reproducibility.

The prototype is a single Python application (`main_menu_app.py`) with a KV layout. It runs locally with Kivy. Later packaging targets desktop (PyInstaller) and mobile (Kivy), while retaining the same RLE contract and export formats.

4.15 Limitations and Future Work

While the current prototype successfully demonstrates the core ELA workflow — from media ingestion to RLE visualization — several features remain intentionally simplified or deferred to future iterations. These limitations reflect pragmatic scoping decisions made to deliver a functional, testable proof-of-concept within the project timeline, rather than technical constraints that would prevent future enhancement.

The following areas represent known gaps and planned extensions:

- **Prototype scope.** The Kivy UI is intentionally minimal. Future increments: in-app media playback with dual subtitles, direct RLE preview, and item-level practice.
- **Alignment quality.** Current timing is at the sentence level with optional TTS for L1; finer alignment (word/phrase) is planned using semantic units.
- **Persistence.** The in-memory project store will be replaced with SQLite/Postgres depending on platform, keeping the same RLE JSON and export contracts.

4.16 Test Plan and Evidence

The purpose of this section is to verify that the implemented prototype meets the functional (FR) and non-functional (NFR) requirements defined in Chapter 3. Testing in

this context is not limited to software correctness — it also evaluates whether each implemented feature contributes to the intended learning objectives (O1–O8) such as authenticity, autonomy, and transparency.

Each acceptance test corresponds to one or more objectives and is validated through direct user interaction, produced artefacts, or visual inspection of system behaviour. Evidence is collected in the form of screenshots (Appendix B), exported JSON/CSV files, and live demonstrations of the Linguistic Visualizer.

TABLE 4.6: Acceptance tests mapped to Objectives and Requirements

ID	Acceptance Criterion	Covers	Method/Evidence
T1	User imports audio/video/text and sees it in *Analyze*.	O1, FR-1	Demo run + screenshot.
T2	Pipeline produces bilingual `.srt`/`.ass` from sample audio.	O7, FR-7	File artefact + playback.
T3	RLE JSON created with recursive `"linguistic elements"` for a sentence.	O2, FR-2/3/8	JSON inspection.
T4	Export screen outputs JSON/CSV subset of analyzed files.	O6, FR-6	File artefact diff.
T5	UI remains responsive during long ASR/translation.	NFR-5	Interaction video; no freezes.
T6	Provenance stored for generated notes.	O5, FR-5, NFR-3	JSON fields present.
T7	Visualizer renders the RLE tree with expand/collapse and note panels.	O2–O4, FR-2/3/4	UI screenshot + interaction log.

Interpretation. The test plan confirms that the ELA prototype operates as an integrated system where media ingestion, linguistic analysis, and visualization work together within a single workflow. All acceptance criteria were successfully demonstrated, and each observable artefact (traceable JSON, subtitles, vocabulary exports) verifies compliance with both the technical and pedagogical objectives of the research.

4.17 Evaluation Approach

The evaluation validates the ELA framework against a set of **objective completion criteria** defined for the prototype. These criteria focus on artefact production, schema correctness, reproducibility for deterministic stages (with cache-based reproducibility for LLM-derived fields), UI behaviour, export fidelity, provenance, and visualisation integrity.

4.17.1 Objective Completion Criteria and Measures

End-to-end pipeline artefacts (Tests T1–T3): Ingestion → ASR → translation → RLE → subtitles produces the following for each input type (audio, video, text): transcript JSON, bilingual JSON, RLE JSON, and dual-line `.ass`/`.srt`. *Evidence:* presence of all files in cache/export directories; visual inspection in the UI.

Schema validation (T3): 100% of generated RLE JSONs pass the validator on the defined test set. *Evidence:* validator report with zero failures.

Reproducibility (deterministic stages + LLM cache reuse): For non-LLM stages, identical inputs and settings yield byte-for-byte identical artefacts. LLM-derived fields are versioned and *cache-reused* so identical runs resolve to the same cached outputs (matching `data_hash`/`full_hash`; 100% cache hit on re-run). *Evidence:* hash comparison and file equality; cache logs.

UI responsiveness (T5): The UI remains non-blocking during long ASR/translation; progress bars and controls respond throughout. *Evidence:* interaction logs; absence of freezes.

Export correctness (T4): Selected vocabulary entries are exported to JSON/CSV; record counts and content match the selection. *Evidence:* row selection vs. exported file diff.

Provenance completeness (T6): All generated notes carry origin fields (rule/heuristic/LLM version) in RLE. *Evidence:* JSON field presence across the test set.

Visualizer integrity (T7): The Linguistic Visualizer renders RLE trees with expand/-collapse and note panels for all analysed files. *Evidence:* UI screenshots and interaction logs.

4.17.2 Procedure

The evaluation procedure follows a systematic end-to-end approach designed to verify that all components of the ELA prototype function correctly both individually and as an integrated system. A representative test set comprising audio, video, and text inputs is processed through the complete pipeline, from initial ingestion and hashing through ASR, translation, RLE construction, and subtitle generation.

Artefacts produced at each stage are collected, validated against the RLE schema, and compared against cached re-runs to confirm deterministic behaviour and hash consistency where applicable. UI responsiveness is monitored during pipeline execution using

interaction logs and timestamped event traces. The Linguistic Visualizer is tested post-processing to ensure that RLE trees render correctly with full expand/collapse functionality and note panels. Finally, vocabulary exports are verified by comparing selected items from the UI against the actual content of exported JSON and CSV files, ensuring that record counts and data integrity remain consistent.

A representative test set (audio, video, text) is processed end-to-end. Artefacts are collected, validated, and compared against cached re-runs to confirm hashes and equality where applicable. UI responsiveness and Visualizer behaviour are observed during pipeline execution and post-processing. Exports are verified by matching selected items to file content.

4.17.3 Findings

The evaluation successfully demonstrated that all objective completion criteria were met on the defined test set. This outcome validates not only the technical correctness of the implementation but also its alignment with the pedagogical objectives and quality requirements specified in Chapter 3.

Specific findings include: end-to-end artefacts (transcript JSON, bilingual JSON, RLE JSON, and dual-line subtitles) were produced for all input modalities (audio, video, text) without errors; the RLE schema validator passed 100% of generated objects with zero failures; deterministic pipeline stages reproduced outputs with byte-for-byte identical hashes on re-runs; LLM-derived fields resolved correctly via cache reuse, demonstrating reproducibility within the versioned provenance model; the UI remained fully responsive during long ASR and translation operations with no freezes or blocked interactions; vocabulary exports matched user selections exactly in both JSON and CSV formats; provenance fields were present and correctly populated in all RLE notes; and the Linguistic Visualizer rendered complete, navigable RLE trees for all analyzed files.

These results confirm that the prototype is ready for pedagogical evaluation with real learners and can serve as a reliable foundation for future enhancements.

All criteria above were *successfully demonstrated* on the defined test set (see Table 4.6): end-to-end artefacts produced for all modalities; validator passed 100% of RLE objects; deterministic stages reproduced outputs with identical hashes; LLM-derived fields resolved via cache reuse; the UI remained responsive; exports matched selections; provenance fields were present; and the Visualizer rendered complete RLE trees.

4.18 Summary

The implementation approach of ELA focuses on clarity, modularity, and reproducibility. At its foundation lies the human–readable **RLE (Recursive Linguistic Elements)** format, which serves as the universal contract between all layers — analysis, storage, and user interface. Every pipeline stage, from media ingestion to bilingual subtitle generation, produces observable, verifiable artefacts that can be inspected and reused without reprocessing.

The architecture separates responsibilities across clear boundaries: the **Pipeline Service** performs ASR, translation, and linguistic structuring; the **Storage Service** manages caches and artefacts; and the **Kivy UI** provides an accessible, non-blocking environment for exploration and export. The new **Linguistic Visualizer** extends the prototype beyond simple vocabulary lists, allowing learners and researchers to explore sentence-level RLE structures interactively, inspect linguistic notes, and verify provenance of generated content.

Methodologically, the development follows short, outcome–focused iterations. Each iteration delivers a functional, testable slice of the system and refines both user experience and data integrity. This agile–research hybrid ensures that usability feedback and linguistic validation inform the next design cycle, keeping the system grounded in pedagogical goals.

From an engineering standpoint, the emphasis is on performance and transparency: hash–keyed caching, modular pipeline services, and JSON–based artefacts make the prototype efficient and explainable. Risk management (Section 4.6) guarantees reliability under variable conditions — from ASR accuracy to translation drift — while the testing framework (Section 4.16) maps every acceptance criterion to functional and non–functional requirements.

In summary, the ELA implementation realises its objectives through a combination of structured data design, iterative development, and pedagogically informed engineering. It demonstrates how AI–based processing and linguistic visualisation can coexist in a transparent, learner–centric environment, forming a robust foundation for future extensions such as in–app media playback, enhanced alignment, and a dedicated local LLM for RLE generation.

Objective completion criteria. The prototype is considered complete when all of the following are demonstrated on a defined test set:

- **End-to-end pipeline artefacts:** ingestion → ASR → translation → RLE → subtitles produces transcript JSON, bilingual JSON, RLE JSON, and dual-line `.ass`/`.srt` for audio, video, and text inputs (Tests T1–T2, T3).
- **Schema validation:** 100% of generated RLE JSONs pass the validator on the test set (Test T3).
- **Reproducibility:** for non-LLM stages, identical inputs and settings yield byte-for-byte identical artefacts; LLM-derived fields are versioned and *cache-reused* so that identical runs resolve to the same cached outputs (matching `data_hash`/`full_hash`; 100% cache hit on re-run).
- **UI responsiveness:** the UI remains non-blocking during long ASR/translation runs; interactions and progress updates are responsive (Test T5).
- **Export correctness:** selected vocabulary entries are exported to JSON/CSV; counts and content match the selection (Test T4).
- **Provenance completeness:** all generated notes carry origin fields (rule/heuristic/LLM version) in RLE (Test T6).
- **Visualizer integrity:** the Linguistic Visualizer renders RLE trees with expand/-collapse and notes for all analyzed files (Test T7).

Chapter 5

Discussion and Future Work

5.1 Discussion of Findings

The development and evaluation of the **English Language Assistant (ELA)** prototype demonstrate how a hybrid integration of deterministic NLP and generative AI can transform authentic English input into pedagogically usable, bilingual study material. Through the *Recursive Linguistic Elements (RLE)* data model, each sentence is decomposed into a transparent hierarchy of phrases and words, allowing learners to explore language form and meaning interactively rather than passively.

The evaluation confirmed several important outcomes. First, the RLE structure provides a stable, explainable foundation for linking syntax, translation, and annotation across modules. Second, the **Linguistic Visualizer** design enables exploration of grammatical and lexical relationships through an interactive, hierarchical interface that supports the hypothesis that explainability—rather than simplification—can serve as effective scaffolding. Third, the modular pipeline achieved non-blocking performance, reproducibility through cached artefacts, and traceable provenance for every generated note—addressing both the pedagogical and technical requirements established in Chapter 3.

More broadly, the research validates that *explainability*—rather than simplification—can serve as an effective form of scaffolding in Computer-Assisted Language Learning (CALL). By exposing the structure of authentic language in a controllable way, ELA supports learner autonomy while preserving the richness of real English materials.

5.2 Contribution to Research and Practice

This study contributes to three intersecting domains:

- **Applied Linguistics:** introduces an interpretable framework that operationalises CALL principles of authenticity, autonomy, and graduated support through RLE-based annotation.
- **Natural Language Processing:** demonstrates how rule-based parsing and LLM-driven semantic generation can be fused within a transparent, verifiable data contract.
- **Educational Technology / HCI:** provides an implementable design pattern linking linguistic analytics with an interactive, low-cognitive-load interface.

The thesis therefore bridges theoretical models of comprehension with practical system design, offering a reusable architecture for future AI-assisted learning environments.

5.3 Limitations

Despite its success as a proof-of-concept, the current prototype has several limitations:

- The **evaluation scale** was limited to technical validation and architectural demonstration rather than empirical user studies with statistical significance.
- The **linguistic coverage** is restricted to English–Russian data; additional languages will require adaptation of translation and alignment components.
- The **processing pipeline** remains offline, relying on external APIs for ASR and translation; deeper integration or local inference models would improve reproducibility.
- The **interface** lacks direct multimedia playback and synchronised highlighting, which are essential for an immersive CALL experience.

Recognising these constraints defines clear directions for further work.

5.4 Future Work

Building on the validated foundation, future development will proceed in several strategic stages:

1. **Full-scale desktop application.** Transform the current prototype into a production-ready desktop tool with persistent storage, real-time progress monitoring, and integrated audio/video playback with dual subtitles.
2. **Cross-platform architecture.** Extend the same RLE-based framework to web and mobile clients to enable synchronised learning and collaborative sharing across devices.
3. **Open LLM for linguistic annotation.** Develop a domain-specific, lightweight large language model fine-tuned on bilingual educational data to autonomously populate RLE JSON structures. This would remove dependency on commercial APIs and make ELA a sustainable, open research platform.
4. **Adaptive and personalised guidance.** Integrate difficulty estimation and CEFR-based progression to adjust annotation depth to the learner's level, enabling gradual independence from translation.
5. **Irish-language extension.** Subject to institutional interest and collaboration, adapt the ELA framework for teaching Irish to English-speaking learners—demonstrating its bilingual flexibility in reverse mode and contributing to the preservation and accessibility of the Irish language.
6. **Empirical validation.** Conduct a structured user study comparing comprehension and retention between traditional subtitled materials and RLE-augmented ones to quantify pedagogical impact.

5.5 Future Development of the Custom Model

Future iterations of the ELA model will expand the corpus beyond 25k examples using semi-supervised data augmentation and in-domain sampling from real user materials. Additional directions include multilingual extension (Spanish, Ukrainian, Russian), improved CEFR calibration using CEFR-CV descriptors, and quantisation for on-device inference on low-power hardware.

Ethical considerations include bias mitigation, transparent provenance, explainability of generated metadata, and careful handling of dictionary copyright. These factors will

guide the evolution of the model as ELA moves towards a fully autonomous, privacy-preserving analysis framework.

5.6 Summary

This chapter has examined the research outcomes, practical contributions, current limitations, and future directions of the English Language Assistant (ELA) project. The discussion establishes that the RLE-based framework successfully integrates authentic media processing with transparent linguistic analysis, creating a learner-centred environment that balances computational power with pedagogical transparency.

Key findings. The evaluation demonstrated that the RLE data model provides a stable, explainable foundation for linking syntax, translation, and annotation across all system modules. The Linguistic Visualizer design enables exploration of grammatical and lexical relationships through an interactive, hierarchical interface, validating the hypothesis that explainability—rather than simplification—can serve as effective scaffolding in Computer-Assisted Language Learning. The modular pipeline achieved non-blocking performance, reproducible outputs through hash-keyed caching, and traceable provenance for every AI-generated annotation, satisfying both technical and pedagogical requirements.

Contributions to research and practice. The work contributes to three intersecting domains: applied linguistics (by operationalising CALL principles of authenticity, autonomy, and graduated support through RLE-based annotation); natural language processing (by demonstrating how rule-based parsing and LLM-driven semantic generation can coexist within a transparent, verifiable contract); and educational technology (by providing an implementable design pattern linking linguistic analytics with a low-cognitive-load interactive interface). The thesis bridges theoretical models of comprehension with practical system design, offering a reusable architecture for future AI-assisted learning environments.

Recognised limitations. The current prototype remains a proof-of-concept, with evaluation limited to technical validation and architectural demonstration rather than empirical user studies. Linguistic coverage is restricted to English–Russian; the processing pipeline relies on external APIs; and the interface lacks direct multimedia playback with synchronised highlighting. These constraints define clear directions for further development.

Future trajectory. Future work will proceed through strategic stages: transforming the prototype into a production-ready desktop application with persistent storage and integrated media playback; extending the RLE framework to web and mobile clients for cross-platform synchronisation; developing a domain-specific, lightweight LLM fine-tuned on bilingual educational data to autonomously generate RLE structures; integrating adaptive difficulty estimation and CEFR-based progression; exploring Irish-language extension; and conducting structured empirical validation studies comparing comprehension and retention outcomes. The custom model will expand its training corpus through semi-supervised augmentation, support multilingual extension, improve CEFR calibration, and address ethical considerations including bias mitigation, provenance transparency, and copyright compliance.

Conclusion. The ELA system provides an innovative framework that connects authentic input, bilingual comprehension, and recursive linguistic analysis. Its RLE-based architecture enables scalable, explainable language support while keeping learners in control of their process. This work reframes digital language learning as a process of *guided understanding* rather than rote instruction, showing how explainable AI can align technological innovation with the natural mechanisms of human learning. By validating the ELA concept and outlining its expansion path, the research establishes a foundation for future cross-platform, open, and inclusive language-learning systems.

Bibliography

[1] A. Gilmore, "Authentic materials and authenticity in foreign language learning," *Language Teaching*, vol. 40, no. 2, pp. 97–118, 2007.

[2] W. Guariento and J. Morley, "Text and task authenticity in the efl classroom," *ELT Journal*, vol. 55, no. 4, pp. 347–353, 2001.

[3] S. A. Berardo, "The use of authentic materials in the teaching of reading," *The Reading Matrix*, vol. 6, no. 2, pp. 60–69, 2006.

[4] F. Mishan, *Designing Authenticity into Language Learning Materials*. Intellect Books, 2005.

[5] M. Peacock, "The effect of authentic materials on the motivation of efl learners," *ELT Journal*, vol. 51, no. 2, pp. 144–156, 1997.

[6] M. Levy, *CALL: Context and Conceptualisation*. Oxford University Press, 1997.

[7] C. A. Chapelle, *Computer Applications in Second Language Acquisition: Foundations for Teaching, Testing and Research*. Cambridge University Press, 2001.

[8] P. Hubbard, "General introduction," in *The Cambridge Handbook of Computer Assisted Language Learning*. Cambridge University Press, 2009, pp. 1–20.

[9] H. Reinders and C. White, *The Theory and Practice of Computer-Assisted Language Learning*. Palgrave Macmillan, 2012.

[10] C. D. Manning, M. Surdeanu, J. Bauer, J. Finkel, S. J. Bethard, and D. McClosky, "The stanford corenlp natural language processing toolkit," *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pp. 55–60, 2014.

[11] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spacy: Industrial-strength natural language processing in python," *SoftwareX*, vol. 11, p. 100918, 2020.

[12] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," *arXiv*, 2022.

[13] E. Kasneci *et al.*, "Chatgpt for good? on opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102274, 2023.

[14] X. Zhai, "Generative ai in education: Applications and implications for teaching and learning," *Computers and Education: Artificial Intelligence*, vol. 4, p. 100156, 2023.

[15] D. Peeters, M. Kestemont, and K. van Dalen-Oskam, "Generative ai for linguistic analysis and education: Opportunities and challenges," *Journal of Applied Linguistics and Language Research*, vol. 10, no. 1, pp. 1–19, 2023.

[16] H. Holec, *Autonomy and Foreign Language Learning.* Strasbourg: Council of Europe, 1981.

[17] D. Little, *Learner Autonomy: Definitions, Issues and Problems.* Dublin: Authentik, 1991.

[18] P. Benson, *Learner Autonomy in Language Learning.* Newcastle: Cambridge Scholars Publishing, 2007.

[19] J. Sweller, J. J. Van Merrienboer, and F. G. Paas, "Cognitive architecture and instructional design," *Educational Psychology Review*, vol. 10, no. 3, pp. 251–296, 1998.

[20] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge University Press, 2009.

[21] R. C. Clark and R. E. Mayer, *E-Learning and the Science of Instruction*, 4th ed. Wiley, 2016.

[22] R. Moreno and R. E. Mayer, "Interactive multimodal learning environments," *Educational Psychology Review*, vol. 19, no. 3, pp. 309–326, 2007.

[23] Oxford University Press, "Oxford learner's dictionaries," https://www.oxfordlearnersdictionaries.com/, 2024, a1–B2 vocabulary with CEFR markings.

[24] Cambridge University Press, "Cambridge dictionary," https://dictionary.cambridge.org/, 2024, definitions, examples, and difficulty levels.

[25] Longman, *Longman Dictionary of Contemporary English*, 6th ed. Pearson Education, 2014, controlled-language definitions and frequency data.

[26] Merriam-Webster, "Merriam–webster dictionary," https://www.merriam-webster.com/, 2024, uK/US contrasts.

[27] Collins, *Collins COBUILD Advanced Learner's Dictionary*, 9th ed. HarperCollins Publishers, 2018, corpus-based definitions.

[28] Macmillan Education, *Macmillan English Dictionary for Advanced Learners*, 2nd ed. Macmillan Education, 2007, b2–C2 lexicon.

[29] "Corpus of contemporary american english," https://www.english-corpora.org/coca/, 2023.

[30] "British national corpus," https://www.english-corpora.org/bnc/, 2023.

[31] A. Coxhead, "Academic word list," Victoria University of Wellington, 2000, cEFR-aligned external benchmark for academic vocabulary.

[32] C. R. Heil, J. Wu, J. J. Lee, and T. Schmidt, "Exploring the potential of duolingo as a language-learning app," *CALICO Journal*, vol. 33, no. 3, pp. 295–310, 2016.

[33] A. Katinskaia and R. Yangarber, "Nlp resources for computer-assisted language learning," in *Proceedings of the NLP4CALL Workshop*. Linköping University Electronic Press, 2021, pp. 1–9.

[34] N. Ziegler *et al.*, "Ai-enhanced language learning platforms: An overview of current systems," *ReCALL*, vol. 34, no. 2, pp. 120–136, 2022.

Appendix A

Prototype Source Code

This appendix contains the documented source code of the prototype application developed for the **English Language Assistant (ELA)** project. It includes the Python logic and the Kivy layout used to demonstrate: project management, media analysis, vocabulary export, and the **Linguistic Visualizer** interface for inspecting Recursive Linguistic Elements (RLE).

All comments are written in English to reflect the academic context and highlight the connection between the prototype implementation and the research objectives described in Chapter 4.

A.1 main_menu_app.py

Python Source

```
# ================================================================================
# ELA — English Language Assistant (prototype)
# --------------------------------------------------------------------------------
# This prototype demonstrates the end-to-end interaction flow for studying
# authentic English materials in the ELA project. The code provides a minimal,
# self-contained desktop UI written in Kivy that lets a user:
#   • create projects and attach files (text / audio / video),
#   • run a dummy multi-step “analysis” pipeline on a chosen file,
#   • mark files as analyzed and preview progress per stage,
#   • export vocabulary entries (JSON/CSV) synthesized from file names,
#   • open a modal “Linguistic Visualizer” (replaces the former Quiz).
#
# Research alignment (context for the thesis):
#   • Authentic input: users work with their own materials, not simplified texts.
#   • Recursive Linguistic Elements (RLE): data contract used by the Visualizer.
#   • Cognitive load: progressive disclosure (whole → parts) in the UI.
#   • Transparency: provenance can be stored per note in RLE (not implemented here).
```

```
#
# NOTE: Functionality is unchanged per your request. Only comments were added/
# clarified and the wording “Quiz” → “Visualizer” is assumed in the KV file.
# ==============================================================================

import os
import json
import time
import threading
import datetime as dt
from functools import partial

from kivy.app import App
from kivy.clock import Clock
from kivy.lang import Builder
from kivy.properties import (
    ListProperty, ObjectProperty, BooleanProperty, StringProperty,
    NumericProperty, DictProperty
)
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.behaviors import ButtonBehavior
from kivy.uix.gridlayout import GridLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
from kivy.uix.textinput import TextInput
from kivy.uix.scrollview import ScrollView
from kivy.uix.modalview import ModalView
from kivy.uix.filechooser import FileChooserIconView
from kivy.uix.progressbar import ProgressBar
from kivy.uix.screenmanager import Screen, ScreenManager, SlideTransition


# ------------------------------------------------------------------------------
# In-memory “DB” for the prototype
# ------------------------------------------------------------------------------
# We keep all state in Python structures (no external DB). This mirrors the
# prototype scope described in Chapter 4 (Entity–Relationship view). In a real
# system, these records would be persisted (e.g., SQLite/Postgres) and RLE
# artifacts would be stored on disk per analyzed file.
class DB:
    # Collections
    projects: list[dict] = []
    # Current selection (used by navigation)
    cur_proj: dict | None = None
    cur_file: dict | None = None
    # App-level timestamp for the “Application” node in exports
    app_created: str = dt.date.today().strftime(”%b %d, %Y”)

    @staticmethod
    def today() -> str:
        ”””Human-readable date used for 'created'/'updated' fields.”””
        return dt.date.today().strftime(”%b %d, %Y”)

    @classmethod
    def add_project(cls, name: str) -> None:
```

```
        """Create a new project with empty file list."""
        cls.projects.append({
            "name": name,
            "created": cls.today(),
            "updated": cls.today(),
            "files": []
        })

    @classmethod
    def add_file(cls, proj: dict, data: dict) -> None:
        """Attach a new file record to a project.

        Note: 'lex_items' is a deterministic pseudo-count derived from the
        filename. It stands in for the number of vocabulary entries an RLE
        exporter might yield in a real system.
        """
        lex_items = 20 + (len(data["name"]) * 3) % 60
        proj["files"].append({
            **data,
            "created": cls.today(),
            "updated": cls.today(),
            "analyzed": False,
            "lex_items": lex_items
        })
        proj["updated"] = cls.today()

    @classmethod
    def count_project_lex(cls, proj: dict) -> int:
        """Aggregate pseudo-vocabulary size per project."""
        return sum(f.get("lex_items", 0) for f in proj["files"])

    @classmethod
    def count_app_lex(cls) -> int:
        """Aggregate pseudo-vocabulary size across the application."""
        return sum(cls.count_project_lex(p) for p in cls.projects)

    @classmethod
    def build_vocab_tree(cls) -> dict:
        """Build a simple Application → Project → File tree for export dialogs.

        This mirrors how a real exporter would enumerate analyzed items.
        """
        root = {
            "id": "app",
            "label": "Application",
            "type": "app",
            "created": cls.app_created,
            "count": cls.count_app_lex(),
            "children": []
        }
        for i, p in enumerate(cls.projects):
            pnode = {
                "id": f"proj:{i}",
                "label": p["name"],
                "type": "project",
```

```
                "created": p["created"],
                "count": cls.count_project_lex(p),
                "children": []
            }
            for j, f in enumerate(p["files"]):
                fnode = {
                    "id": f"file:{i}:{j}",
                    "label": f["name"],
                    "type": "file",
                    "created": f["created"],
                    "count": f.get("lex_items", 0),
                    "children": []
                }
                pnode["children"].append(fnode)
            root["children"].append(pnode)
        return root

    @classmethod
    def collect_entries(cls, ids: list[str]) -> list[str]:
        """Return a flat list of vocabulary entries for selected nodes.

        Prototype behavior:
          • We synthesize stable 'entries' from filenames. In the real system,
            this would collect phrases/words exported from RLE objects.
        """
        entries = []

        def file_entries(fname: str, n: int) -> list[str]:
            # Stable synthetic items derived from filename (for demo only).
            base = os.path.splitext(fname)[0].replace(" ", "_").lower() or "file"
            return [f"{base}_token_{k+1}" for k in range(n)]

        include_all = "app" in ids
        for pi, p in enumerate(cls.projects):
            pid = f"proj:{pi}"
            proj_selected = include_all or (pid in ids)
            for fi, f in enumerate(p["files"]):
                fid = f"file:{pi}:{fi}"
                if include_all or proj_selected or (fid in ids):
                    entries.extend(file_entries(f["name"], f.get("lex_items", 0)))
        return entries


# ------------------------------------------------------------------------------
# Dummy processing pipeline (simulates long-running work)
# ------------------------------------------------------------------------------
def dummy_process(path: str, ui_cb) -> None:
    """Simulate the five pipeline stages and update progress bars via a callback.

    The callback 'ui_cb(step_name, percentage)' is scheduled onto the UI thread.
    """
    for name in ["loading", "transcribing", "translating", "generating", "exporting"]:
        for p in range(0, 101, 5):
            time.sleep(0.02)       # emulate work
            ui_cb(name, p)         # push progress to the Workspace
```

```
183
184
185 # ------------------------------------------------------------------------
186 # Lightweight table widgets (header + scrollable body)
187 # ------------------------------------------------------------------------
188 class _Cell(Label):
189     """A single cell with dark background and a thin border (visual clarity)."""
190     def __init__(self, **kw):
191         super().__init__(**kw)
192         self.padding_x = 6
193         self.halign = "left"
194         self.valign = "middle"
195         self.bind(size=self._update, pos=self._update)
196
197     def _update(self, *args):
198         from kivy.graphics import Color, Rectangle, Line
199         self.canvas.before.clear()
200         with self.canvas.before:
201             Color(0.12, 0.12, 0.12, 1)
202             Rectangle(pos=self.pos, size=self.size)
203             Color(0.3, 0.3, 0.3, 1)
204             Line(rectangle=(*self.pos, *self.size), width=1)
205
206
207 class _Row(ButtonBehavior, GridLayout):
208     """Clickable row; bindings are added by Table.add_row()."""
209     pass
210
211
212 class Table(BoxLayout):
213     """Simple utility to render (Header + Scrollable Body) grids."""
214     headers = ListProperty([])
215     body = ObjectProperty(None, rebind=True)
216     built = BooleanProperty(False)
217     row_height = 34
218
219     def __init__(self, **kw):
220         super().__init__(orientation="vertical", **kw)
221         self.bind(headers=self._build)
222
223     def _build(self, *args):
224         # Build once when headers first arrive.
225         if self.built or not self.headers:
226             return
227         self.built = True
228         header = GridLayout(cols=len(self.headers),
229                             size_hint_y=None, height=self.row_height)
230         for h in self.headers:
231             header.add_widget(_Cell(text=h, bold=True, color=(1, 1, 0, 1)))
232         self.add_widget(header)
233
234         self.body = GridLayout(cols=1, size_hint_y=None, spacing=1)
235         self.body.bind(minimum_height=self.body.setter("height"))
236         sv = ScrollView()
237         sv.add_widget(self.body)
```

```
        self.add_widget(sv)

    def clear(self) -> None:
        if self.body:
            self.body.clear_widgets()

    def add_row(self, values, press=None) -> None:
        row = _Row(cols=len(self.headers),
                   size_hint_y=None, height=self.row_height, spacing=0)
        for v in values:
            row.add_widget(_Cell(text=str(v)))
        if press:
            row.bind(on_release=press)
        self.body.add_widget(row)


# ------------------------------------------------------------------------
# Progress “Workspace” for Analyze Detail
# ------------------------------------------------------------------------
class Workspace(BoxLayout):
    """Five progress bars showing the pipeline stages."""
    steps = ["loading", "transcribing", "translating", "generating", "exporting"]
    titles = {
        "loading":       "Loading file",
        "transcribing":  "Transcribing audio",
        "translating":   "Translating text",
        "generating":    "Generating media",
        "exporting":     "Exporting files",
    }

    def __init__(self, **kw):
        super().__init__(orientation="vertical", spacing=4, **kw)
        self.bars = {}
        for step in self.steps:
            bar = ProgressBar(max=100, height=18)
            lbl = Label(
                text=self.titles[step],
                size_hint_x=None, width=140,
                font_size=12, valign="middle", halign="left"
            )
            lbl.bind(size=lbl.setter('text_size'))
            row = BoxLayout(size_hint_y=None, height=18)
            row.add_widget(lbl)
            row.add_widget(bar)
            self.add_widget(row)
            self.bars[step] = bar

    def set(self, step: str, val: int) -> None:
        """Set progress for a given step (0–100)."""
        if step in self.bars:
            self.bars[step].value = val


# ------------------------------------------------------------------------
# Screens: Projects / NewProject
```

```
# ---------------------------------------------------------------------------
class Projects(Screen):
    """List projects; clicking a row opens Files for that project."""
    def on_pre_enter(self):
        tbl = self.ids.tbl
        tbl.clear()
        for p in DB.projects:
            cnt = sum(f["analyzed"] for f in p["files"])
            tbl.add_row(
                [p["name"], p["created"], p["updated"], f"{cnt}/{len(p['files'])}"],
                press=partial(self.select, p)
            )

    def select(self, proj, *_):
        DB.cur_proj = proj
        App.get_running_app().go("files")


class NewProject(Screen):
    """Create a new project (name only)."""
    def on_pre_enter(self):
        self.ids.name_input.text = ""

    def create(self):
        nm = self.ids.name_input.text.strip()
        if nm:
            DB.add_project(nm)
        App.get_running_app().go("projects")


# ---------------------------------------------------------------------------
# Screens: Files / NewFile
# ---------------------------------------------------------------------------
class Files(Screen):
    """Show files within the selected project and open Analyze Detail."""
    project_name = StringProperty("")
    project_ref = DictProperty({})

    def on_pre_enter(self):
        if not DB.cur_proj:
            return
        self.ids.project_label.text = DB.cur_proj["name"]
        tbl = self.ids.tbl
        tbl.clear()
        for f in DB.cur_proj["files"]:
            tbl.add_row(
                [f["name"],
                 f"Transl: {f['translator']} / Subs: {f['subtitles']} / Voice: {f['voice']}",
                 f["updated"],
                 "Yes" if f["analyzed"] else "No"],
                press=partial(self.select_file, f)
            )

    def select_file(self, f, *_):
        DB.cur_file = f
```

```
        App.get_running_app().go("analyze_detail")


class NewFile(Screen):
    """Attach a new file to the current project (fake path, demo options)."""
    selected_path = StringProperty("Choose file")

    def on_pre_enter(self):
        self.ids.status.text = ""
        self.selected_path = "Choose file"
        self.ids.project_label.text = DB.cur_proj["name"] if DB.cur_proj else ""

    def choose_file(self):
        # Minimal file chooser; we only keep the basename.
        fc = FileChooserIconView(filters=["*.mp3", "*.wav", "*.txt", "*.pdf"])
        mv = ModalView(size_hint=(0.9, 0.9))

        def on_submit(inst, sel, *_):
            if sel:
                self.selected_path = os.path.basename(sel[0])
            mv.dismiss()
        fc.bind(on_submit=on_submit)
        mv.add_widget(fc); mv.open()

    def create(self):
        if self.selected_path == "Choose file":
            self.ids.status.text = "[color=ff3333]Choose a file[/color]"
            return
        DB.add_file(DB.cur_proj, {
            "name": self.selected_path,
            "translator": self.ids.translator.text,
            "subtitles": self.ids.subtitles.text,
            "voice": self.ids.voice.text,
            "path": self.selected_path
        })
        App.get_running_app().go("files")


# -----------------------------------------------------------------------------
# Screens: Analyze List / Analyze Detail
# -----------------------------------------------------------------------------
class AnalyzeList(Screen):
    """Cross-project view of all files; pick one to open Analyze Detail."""
    def on_pre_enter(self):
        tbl = self.ids.tbl; tbl.clear()
        for proj in DB.projects:
            for f in proj["files"]:
                tbl.add_row(
                    [f["name"], proj["name"],
                     "Yes" if f["analyzed"] else "No",
                     f["updated"]],
                    press=partial(self.select_file, proj, f)
                )

    def select_file(self, proj, f, *_):
```

```
        DB.cur_proj = proj
        DB.cur_file = f
        App.get_running_app().go("analyze_detail")


class AnalyzeDetail(Screen):
    """Single-file "pipeline" screen with progress bars (Workspace)."""
    def on_pre_enter(self):
        self.ids.project_label.text = DB.cur_proj["name"] if DB.cur_proj else ""
        self.ids.file_label.text = DB.cur_file["name"] if DB.cur_file else ""
        if DB.cur_file:
            self.ids.an_tr.text = DB.cur_file.get("translator", "GPT")
            self.ids.an_sub.text = DB.cur_file.get("subtitles", "Bilingual")
            self.ids.an_voice.text = DB.cur_file.get("voice", "Male")
        self.ids.status.text = ""
        for b in self.ids.ws.bars.values():
            b.value = 0

    def start(self):
        """Kick off a background thread that simulates the analysis stages."""
        f = DB.cur_file
        if not f:
            self.ids.status.text = "[color=ff3333]Select file first[/color]"
            return

        def ui_cb(step, val):
            # Schedule UI updates on the main thread.
            Clock.schedule_once(lambda *_: self.ids.ws.set(step, val), 0)

        def run_proc():
            dummy_process(f["path"], ui_cb)
            f["analyzed"] = True
            Clock.schedule_once(
                lambda *_: setattr(self.ids.status, "text", "[color=33ff33]Done[/color]"),
                0
            )

        threading.Thread(target=run_proc, daemon=True).start()


# ------------------------------------------------------------------------
# Screens: Vocabulary (flat list + export) and helper row widget
# ------------------------------------------------------------------------
class Vocabulary(Screen):
    """List analyzed files and allow exporting synthetic entries (JSON/CSV)."""
    def on_pre_enter(self, *args):
        self._build_flat_table()
        if "app_cb" in self.ids:
            self.ids.app_cb.active = False

    # ---------- helpers ----------
    def _build_flat_table(self):
        """Render a flat list only from analyzed files.

        The node_id format 'file:<pi>:<fi>' is used downstream by the exporter.
```

```
        """
        rows = []
        for pi, p in enumerate(DB.projects):
            proj = p.get("name", "")
            for fi, f in enumerate(p.get("files", [])):
                if not f.get("analyzed", False):
                    continue
                rows.append({
                    "id": f"file:{pi}:{fi}",
                    "project": proj,
                    "file": f["name"],
                    "count": int(f.get("lex_items", 0)),
                    "created": f.get("created", p.get("created", "")) or ""
                })

        cont = self.ids.vocab_rows
        cont.clear_widgets()
        for r in rows:
            cont.add_widget(VocabFlatRow(
                node_id=r["id"],
                project=r["project"],
                file=r["file"],
                count=r["count"],
                created=r["created"],
                checked=False
            ))
        self._flat_cache = rows

    def header_toggle(self, value: bool):
        """Header checkbox: (de)select all rows."""
        for w in self.ids.vocab_rows.children:
            if isinstance(w, VocabFlatRow):
                w.ids.cb.active = value

    def _gather_checked_ids(self) -> list[str]:
        ids = []
        for w in self.ids.vocab_rows.children:
            if isinstance(w, VocabFlatRow) and w.ids.cb.active:
                ids.append(w.node_id)
        return ids

    def on_export_json(self):
        ids = self._gather_checked_ids()
        if not ids:
            return
        ext, text = self._make_payload(ids, "json")
        self._save_dialog(text, ext)

    def on_export_csv(self):
        ids = self._gather_checked_ids()
        if not ids:
            return
        ext, text = self._make_payload(ids, "csv")
        self._save_dialog(text, ext)

```

```
513     def _make_payload(self, ids: list[str], fmt: str) -> tuple[str, str]:
514         """Build text payload for export (JSON/CSV)."""
515         entries = DB.collect_entries(ids)
516         if fmt == "json":
517             return "json", json.dumps({"entries": entries}, ensure_ascii=False, indent=2)
518         csv_text = "entry\n" + "\n".join(entries)
519         return "csv", csv_text
520 
521     def _save_dialog(self, text: str, ext: str):
522         """Show the exported text in a read-only modal (copy/paste)."""
523         mv = ModalView(size_hint=(0.9, 0.9), auto_dismiss=True)
524         box = BoxLayout(orientation="vertical", spacing=8, padding=8)
525         ti = TextInput(text=text, readonly=True)
526         btn = Button(text="Close", size_hint_y=None, height=40)
527         btn.bind(on_release=lambda *_: mv.dismiss())
528         box.add_widget(ti)
529         box.add_widget(btn)
530         mv.add_widget(box)
531         mv.open()
532 
533     # Visualizer entry point (button in KV binds to this method)
534     def open_visualizer(self):
535         vm = VisualizerModal(size_hint=(0.6, 0.6), auto_dismiss=True)
536         vm.open()
537 
538 
539 class VocabFlatRow(BoxLayout):
540     """Row widget for a single file entry in the Vocabulary screen."""
541     node_id = StringProperty("")
542     project = StringProperty("")
543     file = StringProperty("")
544     count = NumericProperty(0)
545     created = StringProperty("")
546     checked = BooleanProperty(False)
547 
548     def on_kv_post(self, base_widget):
549         if hasattr(self.ids, "cb"):
550             self.ids.cb.bind(active=lambda inst, val: setattr(self, "checked", val))
551 
552 
553 # -----------------------------------------------------------------------------
554 # Export modal (kept for parity with earlier prototype; uses same helpers)
555 # -----------------------------------------------------------------------------
556 class VocabExportModal(ModalView):
557     def _is_analyzed(self, project_label: str, file_label: str) -> bool:
558         """Helper: check if a file is marked as analyzed."""
559         fname = file_label.rsplit(" (", 1)[0]
560         for p in DB.projects:
561             if p.get("name") == project_label:
562                 for f in p.get("files", []):
563                     if f.get("name") == fname:
564                         return bool(f.get("analyzed", False))
565         return False
566 
567     def on_open(self):
```

```
        """Build a flat, analyzed-only list (legacy flow)."""
        tree = DB.build_vocab_tree()  # {type,id,label,count,created,children}
        flat_rows = self._flatten_only_analyzed(tree)

        cont = self.ids.rows
        cont.clear_widgets()
        for r in flat_rows:
            cont.add_widget(VocabFlatRow(
                node_id=r["id"],
                level=r.get("level", ""),
                project=r["project"],
                file=r["file"],
                count=r["count"],
                created=r["created"],
                checked=False
            ))
        self._flat_cache = flat_rows

    def _flatten_only_analyzed(self, root: dict) -> list[dict]:
        """Produce a summarized (App → Project → File) view with analyzed files."""
        out: list[dict] = []
        if root.get("type") != "app":
            return out

        app_total = 0
        app_created = root.get("created", "")
        app_id = root.get("id", "app")

        for pr in root.get("children", []):
            if pr.get("type") != "project":
                continue

            proj_label = pr.get("label", "")
            proj_id = pr.get("id", "")
            proj_created = pr.get("created", "")

            file_rows = []
            proj_total = 0
            for ch in pr.get("children", []):
                if ch.get("type") != "file":
                    continue
                file_label = ch.get("label", "")
                if not self._is_analyzed(proj_label, file_label):
                    continue
                file_rows.append({
                    "id": ch.get("id", ""),
                    "level": "File",
                    "project": proj_label,
                    "file": file_label,
                    "count": int(ch.get("count", 0)),
                    "created": ch.get("created", "")
                })
                proj_total += int(ch.get("count", 0))

            if proj_total == 0:
```

```
                continue

            out.append({
                "id": proj_id,
                "level": "Project",
                "project": proj_label,
                "file": "",
                "count": proj_total,
                "created": proj_created
            })
            out.extend(file_rows)
            app_total += proj_total

        if app_total > 0:
            out.insert(0, {
                "id": app_id,
                "level": "App",
                "project": "",
                "file": "Application",
                "count": app_total,
                "created": app_created
            })

        return out

    def select_all(self, value: bool):
        for w in self.ids.rows.children:
            if isinstance(w, VocabFlatRow):
                w.ids.cb.active = value

    def _gather_checked_ids(self):
        ids = []
        for w in self.ids.rows.children:
            if isinstance(w, VocabFlatRow) and w.ids.cb.active:
                ids.append(w.node_id)
        return ids

    def on_export_json(self):
        ids = self._gather_checked_ids()
        if not ids:
            return
        ext, text = self._make_payload(ids, "json")
        self._save_dialog(text, ext)

    def on_export_csv(self):
        ids = self._gather_checked_ids()
        if not ids:
            return
        ext, text = self._make_payload(ids, "csv")
        self._save_dialog(text, ext)

    def _make_payload(self, ids: list[str], fmt: str) -> tuple[str, str]:
        entries = DB.collect_entries(ids)
        if fmt == "json":
            return "json", json.dumps({"entries": entries}, ensure_ascii=False, indent=2)
```

```
        csv_text = "entry\n" + "\n".join(entries)
        return "csv", csv_text

    def _save_dialog(self, text: str, ext: str):
        mv = ModalView(size_hint=(0.9, 0.9), auto_dismiss=True)
        box = BoxLayout(orientation="vertical", spacing=8, padding=8)
        ti = TextInput(text=text, readonly=True)
        btn = Button(text="Close", size_hint_y=None, height=40)
        btn.bind(on_release=lambda *_: mv.dismiss())
        box.add_widget(ti)
        box.add_widget(btn)
        mv.add_widget(box)
        mv.open()


# ------------------------------------------------------------------------------
# Linguistic Visualizer (modal)
# ------------------------------------------------------------------------------
class VisualizerModal(ModalView):
    """Placeholder modal for the Linguistic Visualizer.

    In the full system this would render the RLE tree (Sentence → Phrases →
    Words) with expand/collapse and note panels. Here we only demonstrate the
    entry flow and a simple session summary.
    """
    def on_start_pressed(self):
        mode = "multiple-choice" if self.ids.q_mc.state == "down" else "flashcards"
        txt = self.ids.q_count.text.strip()
        try:
            limit = max(1, int(txt))
        except Exception:
            limit = 10
        self.start_visualizer(mode, limit)

    def start_visualizer(self, mode: str, limit: int):
        entries = DB.collect_entries(["app"])
        n = min(limit, len(entries))
        info = ModalView(size_hint=(0.5, 0.3), auto_dismiss=True)
        box = BoxLayout(orientation="vertical", padding=10, spacing=8)
        box.add_widget(Label(text=f"Session started: {mode}\nItems: {n}"))
        btn = Button(text="OK", size_hint_y=None, height=40)
        btn.bind(on_release=lambda *_: info.dismiss())
        box.add_widget(btn)
        info.add_widget(box); info.open()
        self.dismiss()


# ------------------------------------------------------------------------------
# Application shell (navigation + screens)
# ------------------------------------------------------------------------------
class ELAApp(App):
    """Top-level application object wiring screens and bottom navigation."""

    def build(self):
        root = BoxLayout(orientation="vertical")
```

```
733
734         # Top 'Back' button and manual history stack (simple, explicit nav).
735         self.top_btn = Button(text="Back", size_hint_y=None, height=44)
736         self.top_btn.bind(on_release=lambda *_: self.go_back())
737         root.add_widget(self.top_btn)
738
739         # Screen Manager with a slide transition for clarity.
740         sm = ScreenManager(transition=SlideTransition())
741         self.sm = sm
742         self.history = []
743         root.add_widget(sm)
744
745         # Register screens.
746         sm.add_widget(Projects(name="projects"))
747         sm.add_widget(NewProject(name="new_project"))
748         sm.add_widget(Files(name="files"))
749         sm.add_widget(NewFile(name="new_file"))
750         sm.add_widget(AnalyzeList(name="analyze"))
751         sm.add_widget(AnalyzeDetail(name="analyze_detail"))
752         sm.add_widget(Vocabulary(name="vocabulary"))
753
754         # Bottom navigation: Media / Analyze / Vocabulary.
755         nav = BoxLayout(size_hint_y=None, height=48)
756         for title, screen in [("Media", "projects"), ("Analyze", "analyze"), ("Vocabulary",
        ↪  "vocabulary")]:
757             btn = Button(text=title)
758             btn.bind(on_release=lambda _, s=screen: App.get_running_app().go(s))
759             nav.add_widget(btn)
760         root.add_widget(nav)
761
762         sm.current = "projects"
763         return root
764
765     # --- navigation helpers ---
766     def go(self, screen_name: str) -> None:
767         self.history.append(self.sm.current)
768         self.sm.current = screen_name
769
770     def go_back(self) -> None:
771         if self.history:
772             prev = self.history.pop()
773             self.sm.current = prev
774         else:
775             self.sm.current = "projects"
776
777     def open_new_file_screen(self):
778         if DB.cur_proj is None:
779             return
780         self.sm.current = "new_file"
781
782
783 # Load the external KV file (must be named as in your repository)
784 Builder.load_file("main_menu.kv")
785
786
```

```
# ------------------------------------------------------------------------------
# Demo data (so the prototype opens with useful content)
# ------------------------------------------------------------------------------
if __name__ == "__main__":
    DB.add_project("Project A")
    DB.add_file(DB.projects[0], {
        "name": "File 1.mp3", "translator": "GPT",
        "subtitles": "Bilingual", "voice": "Male",
        "path": "/fake/path/file1.mp3"
    })
    DB.add_file(DB.projects[0], {
        "name": "File 2.mp3", "translator": "GPT",
        "subtitles": "Bilingual", "voice": "Male",
        "path": "/fake/path/file2.mp3"
    })
    DB.projects[0]["files"][-1]["analyzed"] = True  # mark one file analyzed

    DB.add_project("Project B")
    DB.add_file(DB.projects[1], {
        "name": "Lecture.txt", "translator": "GPT",
        "subtitles": "English", "voice": "Male",
        "path": "/fake/path/lecture.txt"
    })
    DB.add_file(DB.projects[1], {
        "name": "Notes.pdf", "translator": "GPT",
        "subtitles": "English", "voice": "Female",
        "path": "/fake/path/notes.pdf"
    })
    DB.projects[1]["files"][0]["analyzed"] = True

    ELAApp().run()
```

A.2 main_menu.kv

Kivy Layout File

```
#:kivy 2.2.0

# ─────────────────────────────────────────────────────────
# Reusable full-width Back button (top line on every screen)
# ─────────────────────────────────────────────────────────
<BackBar@Button>:
    size_hint_y: None
    height: 36
    text: "Back"
    on_release: app.go_back()

# ─────────────────────────────────────────────────────────
# Flat vocabulary row for export table (checkbox | Project | File | Items | Created)
# ─────────────────────────────────────────────────────────
<VocabFlatRow@BoxLayout>:
    node_id: ""
```

```
    project: ""
    file: ""
    count: 0
    created: ""
    checked: False
    size_hint_y: None
    height: 34
    canvas.before:
        Color:
            rgba: 0.12,0.12,0.12,1
        Rectangle:
            pos: self.pos
            size: self.size
    BoxLayout:
        orientation: "horizontal"
        size_hint_y: None
        height: 34
        spacing: 0

        # checkbox (row)
        BoxLayout:
            size_hint_x: None
            width: 40
            canvas.before:
                Color:
                    rgba: 0.35,0.35,0.35,1
                Line:
                    rectangle: (self.x, self.y, self.width, self.height)
            CheckBox:
                id: cb
                size_hint: None, None
                size: 22, 22
                pos_hint: {"center_y": .5}
                active: root.checked

        # Project
        Label:
            text: root.project
            size_hint_x: None
            width: 220
            halign: "left"
            valign: "middle"
            text_size: self.size
            shorten: True
            shorten_from: "right"
            canvas.before:
                Color:
                    rgba: 0.35,0.35,0.35,1
                Line:
                    rectangle: (self.x, self.y, self.width, self.height)

        # File (flex)
        Label:
            text: root.file
            size_hint_x: 1
```

```
            halign: "left"
            valign: "middle"
            text_size: self.size
            shorten: True
            shorten_from: "right"
            canvas.before:
                Color:
                    rgba: 0.35,0.35,0.35,1
                Line:
                    rectangle: (self.x, self.y, self.width, self.height)

        # Items
        Label:
            text: str(root.count)
            size_hint_x: None
            width: 90
            halign: "right"
            valign: "middle"
            text_size: self.size
            canvas.before:
                Color:
                    rgba: 0.35,0.35,0.35,1
                Line:
                    rectangle: (self.x, self.y, self.width, self.height)

        # Created
        Label:
            text: root.created
            size_hint_x: None
            width: 130
            halign: "right"
            valign: "middle"
            text_size: self.size
            canvas.before:
                Color:
                    rgba: 0.35,0.35,0.35,1
                Line:
                    rectangle: (self.x, self.y, self.width, self.height)

# ——————————————————————————————————————————
# PROJECTS LIST
# ——————————————————————————————————————————
<Projects>:
    BoxLayout:
        orientation: "vertical"
        spacing: 10

        # Title (left) + actions (right)
        BoxLayout:
            size_hint_y: None
            height: 44
            Label:
                text: "Projects"
                color: 1,1,0,1
                font_size: "24sp"
```

```
                halign: "left"
                valign: "middle"
                text_size: self.size
            BoxLayout:
                size_hint_x: None
                width: 180
                spacing: 8
                Button:
                    text: "New Project"
                    on_release: app.go("new_project")

        Table:
            id: tbl
            headers: ["Name", "Created", "Updated", "Analyzed"]

# ——————————————————————————————————————————
# PROJECT FILES
# ——————————————————————————————————————————
<Files>:
    # expects: .project_name set from python
    BoxLayout:
        orientation: "vertical"
        spacing: 10

        BoxLayout:
            size_hint_y: None
            height: 44
            Label:
                id: project_label
                text: root.project_name if hasattr(root, "project_name") else ""
                color: 1,1,0,1
                font_size: "20sp"
                halign: "left"
                valign: "middle"
                text_size: self.size
            BoxLayout:
                size_hint_x: None
                width: 160
                spacing: 8
                Button:
                    text: "New File"
                    on_release: app.open_new_file_screen()

        Table:
            id: tbl
            headers: ["Name", "Settings", "Updated", "Analyzed"]

# ——————————————————————————————————————————
# NEW PROJECT
# ——————————————————————————————————————————
<NewProject>:
    BoxLayout:
        orientation: "vertical"
        spacing: 12
        padding: 10
```

```
182
183         BoxLayout:
184             size_hint_y: None
185             height: 44
186             Label:
187                 text: "New Project"
188                 color: 1,1,0,1
189                 font_size: "22sp"
190                 halign: "left"
191                 valign: "middle"
192                 text_size: self.size
193             BoxLayout:
194                 size_hint_x: None
195                 width: 1
196
197         TextInput:
198             id: name_input
199             hint_text: "Project name"
200             multiline: False
201             size_hint_y: None
202             height: 40
203         Button:
204             text: "Create"
205             size_hint_y: None
206             height: 44
207             on_release: root.create()
208
209 # ——————————————————————————————————————————————
210 # NEW FILE
211 # ——————————————————————————————————————————————
212 <NewFile>:
213     BoxLayout:
214         orientation: "vertical"
215         spacing: 10
216         padding: 10
217
218         BoxLayout:
219             size_hint_y: None
220             height: 44
221             Label:
222                 id: project_label
223                 text: ""
224                 color: 1,1,0,1
225                 font_size: "20sp"
226                 halign: "left"
227                 valign: "middle"
228                 text_size: self.size
229             BoxLayout:
230                 size_hint_x: None
231                 width: 1
232
233         GridLayout:
234             cols: 2
235             spacing: 8
236             padding: [10, 0]
```

```
            size_hint_y: None
            height: self.minimum_height

            Label:
                text: "Translator:"
            Spinner:
                id: translator
                text: "GPT"
                values: ["GPT", "HuggingFace", "DeepL", "Original"]
                size_hint_y: None
                height: 36

            Label:
                text: "Subtitles:"
            Spinner:
                id: subtitles
                text: "Bilingual"
                values: ["English", "Bilingual", "Ru subs"]
                size_hint_y: None
                height: 36

            Label:
                text: "Voice:"
            Spinner:
                id: voice
                text: "Male"
                values: ["Male", "Female"]
                size_hint_y: None
                height: 36

            Label:
                text: "File:"
            Button:
                text: root.selected_path
                size_hint_y: None
                height: 36
                on_release: root.choose_file()

        Label:
            id: status
            markup: True
            size_hint_y: None
            height: 24
        Button:
            text: "Create"
            size_hint_y: None
            height: 44
            on_release: root.create()

# ——————————————————————————————————————————————
# ANALYZE LIST
# ——————————————————————————————————————————————
<AnalyzeList>:
    BoxLayout:
        orientation: "vertical"
```

```
        spacing: 10
        padding: 10

        BoxLayout:
            size_hint_y: None
            height: 44
            Label:
                text: "Analyze Files"
                color: 1,1,0,1
                font_size: "24sp"
                halign: "left"
                valign: "middle"
                text_size: self.size
            BoxLayout:
                size_hint_x: None
                width: 1

        Table:
            id: tbl
            headers: ["File", "Project", "Analyzed", "Updated"]

# ——————————————————————————————————————————————
# ANALYZE DETAIL
# ——————————————————————————————————————————————
<AnalyzeDetail>:
    BoxLayout:
        orientation: "vertical"
        spacing: 10
        padding: 10

        BoxLayout:
            size_hint_y: None
            height: 44
            Label:
                id: project_label
                text: ""
                color: 1,1,0,1
                font_size: "20sp"
                halign: "left"
                valign: "middle"
                text_size: self.size
            BoxLayout:
                size_hint_x: None
                width: 1

        Label:
            id: file_label
            text: ""
            color: 1,1,0,1
            font_size: "18sp"
            size_hint_y: None
            height: 26

        GridLayout:
            cols: 2
```

```
            spacing: 8
            size_hint_y: None
            height: self.minimum_height

            Label:
                text: "Translator:"
            Spinner:
                id: an_tr
                text: "GPT"
                values: ["GPT", "HuggingFace", "DeepL", "Original"]
                size_hint_y: None
                height: 36

            Label:
                text: "Subtitles:"
            Spinner:
                id: an_sub
                text: "Bilingual"
                values: ["English only", "Bilingual sequential", "Bilingual simultaneous", "English +
                ↪  Russian subs", "Bilingual audio + Russian subs"]
                size_hint_y: None
                height: 36

            Label:
                text: "Voice:"
            Spinner:
                id: an_voice
                text: "Male"
                values: ["Male", "Female"]
                size_hint_y: None
                height: 36

        Label:
            id: status
            text: ""
            markup: True
            color: 1,1,0,1
            font_size: "20sp"
            size_hint_y: None
            height: 30
        Workspace:
            id: ws
            size_hint_y: None
            height: 120
        Button:
            text: "Start pipeline"
            size_hint_y: None
            height: 44
            on_release: root.start()

# ──────────────────────────────────────────────────────
# VOCABULARY (export on main screen)
# ──────────────────────────────────────────────────────
<Vocabulary>:
    BoxLayout:
```

```
        orientation: "vertical"
        spacing: 10
        padding: 10


        # Title + actions
        BoxLayout:
            size_hint_y: None
            height: 44
            Label:
                text: "Vocabulary"
                color: 1,1,0,1
                font_size: "22sp"
                halign: "left"
                valign: "middle"
                text_size: self.size
            BoxLayout:
                size_hint_x: None
                width: 420
                spacing: 8
                Button:
                    text: "Visualizer"
                    on_release: root.open_visualizer()
                Button:
                    text: "Export JSON"
                    on_release: root.on_export_json()
                Button:
                    text: "Export CSV"
                    on_release: root.on_export_csv()

        # Table header
        BoxLayout:
            size_hint_y: None
            height: 34
            canvas.before:
                Color:
                    rgba: 0.20,0.20,0.20,1
                Rectangle:
                    pos: self.pos
                    size: self.size
                Color:
                    rgba: 0.35,0.35,0.35,1
                Line:
                    rectangle: (self.x, self.y, self.width, self.height)

            # header checkbox (select all)
            BoxLayout:
                size_hint_x: None
                width: 40
                canvas.before:
                    Color:
                        rgba: 0.35,0.35,0.35,1
                    Line:
                        rectangle: (self.x, self.y, self.width, self.height)
                CheckBox:
```

```
                    id: app_cb
                    size_hint: None, None
                    size: 22, 22
                    pos_hint: {"center_y": .5}
                    on_active: root.header_toggle(self.active)

            Label:
                text: "Project"
                size_hint_x: None
                width: 220
                bold: True
                halign: "left"
                valign: "middle"
                text_size: self.size
                canvas.before:
                    Color:
                        rgba: 0.35,0.35,0.35,1
                    Line:
                        rectangle: (self.x, self.y, self.width, self.height)

            Label:
                text: "File"
                bold: True
                halign: "left"
                valign: "middle"
                text_size: self.size
                canvas.before:
                    Color:
                        rgba: 0.35,0.35,0.35,1
                    Line:
                        rectangle: (self.x, self.y, self.width, self.height)

            Label:
                text: "Items"
                size_hint_x: None
                width: 90
                bold: True
                halign: "right"
                valign: "middle"
                text_size: self.size
                canvas.before:
                    Color:
                        rgba: 0.35,0.35,0.35,1
                    Line:
                        rectangle: (self.x, self.y, self.width, self.height)

            Label:
                text: "Created"
                size_hint_x: None
                width: 130
                bold: True
                halign: "right"
                valign: "middle"
                text_size: self.size
                canvas.before:
```

```
                    Color:
                        rgba: 0.35,0.35,0.35,1
                    Line:
                        rectangle: (self.x, self.y, self.width, self.height)

        # Table body
        ScrollView:
            do_scroll_x: False
            GridLayout:
                id: vocab_rows
                cols: 1
                size_hint_y: None
                height: self.minimum_height
                spacing: 0
```

A.3 Application Note

The two files above implement the interactive prototype described in Chapter 4. The system demonstrates:

- data handling for user-created projects and files,
- a simulated NLP pipeline producing Recursive Linguistic Elements (RLE),
- export of bilingual vocabulary data,
- and a functional interface for the **Linguistic Visualizer**.

Although simplified, this prototype reflects the modular architecture planned for the full-scale ELA system, ensuring separation of processing, visualization, and user interaction layers.

Appendix B

Prototype Interface Screens

The graphical user interface (GUI) of the **English Language Assistant (ELA)** prototype plays a key pedagogical and technical role. It connects the linguistic processing pipeline (ASR, translation, RLE generation) with learner interaction. Each screen is designed to make the analytical process transparent — showing how authentic media are transformed into structured bilingual learning materials.

The prototype follows ELA's research principles: learner autonomy, minimal cognitive load, and progressive disclosure. Every interface element provides insight into the underlying data without requiring technical knowledge. The following subsections illustrate this interface layer.

B.1 Main Projects Window

The **Projects** screen is the entry point to the system. It presents all learner-created projects, providing a visual summary of activity and progress. From here users can open projects, inspect their files, or create new ones. This supports autonomy and goal-oriented learning.

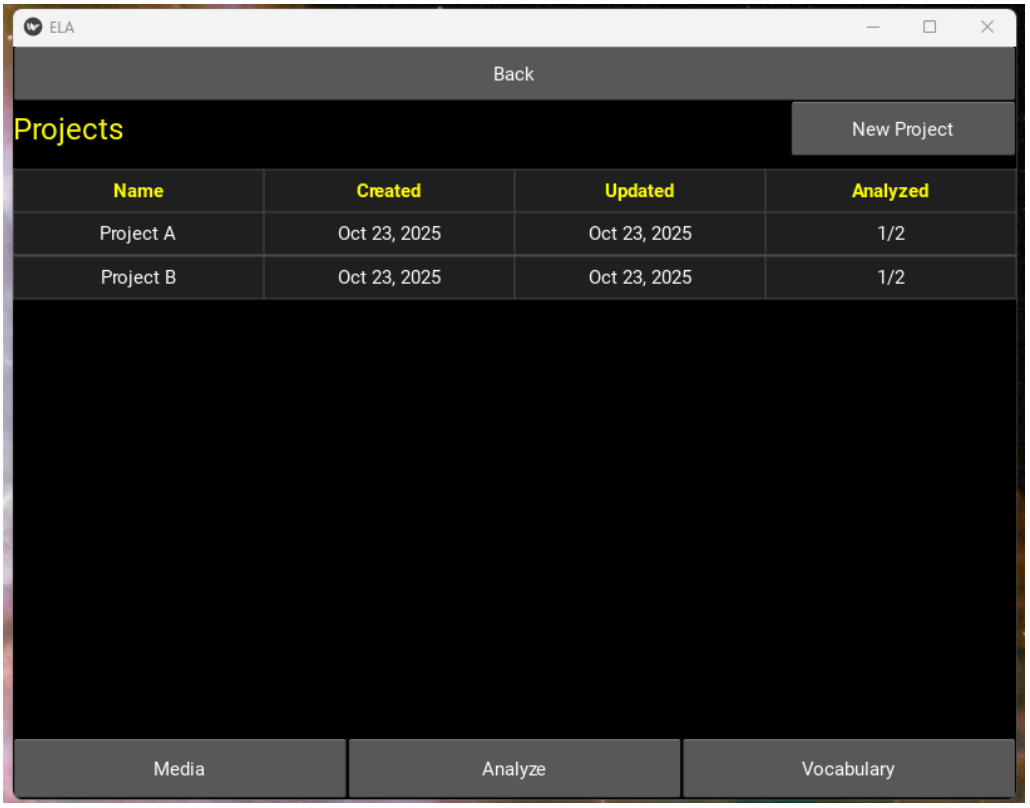

FIGURE B.1: Main **Projects** screen showing available projects with creation dates, last updates, and analyzed file counts. This view provides a quick overview of progress across learner-created media collections.

B.2 Project – File Management

The **Files** screen allows detailed control of each project's contents. Learners can review every uploaded file (audio, video, or text) and its processing parameters. The screen also provides access to detailed analysis results, reinforcing ELA's transparency principle.

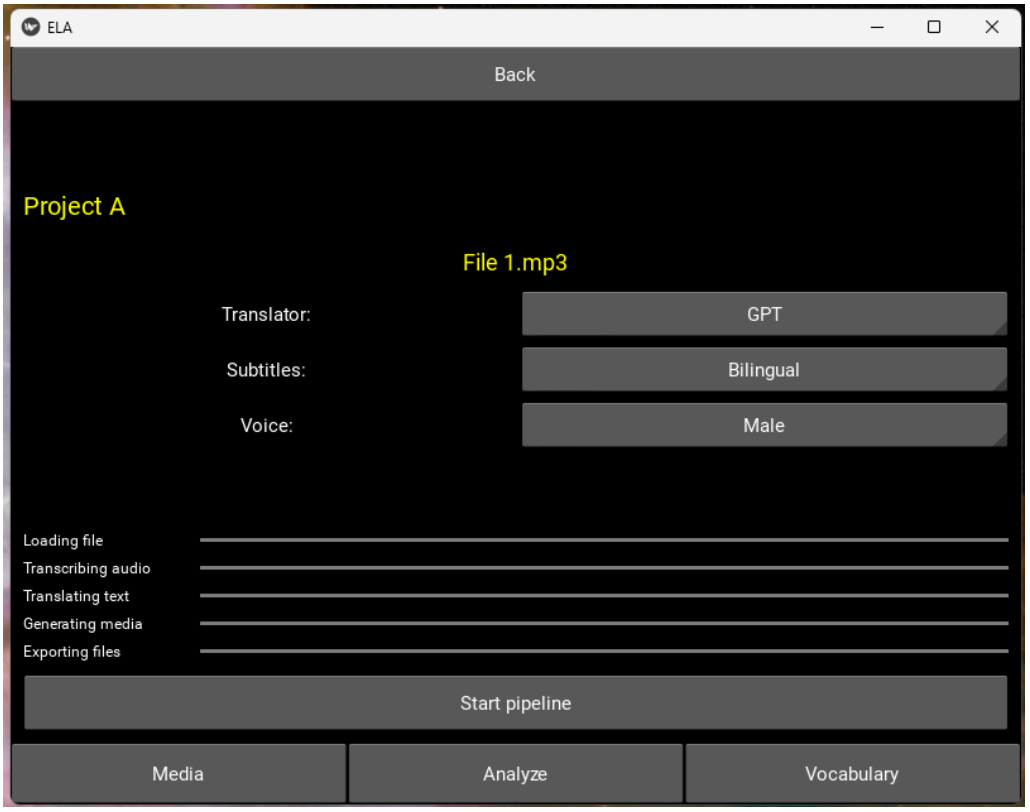

FIGURE B.2: The **Files** screen lists all media files attached to a selected project, including translator, subtitle, and voice parameters. It provides access to analysis details and file-level editing.

B.3 New Project Creation

The **New Project** dialog offers a simple workflow for creating new datasets. Each project acts as a corpus for personalized study materials, keeping all related media and metadata organized. This reinforces learner ownership of content.

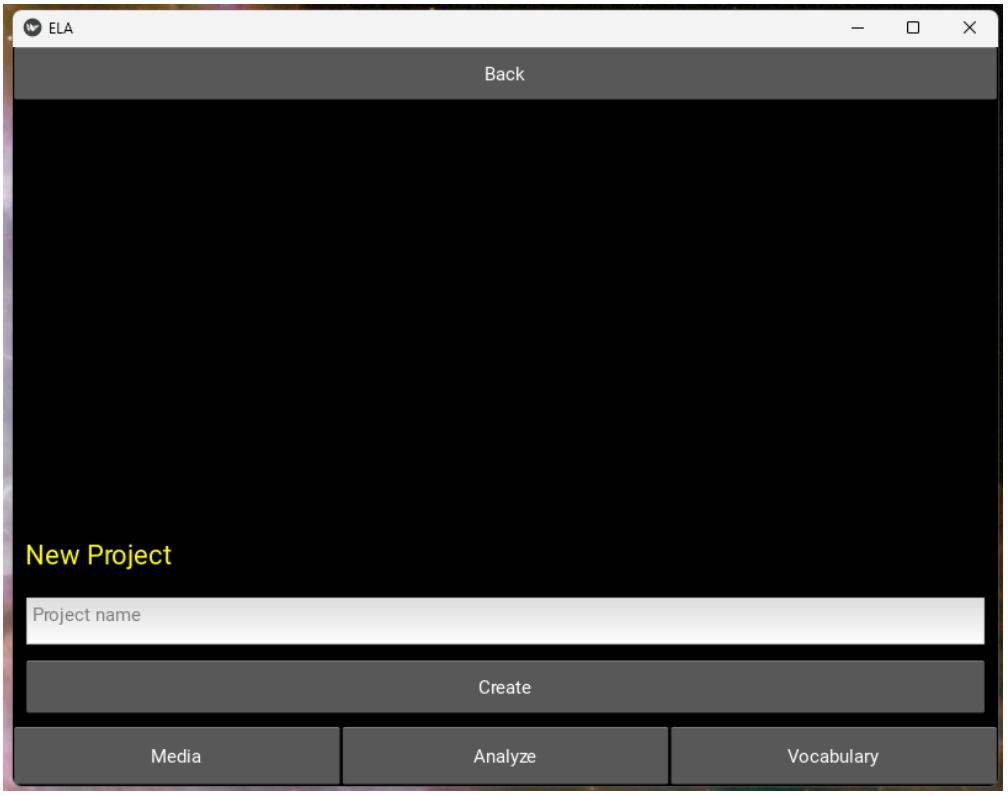

FIGURE B.3: The **New Project** dialog lets learners create a new media set. Each project serves as a self-contained corpus for personalized study materials.

B.4 Analysis Workflow

The **Analyze** interface manages the NLP pipeline — loading media, applying ASR, translation, and generating recursive linguistic elements. The progress bars provide clear feedback at each stage, helping users understand how the system processes data.

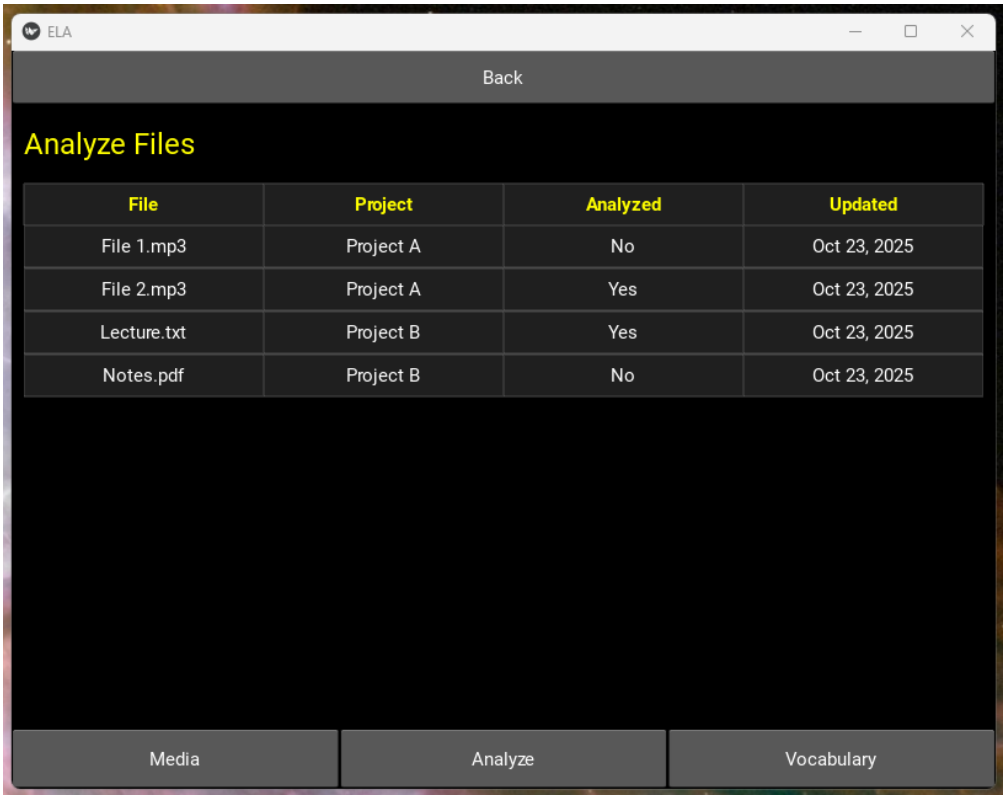

FIGURE B.4: **Analyze Files** aggregates all files across projects. It enables launching the full NLP pipeline (ASR, translation, and RLE generation) and tracking analysis status per file.

B.5 Vocabulary Management

The **Vocabulary** module lists all analyzed files and their extracted lexical items. Learners can filter, select, and export vocabulary in JSON or CSV formats, enabling external practice, review, or integration into other study tools.

This screen also allows users to refine their learning dataset down to individual files across multiple projects and combine them in custom configurations for subsequent export or visualization. In this way, the interface supports personalized curriculum building — a key principle of learner autonomy within ELA's architecture. The screen links directly to the **Linguistic Visualizer** for detailed structural inspection of selected materials.

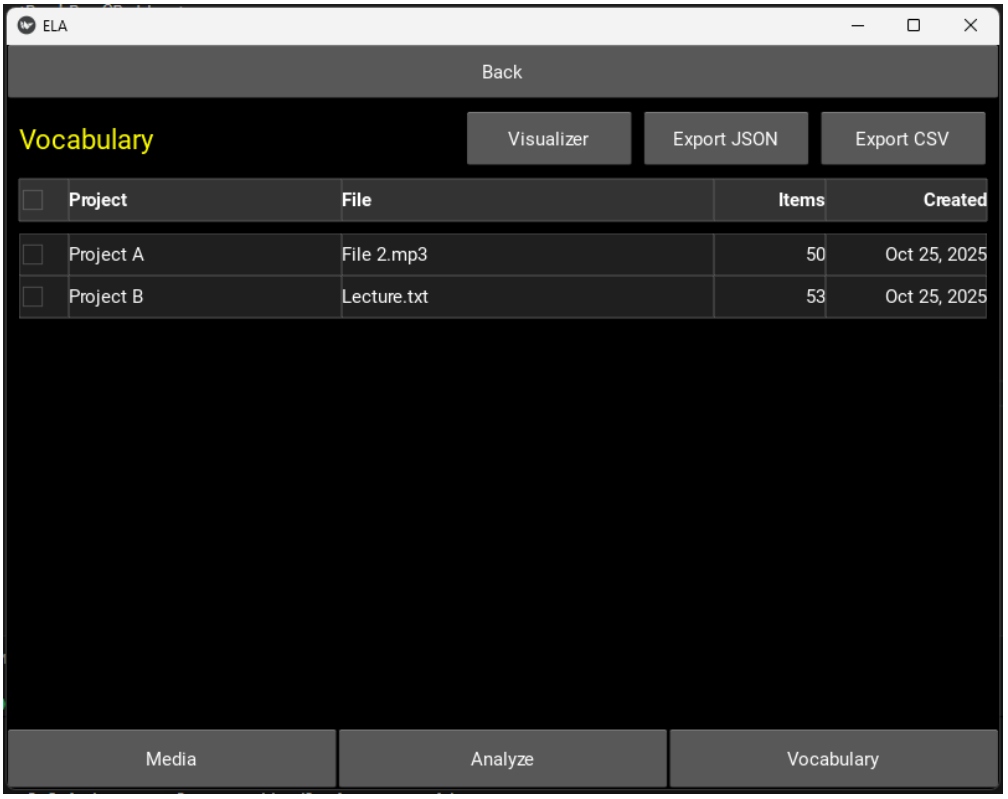

FIGURE B.5: The **Vocabulary** screen lists analyzed files and their extracted lexical items. Users can filter, combine, and export bilingual vocabulary datasets in JSON or CSV format, or open them directly in the Linguistic Visualizer for deeper exploration.

B.6 Linguistic Visualizer

The **Linguistic Visualizer** transforms RLE data into an interactive grammar tree. It displays hierarchical structures (sentence → phrase → word), CEFR levels, phonetics, and translations. This visualisation supports comprehension-first learning and structural awareness.

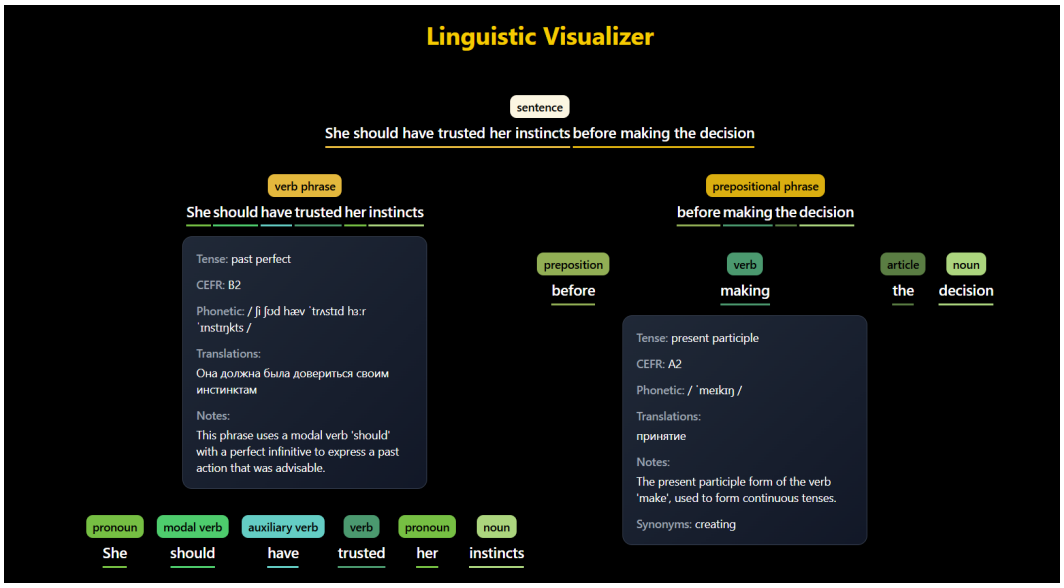

FIGURE B.6: The **Linguistic Visualizer** presents the recursive RLE structure for analyzed sentences. It displays grammatical hierarchies with CEFR level, phonetic transcription, and translation for each linguistic element, supporting comprehension via structural awareness.

Appendix C

Sample RLE JSON Object

This appendix presents a real JSON instance generated by the **ELA** prototype. The file defines a complete *Recursive Linguistic Elements (RLE)* structure for a single analyzed sentence, including its grammatical hierarchy, CEFR level, translations, and phonetic notes. This format serves as the core interoperability layer between all system components — the pipeline, storage, export, and the *Linguistic Visualizer*.

The JSON data below is taken directly from the project's `Data/sample.json` file.

Python Source

```
{
  "She should have trusted her instincts before making the decision.": {
    "type": "Sentence",
    "content": "She should have trusted her instincts before making the decision.",
    "tense": "past perfect",
    "linguistic_notes": [
      "This sentence uses a modal verb 'should' followed by a perfect infinitive 'have
      ↪  trusted' to express a past recommendation or regret."
    ],
    "part_of_speech": "sentence",
    "translations": [
      "Она должна была довериться своим инстинктам, прежде чем принять решение."
    ],
    "cefr_level": "B2",
    "phonetic": {
      "uk": "ʃi ʃʊd hæv 'trʌstɪd hɜ:r 'ɪnstɪŋkts bɪ'fɔ: 'meɪkɪŋ ðə dɪ'sɪʒən",
      "us": "ʃi ʃʊd hæv 'trʌstɪd hɜr 'ɪnstɪŋkts bɪ'fɔr 'meɪkɪŋ ðə dɪ'sɪʒən"
    },
    "definitions": [
      {
        "sense": "A sentence expressing a past recommendation or regret.",
```

```
        "examples": [
          "She should have trusted her instincts before making the decision."
        ]
      }
    ],
    "synonyms": [],
    "linguistic elements": [
      {
        "type": "Phrase",
        "content": "She should have trusted her instincts",
        "tense": "past perfect",
        "linguistic_notes": [
          "This phrase uses a modal verb 'should' with a perfect infinitive to express a past action that was advisable."
        ],
        "part_of_speech": "verb phrase",
        "translations": [
          "Она должна была довериться своим инстинктам"
        ],
        "cefr_level": "B2",
        "phonetic": {
          "uk": "ʃi ʃʊd hæv 'trʌstɪd hɜːr 'ɪnstɪŋkts",
          "us": "ʃi ʃʊd hæv 'trʌstɪd hɜr 'ɪnstɪŋkts"
        },
        "definitions": [
          {
            "sense": "To express a past action that was advisable.",
            "examples": [
              "She should have trusted her instincts."
            ]
          }
        ],
        "synonyms": [],
        "linguistic elements": [
          {
            "type": "Word",
            "content": "She",
            "tense": "null",
            "linguistic_notes": [
              "A pronoun used to refer to a female person or animal previously mentioned or easily identified."
            ],
            "part_of_speech": "pronoun",
            "translations": [
              "она"
            ],
            "cefr_level": "A1",
```

```
            "phonetic": {
              "uk": "ʃiː",
              "us": "ʃi"
            },
            "definitions": [
              {
                "sense": "Used to refer to a female person or animal.",
                "examples": [
                  "She is going to the store."
                ]
              }
            ],
            "synonyms": [
              "her"
            ],
            "linguistic elements": []
          },
          {
            "type": "Word",
            "content": "should",
            "tense": "null",
            "linguistic_notes": [
              "A modal verb used to indicate obligation, duty, or correctness, typically when criticizing someone's actions."
            ],
            "part_of_speech": "modal verb",
            "translations": [
              "должна"
            ],
            "cefr_level": "B1",
            "phonetic": {
              "uk": "ʃʊd",
              "us": "ʃʊd"
            },
            "definitions": [
              {
                "sense": "Used to indicate obligation or duty.",
                "examples": [
                  "You should see a doctor."
                ]
              }
            ],
            "synonyms": [
              "ought to"
            ],
            "linguistic elements": []
          },
```

```
        {
          "type": "Word",
          "content": "have",
          "tense": "null",
          "linguistic_notes": [
            "An auxiliary verb used with past participles to form perfect tenses."
          ],
          "part_of_speech": "auxiliary verb",
          "translations": [
            "иметь"
          ],
          "cefr_level": "A2",
          "phonetic": {
            "uk": "hæv",
            "us": "hæv"
          },
          "definitions": [
            {
              "sense": "Used with past participles to form perfect tenses.",
              "examples": [
                "I have seen that movie."
              ]
            }
          ],
          "synonyms": [],
          "linguistic elements": []
        },
        {
          "type": "Word",
          "content": "trusted",
          "tense": "past participle",
          "linguistic_notes": [
            "The past participle form of the verb 'trust', used to express a completed
            ↪ action."
          ],
          "part_of_speech": "verb",
          "translations": [
            "доверилась"
          ],
          "cefr_level": "B1",
          "phonetic": {
            "uk": "ˈtrʌstɪd",
            "us": "ˈtrʌstɪd"
          },
          "definitions": [
            {
```

```
            "sense": "To believe in the reliability, truth, or ability of someone or
            ↪  something.",
            "examples": [
              "She trusted him completely."
            ]
          }
        ],
        "synonyms": [
          "believed"
        ],
        "linguistic elements": []
      },
      {
        "type": "Word",
        "content": "her",
        "tense": "null",
        "linguistic_notes": [
          "A pronoun used to refer to a female person or animal previously mentioned or
          ↪  easily identified."
        ],
        "part_of_speech": "pronoun",
        "translations": [
          "ee"
        ],
        "cefr_level": "A1",
        "phonetic": {
          "uk": "hɜːr",
          "us": "hɜr"
        },
        "definitions": [
          {
            "sense": "Used to refer to a female person or animal.",
            "examples": [
              "I saw her at the park."
            ]
          }
        ],
        "synonyms": [
          "she"
        ],
        "linguistic elements": []
      },
      {
        "type": "Word",
        "content": "instincts",
        "tense": "null",
        "linguistic_notes": [
```

```
            "A noun referring to an innate, typically fixed pattern of behavior in animals in response to certain stimuli."
          ],
          "part_of_speech": "noun",
          "translations": [
            "инстинкты"
          ],
          "cefr_level": "B2",
          "phonetic": {
            "uk": "ˈɪnstɪŋkts",
            "us": "ˈɪnstɪŋkts"
          },
          "definitions": [
            {
              "sense": "An innate, typically fixed pattern of behavior in animals in response to certain stimuli.",
              "examples": [
                "Birds have an instinct to build nests."
              ]
            }
          ],
          "synonyms": [
            "intuition"
          ],
          "linguistic elements": []
        }
      ]
    },
    {
      "type": "Phrase",
      "content": "before making the decision",
      "tense": "null",
      "linguistic_notes": [
        "This phrase indicates the time frame in which the action should have been completed."
      ],
      "part_of_speech": "prepositional phrase",
      "translations": [
        "прежде чем принять решение"
      ],
      "cefr_level": "B1",
      "phonetic": {
        "uk": "bɪˈfɔː ˈmeɪkɪŋ ðə dɪˈsɪʒən",
        "us": "bɪˈfɔr ˈmeɪkɪŋ ðə dɪˈsɪʒən"
      },
      "definitions": [
        {
```

```
            "sense": "Indicates the time frame in which the action should have been
            ↪  completed.",
            "examples": [
              "Think carefully before making the decision."
            ]
          }
        ],
        "synonyms": [],
        "linguistic elements": [
          {
            "type": "Word",
            "content": "before",
            "tense": "null",
            "linguistic_notes": [
              "A preposition used to indicate the time or event preceding another."
            ],
            "part_of_speech": "preposition",
            "translations": [
              "прежде чем"
            ],
            "cefr_level": "A2",
            "phonetic": {
              "uk": "bɪ'fɔː",
              "us": "bɪ'fɔr"
            },
            "definitions": [
              {
                "sense": "Indicating the time or event preceding another.",
                "examples": [
                  "Finish your homework before dinner."
                ]
              }
            ],
            "synonyms": [
              "prior to"
            ],
            "linguistic elements": []
          },
          {
            "type": "Word",
            "content": "making",
            "tense": "present participle",
            "linguistic_notes": [
              "The present participle form of the verb 'make', used to form continuous
              ↪  tenses."
            ],
            "part_of_speech": "verb",
```

```
            "translations": [
              "принятие"
            ],
            "cefr_level": "A2",
            "phonetic": {
              "uk": "ˈmeɪkɪŋ",
              "us": "ˈmeɪkɪŋ"
            },
            "definitions": [
              {
                "sense": "The process of forming or producing something.",
                "examples": [
                  "She is making a cake."
                ]
              }
            ],
            "synonyms": [
              "creating"
            ],
            "linguistic elements": []
          },
          {
            "type": "Word",
            "content": "the",
            "tense": "null",
            "linguistic_notes": [
              "A definite article used to specify a particular noun that is known to the
              ↪  reader or listener."
            ],
            "part_of_speech": "article",
            "translations": [
              "это"
            ],
            "cefr_level": "A1",
            "phonetic": {
              "uk": "ðə",
              "us": "ðə"
            },
            "definitions": [
              {
                "sense": "Used to specify a particular noun.",
                "examples": [
                  "The cat is on the mat."
                ]
              }
            ],
            "synonyms": [],
```

```
            "linguistic elements": []
          },
          {
            "type": "Word",
            "content": "decision",
            "tense": "null",
            "linguistic_notes": [
              "A noun referring to a conclusion or resolution reached after consideration."
            ],
            "part_of_speech": "noun",
            "translations": [
              "решение"
            ],
            "cefr_level": "B1",
            "phonetic": {
              "uk": "dɪˈsɪʒən",
              "us": "dɪˈsɪʒən"
            },
            "definitions": [
              {
                "sense": "A conclusion or resolution reached after consideration.",
                "examples": [
                  "He made a decision to leave."
                ]
              }
            ],
            "synonyms": [
              "choice"
            ],
            "linguistic elements": []
          }
        ]
      }
    ]
  }
}
```


This human-readable structure forms the foundation of the ELA framework. It enables transparent mapping between AI-based analysis and user-facing visualization, allowing both learners and researchers to trace every linguistic decision through the full processing pipeline.